

[OBJ]
[OBJ]
[OBJ]
[OBJ]

Parte I.

Plan de Proyecto

1 Estado del Arte y Contextualización	3
1.1 La neumonía, un problema de salud global	4
1.2 La radiografía de tórax como imagen digital	6
1.2.1 ¿Qué es una radiografía de tórax y por qué es la prueba de referencia?	6
1.2.2 ¿Cómo ve el ordenador una radiografía?	6
1.2.3 Conjuntos de datos utilizados	7
1.3 Inteligencia Artificial, aprendizaje profundo y visión artificial	8
1.3.1 De la IA al aprendizaje profundo	8
1.3.2 La visión artificial	9
1.3.3 CheXNet y el estudio de Kermany	9
1.4 Las redes neuronales convolucionales (CNN)	10
1.4.1 Qué es una red neuronal convolucional	10
1.4.2 Las arquitecturas convolucionales del banco de pruebas	11
1.5 Los Transformers en visión artificial	14
1.5.1 El mecanismo de atención	14
1.5.2 De las palabras a los píxeles	15
1.5.3 Las arquitecturas Transformer del proyecto	16
1.6 La explicabilidad: inteligencia artificial explicable	17
1.6.1 El problema de la caja negra	17
1.6.2 Técnicas de explicabilidad	18
1.6.3 La evaluación cuantitativa	19
1.6.4 La calibración	19
1.7 Las barreras para la adopción clínica de la IA	21
1.8 Los asistentes conversacionales y los grandes modelos de lenguaje	23
1.8.1 ¿Qué son los grandes modelos de lenguaje?	23
1.8.2 El panorama de los modelos de lenguaje y la elección de Llama 3	24
1.9 Metodología experimental	25
1.9.1 Fase 1: Entrenamiento con validación cruzada	25
1.9.2 Fase 2: Comparación estadística entre modelos	29
1.9.3 Fase 3: Validación externa y test de DeLong	31
1.9.4 Fase 4: Análisis de explicabilidad	33
1.10 Tecnologías y arquitectura del sistema	36
2 Metas y Propósitos del Proyecto	36
3 Estructura organizativa y partes interesadas del proyecto	40
3.1 Estructura organizativa del proyecto	41
3.2 Partes interesadas del proyecto	43
3.3 Responsabilidades, funciones y participación de las partes interesadas	44
4 Marco Metodológico de Gestión	47
4.1 Justificación del enfoque metodológico	48
4.2 Roles y recursos del proyecto	49

4.2.1 Roles dentro del marco de trabajo	49
4.2.2 Dedicación y recursos materiales	49
4.3 Ciclo de trabajo iterativo	50
4.3.1 Los sprints y la cadencia del trabajo	50
4.3.2 Ceremonias y reuniones de seguimiento	50
4.3.3 Artefactos del proceso de gestión	51
4.4 Seguimiento visual del trabajo con Kanban	51
4.5 Alcance de la metodología	52
5 Cronograma y Plan de Trabajo	53
5.1 Marco general y objetivos de la planificación	53
5.2 Desglose de tareas por Sprint	54
5.2.1 Sprint 0 – Planificación del proyecto	56
5.2.2 Sprint 1 – Infraestructura y seguridad	58
5.2.3 Sprint 2 – Motor de diagnóstico	61
5.2.4 Sprint 3 – Explicabilidad	63
5.2.5 Sprint 4 – Entrenamiento CNN	66
5.2.6 Sprint 5 – Transformers y análisis XAI	69
5.2.7 Sprint 6 – Validación externa	72
5.2.8 Sprint 7 – Laboratorio MLOps	74
5.2.9 Sprint 8 – Documentación y cierre	77
5.3 Recursos y costes del proyecto	80
5.4 Reparto de responsabilidades	82
6 Identificación y Mitigación de Riesgos	83
6.1 Escala de valoración de riesgos	84
6.2 Catálogo de riesgos identificados	86
6.3 Plan de mitigación (medidas preventivas)	88
6.4 Plan de contingencia (medidas correctivas)	90
7 Planes de apoyo a la gestión del proyecto	91
7.1 Plan de Comunicación	92
7.2 Plan de Calidad	92
7.3 Plan de Gestión de la Configuración	93
8 Cuestiones abiertas y decisiones adoptadas	94
9 Consideraciones complementarias del proyecto	98

1 Estado del Arte y Contextualización

El presente documento constituye el Plan de Proyecto para el Trabajo Fin de Grado titulado "vitalXAI: Plataforma MLOps con inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax", realizado por el alumno Luis Carmona Berdugo y tutorizado por el profesor Aurelio López Fernández. En él se detallan los objetivos identificados para el proyecto, la organización y planificación de las tareas, los riesgos evaluados y gestionados, así como los planes de gestión auxiliares necesarios para una ejecución correcta del mismo.

Este primer capítulo tiene una doble misión. Por una parte, sitúa al lector en el problema que motiva el proyecto: la neumonía, su diagnóstico mediante radiografías de tórax y las limitaciones de los métodos actuales. Por otra, recorre el estado del arte de las tecnologías implicadas—inteligencia artificial, redes neuronales convolucionales, Transformers, inteligencia artificial explicable y asistentes conversacionales basados en grandes modelos de lenguaje—, explicando cada concepto desde sus fundamentos para que el texto resulte comprensible para cualquier persona, tenga o no formación técnica. El capítulo concluye con una descripción de la metodología experimental empleada y de las tecnologías sobre las que se construye el sistema.

1.1 *La neumonía, un problema de salud global*

Para comprender la motivación de este trabajo es necesario empezar por el principio: qué es exactamente la neumonía. La neumonía es una infección que afecta a los pulmones, más concretamente a los alvéolos, que son las pequeñas bolsas de aire situadas al final de las vías respiratorias en las que se produce el intercambio de gases, es decir, el proceso por el cual la sangre recoge el oxígeno que respiramos y elimina el dióxido de carbono. Cuando una persona sufre una neumonía, los alvéolos se llenan de líquido o de pus, lo que dificulta gravemente esa función y provoca que el oxígeno no llegue con normalidad a la sangre. Esta es la razón por la que los síntomas típicos de la enfermedad incluyen tos, fiebre, dificultad para respirar y dolor en el pecho (Torres & al., 2021).

La neumonía puede estar causada por diferentes microorganismos, siendo los más frecuentes las bacterias y los virus, aunque también existen neumonías producidas por hongos. La causa concreta condiciona el tratamiento: las neumonías bacterianas se tratan habitualmente con antibióticos, mientras que las virales no responden a estos medicamentos y requieren, en general, cuidados de soporte. Distinguir el tipo de neumonía no siempre es sencillo, y las radiografías de tórax, junto con los síntomas y las pruebas de laboratorio, son herramientas clave para orientar el diagnóstico.

La relevancia clínica de la neumonía es enorme. Según la Organización Mundial de la Salud, la neumonía fue responsable de aproximadamente 2,5 millones de fallecimientos en el año 2019, lo que la convierte en la principal causa de muerte por infección en todo el mundo (World Health Organization, 2024). La enfermedad afecta de manera desproporcionada a los grupos más vulnerables de la población: los niños menores de cinco años y los adultos mayores de sesenta y cinco. En los países de ingresos bajos y medios, donde el acceso a la atención sanitaria es más limitado, la neumonía infantil es una de las primeras causas de mortalidad.

La detección temprana y el diagnóstico correcto son, por tanto, cuestiones de vida o muerte. Un diagnóstico tardío retrasa el inicio del tratamiento adecuado y puede derivar en complicaciones graves. Ahora bien, diagnosticar neumonía no siempre es una tarea trivial, ni siquiera para especialistas con mucha experiencia. Los estudios muestran una variabilidad significativa en la interpretación de las radiografías de tórax entre distintos observadores: dos radiólogos pueden no coincidir sobre si una imagen muestra o no signos de neumonía, y esta variabilidad puede conducir a diagnósticos erróneos o retrasados con consecuencias clínicas importantes (Hopstaken & al., 2004). Esta es precisamente una de las grietas sobre las que se asienta la motivación de este proyecto: la posibilidad de construir herramientas informáticas que ayuden a los profesionales sanitarios a tomar decisiones mejor fundamentadas.

1.2 La radiografía de tórax como imagen digital

1.2.1 ¿Qué es una radiografía de tórax y por qué es la prueba de referencia?

La radiografía de tórax es una prueba de imagen que utiliza rayos X para obtener una representación del interior del tórax. Los rayos X son una forma de radiación electromagnética de alta energía que atraviesa los tejidos del cuerpo en mayor o menor medida según su densidad. Los huesos, que son muy densos, absorben la mayor parte de la radiación y aparecen en la imagen de color blanco; los tejidos blandos, como el músculo o el corazón, la absorben en menor grado y se muestran en tonos grises; y las zonas llenas de aire, como los pulmones sanos, apenas la absorben y aparecen prácticamente en negro (Gonzalez & Woods, 2008).

Esta propiedad es la que permite al radiólogo detectar una neumonía: cuando los alvéolos se llenan de líquido o de pus, la zona afectada del pulmón deja de ser transparente a los rayos X y se muestra como una mancha más blanca y densa de lo normal, lo que se conoce como opacidad o infiltrado (Franquet, 2001). La radiografía de tórax es la técnica de imagen de primera línea para el diagnóstico de la neumonía porque combina tres ventajas fundamentales: es muy accesible (existe en prácticamente todos los centros sanitarios), tiene un coste bajo en comparación con otras técnicas como el TAC, y es muy rápida de adquirir. Por estos motivos, sigue siendo la herramienta estándar que se utiliza en primer lugar ante la sospecha de neumonía (Mandell & al., 2007).

1.2.2 ¿Cómo ve el ordenador una radiografía?

Una comprensión importante para el resto del capítulo es cómo se representa una radiografía en términos informáticos, porque es la información con la que trabajarán los algoritmos. Una imagen digital no es más que una cuadrícula bidimensional de puntos llamados píxeles. En el caso de una radiografía en escala de grises, cada píxel guarda un único número que indica el nivel de brillo en ese punto, que normalmente va de 0 (negro puro, es decir, zona muy transparente a los rayos X) a 255 (blanco puro, zona muy densa). Así, una radiografía de, por ejemplo, 512 píxeles de ancho por 512 de alto no es, para un ordenador, una "foto de un tórax", sino una tabla de $512 \times 512 = 262.144$ números (Szeliski, 2010).

Este punto es crucial para entender todo lo que viene después: los algoritmos de visión artificial no "ven" imágenes como los humanos, sino que trabajan con grandes cantidades de números organizados en estructuras matemáticas llamadas tensores. Las redes neuronales profundas aprenden a extraer patrones a partir de esos números, como se explicará en las siguientes secciones.

1.2.3 Conjuntos de datos utilizados

Para entrenar y evaluar los modelos de inteligencia artificial es imprescindible disponer de un conjunto de imágenes etiquetadas, es decir, imágenes de las que se sabe con certeza si corresponden o no a un paciente con neumonía. En este proyecto se emplean dos conjuntos de datos públicos y anonimizados, siguiendo la metodología establecida en el estudio de referencia (Kermany & al., 2018).

El conjunto de datos de entrenamiento es el publicado por Kermany y colaboradores, procedente del Guangzhou Women and Children's Medical Center y disponible en la plataforma Kaggle. Este dataset contiene 5.856 radiografías de tórax pediátricas (de pacientes niños) en formato JPEG, clasificadas en dos categorías: NORMAL, con 1.583 imágenes, y PNEUMONIA, con 4.273 imágenes. La categoría PNEUMONIA se subdivide a su vez en neumonía bacteriana y neumonía viral. El hecho de que el dataset contenga muchas más imágenes de neumonía que de pacientes sanos tiene un nombre técnico, desbalanceo de clases, y constituye un desafío para el entrenamiento de los modelos que se abordará más adelante en la metodología. Este conjunto se selecciona por ser el mismo utilizado en el estudio de referencia, lo que permite establecer una comparación directa con los resultados publicados por Kermany y colaboradores.

El segundo conjunto de datos se emplea para la validación externa, un concepto que se desarrollará en profundidad más adelante y que, de forma resumida, consiste en comprobar que los modelos funcionan también con imágenes que nunca han visto y que proceden de un entorno distinto al del entrenamiento. Este segundo conjunto es el COVID-19 Radiography Database, desarrollado por investigadores de la Universidad de Qatar y la Universidad de Dhaka (Chowdhury & al., 2020; Rahman & al., 2021). Se trata de radiografías de tórax de pacientes adultos, adquiridas con equipos y protocolos diferentes a los del dataset pediátrico de entrenamiento. De este conjunto se extraen quinientas imágenes de pacientes sanos y quinientas de pacientes con neumonía viral, garantizando así un balance perfecto entre las dos clases. La población adulta, el equipo de adquisición y los protocolos de imagen diferentes convierten esta prueba en un escenario potencialmente exigente para evaluar la capacidad de generalización de los modelos. No obstante, el conjunto combina imágenes normales y patológicas procedentes de fuentes y repositorios distintos, por lo que existe el riesgo de que un modelo aprenda señales relacionadas con el origen de la imagen en lugar de la patología (DeGrave, Janizek, & Lee, 2021).

Es importante señalar una cuestión de honestidad metodológica: durante la fase de desarrollo y depuración del código, y para poder ejecutar pruebas de forma ágil y con tiempos de computación razonables, se han utilizado subconjuntos reducidos de ambos conjuntos de datos. Estos subconjuntos permiten validar que el código funciona correctamente sin necesidad de procesar las miles de imágenes completas en cada iteración de desarrollo. Sin embargo, la metodología del proyecto está concebida y configurada para trabajar con los conjuntos de datos completos originales: las 5.856 imágenes de Kermany para el entrenamiento y las mil imágenes (500 normales y 500 con neumonía) del COVID-19 Radiography Database para la validación externa.

1.3 *Inteligencia Artificial, aprendizaje profundo y visión artificial*

1.3.1 De la IA al aprendizaje profundo

El término inteligencia artificial (IA) engloba todas las técnicas que pretenden dotar a las máquinas de capacidades consideradas propias de la inteligencia humana, como el razonamiento, el aprendizaje o el reconocimiento de patrones. Dentro de la IA existe una rama denominada aprendizaje automático (machine learning), cuyo principio es que el ordenador no es programado con reglas explícitas para resolver una tarea, sino que aprende esas reglas a partir de ejemplos (Bishop, 2006). En lugar de decirle a la máquina "una neumonía se ve como una mancha blanca con tal forma y tal posición", se le muestran miles de radiografías con su diagnóstico correcto y se le permite deducir por sí misma cuáles son los patrones que distinguen una imagen con neumonía de una sana.

El aprendizaje profundo (deep learning) es una subdisciplina del aprendizaje automático que se basa en redes neuronales artificiales con muchas capas (de ahí lo de "profundo"). Una red neuronal es un modelo matemático inspirado, de forma muy simplificada, en el funcionamiento del cerebro. Está formada por unidades de procesamiento llamadas neuronas, organizadas en capas. Cada neurona recibe varias entradas, las multiplica por unos valores numéricos llamados pesos (que representan la "importancia" de cada entrada), las suma y aplica una función matemática denominada función de activación que decide si la neurona se "activa" o no. La primera capa recibe los datos de entrada (por ejemplo, los píxeles de una radiografía), la última produce el resultado (por ejemplo, la probabilidad de que haya neumonía), y las capas intermedias, llamadas capas ocultas, transforman progresivamente la información (LeCun, Bengio, & Hinton, 2015).

El proceso por el cual la red aprende se denomina entrenamiento, y su idea es sorprendentemente sencilla aunque matemáticamente sofisticada (Goodfellow, Bengio, & Courville, 2016):

1. **Propagación hacia delante:** se introduce una imagen en la red y se calcula su salida con los pesos actuales.
2. **Cálculo del error:** se compara la salida de la red con la respuesta correcta mediante una función matemática denominada función de coste o pérdida (loss), que cuantifica cuánto se ha equivocado la red.
3. **Propagación hacia atrás:** se calcula cómo afectaría una pequeña variación de cada peso al error final, mediante un algoritmo llamado retropropagación del error.
4. **Actualización de los pesos:** se modifican ligeramente los pesos en la dirección que reduce el error, mediante un algoritmo de optimización como el descenso del gradiente.

Este ciclo se repite con todas las imágenes del conjunto de entrenamiento, y el resultado es que, progresivamente, la red va ajustando sus pesos hasta producir salidas cada vez más cercanas a las correctas. En el caso concreto de este proyecto, las redes se entrenan para resolver un problema de clasificación binaria: dada una radiografía, decidir si pertenece a la clase PNEUMONIA o a la clase NORMAL. La red no devuelve una etiqueta rígida, sino una probabilidad entre 0 y 1 (la salida de una neurona con activación sigmoide), que puede interpretarse como el nivel de confianza del modelo.

1.3.2 La visión artificial

La visión artificial (computer vision) es la rama de la inteligencia artificial que se ocupa de que las máquinas puedan interpretar el mundo visual: imágenes, vídeos, fotografías y, en el caso que nos ocupa, radiografías (Szeliski, 2010). La tarea concreta que aborda este proyecto es la clasificación de imágenes: asignar una categoría (en este caso, "neumonía" o "sano") a cada imagen de entrada.

Aplicar visión artificial a la radiografía de tórax es especialmente atractivo porque convierte un proceso que requiere años de formación médica en una tarea automatizable: dada una imagen, producir una decisión. Pero automatizar no significa simplificar; la interpretación de radiografías es sutil y los algoritmos deben ser capaces de captar diferencias muy finas en las texturas y densidades del pulmón.

1.3.3 CheXNet y el estudio de Kermany

Para contextualizar el estado del arte es útil repasar los dos hitos científicos que sustentan directamente este proyecto.

El primero es CheXNet, presentado por Rajpurkar y colaboradores en 2017 (Rajpurkar & al., 2017). CheXNet es una red neuronal convolucional de 121 capas (basada en la arquitectura DenseNet, que se explicará más adelante) entrenada sobre el conjunto ChestX-ray14, una base de datos que contiene más de 112.000 radiografías de tórax de unos 30.000 pacientes, con anotaciones para catorce enfermedades torácicas diferentes. El logro principal del estudio fue demostrar, de manera empírica, que una red neuronal podía igualar e incluso superar el rendimiento de un panel de radiólogos expertos en la detección de neumonía a partir de radiografías de tórax. Este resultado supuso un punto de inflexión porque mostró que la tecnología ya no era solo prometedora, sino que alcanzaba niveles de rendimiento comparables a los humanos en una tarea clínica real.

El segundo hito es el estudio de Kermany y colaboradores, publicado en la revista Cell en 2018 (Kermany & al., 2018), que constituye una referencia metodológica y de datos para este proyecto. El trabajo presenta un marco de aprendizaje profundo basado en transferencia de conocimiento y demuestra su aplicación a la identificación de neumonía pediátrica mediante radiografías de tórax, utilizando el conjunto de 5.856 imágenes descrito en la sección anterior. Su aportación no consiste en comparar sistemáticamente varias arquitecturas convolucionales, sino en mostrar cómo un sistema de aprendizaje profundo puede utilizarse para apoyar el diagnóstico por imagen y acompañar sus predicciones de elementos interpretables. La comparación con el rendimiento de expertos se refiere principalmente a las tareas de diagnóstico oftalmológico analizadas en el estudio, por lo que no debe trasladarse directamente a una afirmación de equivalencia entre el modelo de neumonía y los radiólogos. El artículo resulta relevante para este proyecto por el uso del dataset pediátrico, la transferencia de conocimiento y la preocupación por la interpretabilidad, pero no constituye un benchmark de cinco arquitecturas de neumonía.

El presente trabajo toma esas ideas como punto de partida y desarrolla una propuesta experimental propia en tres direcciones: compara diecinueve arquitecturas —incluyendo redes convolucionales y Vision Transformers— bajo un mismo pipeline, incorpora una evaluación cuantitativa de la explicabilidad y de la calibración de las predicciones, y envuelve toda la metodología en una plataforma web MLOps accesible para profesionales sanitarios sin formación técnica. Por tanto, la novedad no se formula como la ampliación de un supuesto banco de cinco modelos de Kermany, sino como la construcción de un entorno comparativo y operativo que aplica el dataset y la motivación metodológica del trabajo de referencia a una evaluación más amplia.

1.4 Las redes neuronales convolucionales (CNN)

1.4.1 Qué es una red neuronal convolucional

Las redes neuronales convolucionales (del inglés convolutional neural networks, CNN) son el tipo de arquitectura que ha dominado históricamente la visión artificial. Su nombre procede de la operación matemática central que realizan: la convolución.

Para entender qué es una convolución conviene partir de una idea intuitiva. La visión de una CNN se puede comparar con la de un detective con una lupa que recorre la imagen sistemáticamente. La red utiliza pequeños filtros o kernels, que no son más que cuadrículas de números (por ejemplo, de 3×3 o 5×5) que se deslizan por toda la imagen y detectan patrones locales. Cada filtro está especializado en buscar un patrón concreto: algunos detectan bordes verticales, otros horizontales, otros texturas determinadas. El resultado de aplicar un filtro sobre la imagen se denomina mapa de características o feature map, y la particularidad fundamental de la convolución es que el mismo filtro se aplica sobre toda la imagen, lo que se denomina compartición de parámetros. Esto reduce enormemente el número de parámetros del modelo y hace que la respuesta del filtro se desplace de forma coherente cuando también se desplaza el patrón detectado; esta propiedad se conoce como equivanianza a la traslación. Posteriormente, las capas de agrupación (pooling) reducen la resolución espacial y condensan la información de regiones próximas, favoreciendo una mayor invariancia local a pequeños desplazamientos (Krizhevsky, Sutskever, & Hinton, 2012).

Las CNN se estructuran en capas que extraen progresivamente características cada vez más abstractas. Las primeras capas detectan elementos sencillos como bordes y cambios de brillo; las capas intermedias combinan estos elementos en texturas y formas; y las capas finales integran el todo en conceptos de alto nivel relevantes para la tarea, en nuestro caso, la presencia o ausencia de las opacidades características de la neumonía. Entre las capas convolucionales se intercalan capas de agrupación (pooling) que reducen el tamaño de los mapas de características, condensando la información y ayudando a la red a ser más robusta y eficiente. Al final de la red, las características se aplanan y se conectan a una o varias capas densas (como las de cualquier red neuronal clásica) que producen la clasificación final.

En el contexto de la neumonía, lo que una CNN bien entrenada aprende a hacer es identificar en la radiografía las regiones de densidad anómala en los campos pulmonares que los radiólogos asocian con la infección, sin que se le haya indicado explícitamente qué aspecto tiene una opacidad. Aprende por sí misma a partir de los ejemplos.

1.4.2 Las arquitecturas convolucionales del banco de pruebas

Este proyecto evalúa dieciséis arquitecturas convolucionales. Para no abrumar al lector con una descripción técnica exhaustiva de cada una, se agrupan a continuación por familias, explicando la idea esencial de cada familia y la particularidad de cada modelo, siempre en el contexto de la detección de neumonía. Todas ellas están preentrenadas sobre el conjunto de imágenes naturales ImageNet, lo que se explicará con más detalle en la sección de metodología.

Familia ResNet (He, Zhang, Ren, & Sun, 2016): el problema que ResNet vino a resolver es el de la degradación de las redes muy profundas. Cuando una red tiene decenas o cientos de capas, entrenarla de forma directa se vuelve extremadamente difícil, y el rendimiento, lejos de mejorar, empeora. La solución propuesta son las conexiones residuales o saltos: cada bloque de la red aprende no una transformación completa, sino la diferencia (el residuo) entre su entrada y su salida, sumando la entrada original. Esta idea, aparentemente sencilla, permitió entrenar redes de cientos de capas y es una de las arquitecturas más influyentes de la década. En este proyecto se evalúan:

- ResNet50: la versión de 50 capas, la más popular y equilibrada de la familia.
- ResNet101: versión más profunda (101 capas), que puede captar patrones más complejos a costa de mayor coste computacional.
- ResNet152: la más profunda de la serie (152 capas), con mayor capacidad pero también mayor demanda de recursos.
- ResNet50V2: una revisión de ResNet50 que reorganiza el orden de las operaciones dentro de cada bloque (normalizando antes de la convolución), lo que mejora el flujo de gradientes y, en general, la convergencia del entrenamiento (He, Zhang, Ren, & Sun, 2016).

Familia DenseNet (Huang, Liu, Van Der Maaten, & Weinberger, 2017): DenseNet lleva la idea de las conexiones a un extremo: en lugar de saltos puntuales, cada capa recibe como entrada las características de todas las capas anteriores, concatenándolas. Esto se denomina conectividad densa y favorece la reutilización de las características, el flujo de información y la regularización natural. DenseNet es especialmente relevante para este proyecto porque CheXNet, el sistema que igualó a los radiólogos en la detección de neumonía, se basaba precisamente en esta arquitectura (Rajpurkar & al., 2017). Se evalúan:

- DenseNet121: la versión de 121 capas, la más ligera y utilizada de la familia.
- DenseNet201: la versión más profunda (201 capas), con mayor capacidad de representación.

Familia EfficientNet (Tan & Le, 2019): EfficientNet plantea la pregunta de cómo escalar una red de forma óptima. En lugar de decidir a mano cuánto agrandar la profundidad, la anchura o la resolución de la imagen, propone un método de escalado compuesto que equilibra las tres dimensiones mediante un coeficiente único. El resultado es una familia de modelos que ofrece una excelente relación entre precisión y coste computacional. Se evalúan:

- EfficientNetB0: el modelo base de la familia, el más ligero.
- EfficientNetB3 y EfficientNetB7: versiones escaladas con mayor capacidad.
- EfficientNetV2S: la segunda generación de EfficientNet, que mejora la eficiencia del entrenamiento introduciendo capas de entrenamiento progresivo y ajustes en las operaciones (Tan & Le, 2021).

Familia MobileNet (Howard & al., 2017): MobileNet está diseñada para funcionar en dispositivos con recursos limitados, como teléfonos móviles. Su idea clave es la convolución separable por profundidad, que descompone una convolución estándar en dos operaciones más baratas, reduciendo drásticamente el número de parámetros y de operaciones sin sacrificar demasiada precisión. Se evalúan:

- MobileNetV2: segunda generación, que introduce los bloques residuales invertidos (Sandler & al., 2018).
- MobileNetV3Large: tercera generación, optimizada además con técnicas de búsqueda de arquitecturas (Howard & al., 2019).

VGG16 (Simonyan & Zisserman, 2015): VGG demostró que una arquitectura muy simple —una pila uniforme de capas convolucionales de 3×3 seguidas de capas de agrupación— podía alcanzar resultados excelentes si era lo suficientemente profunda. Su simplicidad la convierte en un modelo de referencia ideal para comparar arquitecturas más sofisticadas.

InceptionV3 (Szegedy & al., 2016): la idea de Inception es aplicar varios filtros de distinto tamaño (1×1 , 3×3 y 5×5) en paralelo sobre la misma región, de modo que la red pueda captar patrones a diferentes escalas simultáneamente. InceptionV3 refina esta idea con convoluciones factorizadas que reducen el coste computacional.

Xception (Chollet, 2017): Xception lleva la idea de Inception al extremo y se basa exclusivamente en convoluciones separables por profundidad, demostrando que esta operación, combinada de la forma adecuada, puede superar a las arquitecturas clásicas de Inception. Es una arquitectura especialmente eficiente en términos de parámetros.

ConvNeXtTiny (Liu & al., 2022): ConvNeXt es una relectura moderna de las CNN: tomando como base ResNet, incorpora de forma sistemática las ideas que hicieron exitosos a los Transformers (normalizaciones, funciones de activación modernas, menor número de bloques de normalización), demostrando que las CNN "modernizadas" pueden competir con los Transformers en igualdad de condiciones. ConvNeXtTiny es la versión pequeña de esta familia.

La selección de estas dieciséis arquitecturas no es casual: cubre un espectro muy amplio del diseño de redes convolucionales (desde modelos ligeros y eficientes como MobileNetV2 hasta modelos de gran capacidad como ConvNeXt-Tiny), y todas ellas han demostrado un rendimiento destacado en clasificación de imágenes, lo que permite realizar una comparativa rica y representativa sobre la tarea de la detección de neumonía (Zhang & al., 2024).

RED NEURONAL CONVOLUCIONAL (CNN) PARA RADIOGRAFÍAS DE TÓRAX

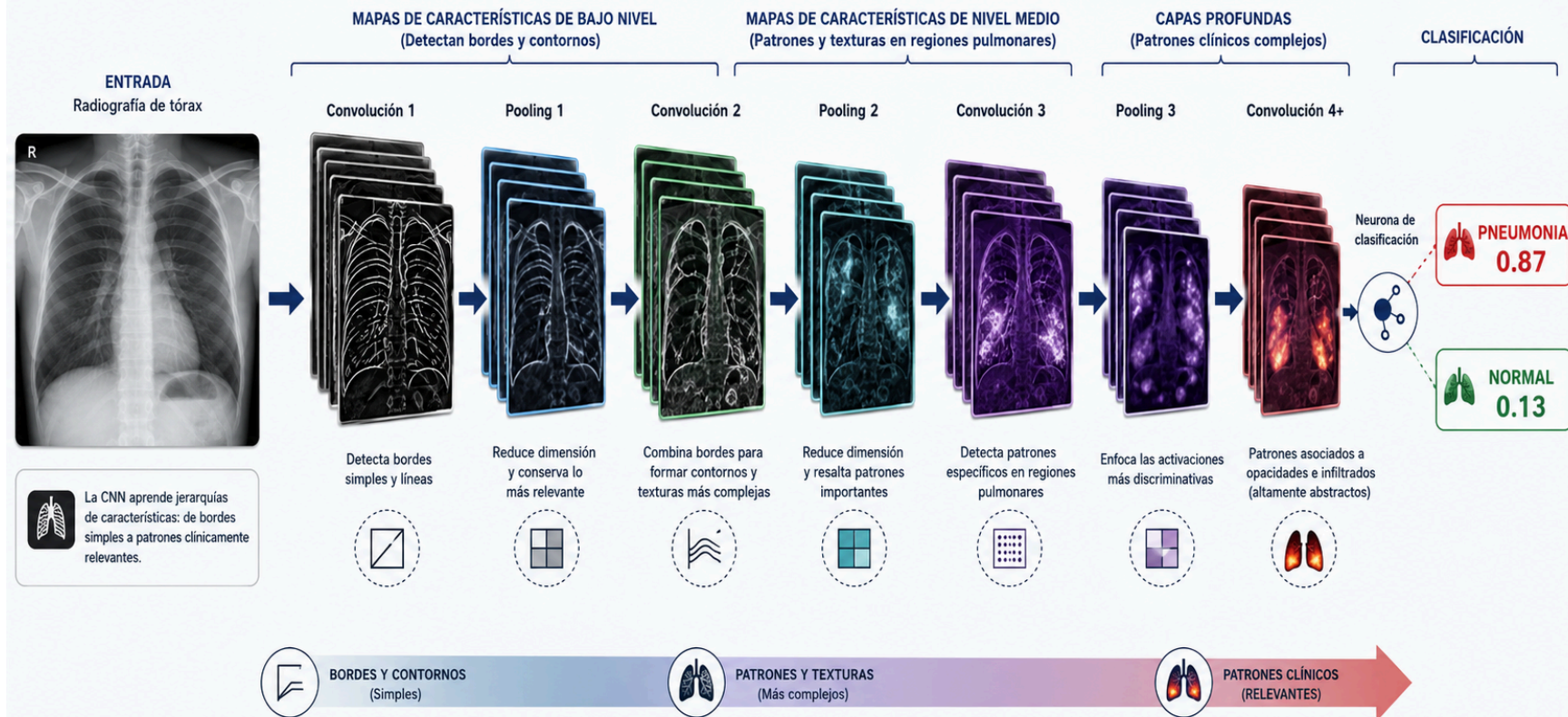


Ilustración x - Representación CNN para Radiografías de Tórax

1.5 *Los Transformers en visión artificial*

1.5.1 El mecanismo de atención

Para entender los modelos más recientes de este proyecto es necesario introducir otra familia de arquitecturas: los Transformers. Su historia comienza en el procesamiento del lenguaje natural. Durante años, las tareas de lenguaje se resolvían con redes recurrentes que procesaban las palabras de una frase una a una, en orden, manteniendo un estado interno. Este enfoque era lento y tenía dificultades para recordar relaciones entre palabras muy separadas dentro del texto.

En 2017, el artículo "Attention is all you need" (Vaswani & al., 2017) propuso una arquitectura radicalmente distinta: el Transformer, basada en un mecanismo denominado atención. La idea esencial de la atención es que, al procesar un elemento de una secuencia (por ejemplo, una palabra), el modelo puede "prestar atención" a todos los demás elementos de la secuencia simultáneamente, ponderando cuánto influye cada uno de ellos. Así, al interpretar la palabra "neumonía" en la frase "la radiografía muestra una neumonía en el lóbulo inferior", el modelo aprende a relacionarla con "radiografía" y "lóbulo inferior" aunque estén lejos en la frase.

Dos conceptos acompañan a la atención: la **autoatención**, que establece relaciones entre todos los elementos de una misma secuencia (cada palabra se relaciona con todas las demás), y las **cabezas de atención**, que son múltiples mecanismos de atención ejecutados en paralelo, cada uno especializado en un tipo distinto de relación. Además, como el Transformer no procesa los elementos en orden, necesita incorporar una **codificación posicional**, una información que le indica en qué posición de la secuencia se encuentra cada elemento (Vaswani & al., 2017).

1.5.2 De las palabras a los píxeles

Los Transformers demostraron ser extraordinarios en el lenguaje, pero las imágenes parecían un territorio hostil para ellos, porque una imagen no es una secuencia de símbolos. La idea que resolvió este problema fue sorprendentemente directa: tratar la imagen como si fuera una secuencia de palabras.

El Vision Transformer (ViT) propuesto por Dosovitskiy y colaboradores en 2021 (Dosovitskiy & al., 2021) divide la imagen en una cuadrícula de pequeños fragmentos cuadrados llamados *patches* (parches). Cada parche se convierte en un vector numérico mediante una transformación lineal, de forma análoga a como cada palabra se convierte en un vector que la representa. Estos vectores, llamados *tokens*, se introducen en un Transformer junto con su codificación posicional, de modo que el modelo aplica autoatención entre todos los parches de la imagen. Para producir la clasificación final, el ViT utiliza un token especial denominado token de clasificación, cuyo estado final después de atravesar el modelo se conecta a una capa de clasificación.

La diferencia conceptual con las CNN es profunda y muy relevante para este proyecto. Una CNN procesa la imagen de forma local y progresiva: cada neurona solo "ve" una pequeña región, y la visión global se construye capa a capa. Un Transformer, en cambio, relaciona desde la primera capa todas las regiones de la imagen entre sí, lo que le permite captar dependencias de largo alcance que a una CNN podrían pasarle desapercibidas. En el contexto de la neumonía, esto significa que un Transformer puede relacionar directamente, por ejemplo, la densidad anómala de un lóbulo pulmonar con la expansión del pulmón opuesto, sin necesidad de que esa relación emerja capa tras capa. La contrapartida es que los Transformers son muy "hambrientos de datos": necesitan grandes cantidades de imágenes para aprender correctamente y tienen un coste computacional elevado, especialmente cuando la imagen se divide en muchos parches (Dosovitskiy & al., 2021). Precisamente por eso su aplicación al ámbito médico, donde los conjuntos de datos suelen ser reducidos, exige una evaluación cuidadosa, que es uno de los objetivos de este trabajo.

1.5.3 Las arquitecturas Transformer del proyecto

Este proyecto evalúa tres arquitecturas basadas en atención:

- **DeiT** (Data-efficient Image Transformers) (Touvron & al., 2021): DeiT aborda directamente el problema del hambre de datos de los ViT. Su idea central es el uso de un token de destilación: durante el entrenamiento, un profesor (típicamente una CNN muy potente) "enseña" al Transformer, y el token de destilación es un elemento adicional que el modelo aprende a usar para imitar al profesor. Esto permite que un Transformer se entrene correctamente con muchos menos datos. En este proyecto se emplea la variante *deit-base-distilled*, preentrenada a 224×224 píxeles.
- **Swin Transformer** (Liu & al., 2021): Swin ataca el problema del coste computacional de los Transformers. En lugar de calcular la autoatención sobre todos los parches de la imagen (lo que crece de forma cuadrática con el número de parches), Swin la calcula dentro de ventanas locales, que además se desplazan entre capas consecutivas (ventanas desplazadas). Este diseño jerárquico, inspirado en las CNN, reduce el coste y permite captar información a múltiples escalas. La variante evaluada es *swin-base*, con parches de 4×4 y ventanas de 7×7 , a resolución 224×224 .
- **ViT-384**: es el Vision Transformer estándar, pero configurado para trabajar con una resolución de entrada mayor (384×384 píxeles en lugar de los 224 habituales). Trabajar con más píxeles equivale a dividir la imagen en más parches, lo que proporciona más detalle espacial al modelo (al precio de un mayor coste computacional). La variante utilizada es *vit-base-patch16-384*.

La inclusión de estos tres modelos permite comparar de forma sistemática el paradigma de las CNN con el de los Transformers dentro de un mismo diseño de evaluación, aunque no bajo un protocolo de optimización idéntico. Se mantienen comunes las particiones estratificadas, el número de pliegues, el balanceo del entrenamiento, la aumentación, el conjunto de validación y las métricas; sin embargo, las CNN utilizan una base convolucional congelada y entrenan un nuevo cabezal, mientras que los Transformers ajustan sus pesos preentrenados con una tasa de aprendizaje menor y AdamW. Por ello, los resultados deben interpretarse como una comparación exploratoria de dos familias mediante protocolos adaptados a sus características, no como una atribución causal de cualquier diferencia exclusivamente a la arquitectura (Zhang & al., 2024).

1.6 La explicabilidad: inteligencia artificial explicable

1.6.1 El problema de la caja negra

Los modelos de aprendizaje profundo, tanto las CNN como los Transformers, comparten una característica que es, a la vez, su mayor fortaleza y su mayor debilidad: son cajas negras. Internamente están formados por millones de pesos numéricos que se ajustan durante el entrenamiento, y aunque la red sea capaz de clasificar correctamente una radiografía, no es evidente por qué ha llegado a esa conclusión. Un médico no puede aceptar sin más que un sistema le diga "este paciente tiene neumonía": necesita saber si el modelo se está fijando en la zona correcta del pulmón o si, por el contrario, está siendo engañado por algún artefacto de la imagen, una etiqueta o una peculiaridad del conjunto de entrenamiento.

Esta exigencia de transparencia es especialmente crítica en el ámbito clínico, donde las decisiones deben estar fundamentadas y ser auditables. La inteligencia artificial explicable (XAI) es la disciplina que estudia cómo hacer comprensibles las decisiones de los modelos de aprendizaje automático (Holzinger & al., 2022; Linardatos, Papastefanopoulos, & Kotsiantis, 2021; Tjoa & Guan, 2020). No se trata únicamente de generar una imagen bonita con zonas coloreadas, sino de proporcionar al profesional sanitario una herramienta que le permita verificar, de forma visual e intuitiva, qué regiones de la radiografía han sido determinantes para la decisión del modelo. De este modo, si el sistema apunta correctamente a las zonas de opacidad características de la neumonía, el clínico puede confiar en la predicción; si apunta a regiones irrelevantes, la predicción debe ser puesta en duda (Rajaraman & Antani, 2018).

1.6.2 Técnicas de explicabilidad

Este proyecto utiliza tres familias de técnicas de explicabilidad:

Mapas de prominencia o Saliency Maps (Simonyan, Vedaldi, & Zisserman, 2014): la idea es preguntarse directamente al modelo qué píxeles de la imagen han influido más en su decisión. Para ello se calcula la derivada (el gradiente) de la salida de la red respecto a cada píxel de la imagen de entrada. Intuitivamente, el gradiente indica cómo cambiaría la predicción si se modificase un píxel concreto: si un pequeño cambio en un píxel altera mucho la decisión, ese píxel es importante. El resultado es un mapa que resalta las regiones más influyentes.

SmoothGrad (Smilkov & al., 2017): los mapas de saliencia crudos suelen ser muy ruidosos, con pequeñas variaciones que no tienen significado clínico. SmoothGrad mejora esta situación añadiendo ruido aleatorio a la imagen de entrada y promediando los mapas de saliencia obtenidos sobre varias versiones con ruido. El resultado es un mapa más suave y estable, en el que los patrones realmente consistentes del modelo destacan sobre el ruido.

Grad-CAM (Selvaraju & al., 2017): Grad-CAM produce mapas de activación de clase, que resaltan las regiones de la imagen que más contribuyen a la clase predicha. Para ello combina los mapas de características de la última capa convolucional del modelo con los gradientes de la clase predicha, ponderando cada mapa de características por su importancia. El resultado es un mapa de calor grueso y fácil de interpretar, que se superpone a la radiografía original. Grad-CAM es especialmente valioso en medicina porque sus mapas son suaves y corresponden de forma natural a las regiones anatómicas que el clínico puede inspeccionar.

Para las arquitecturas Transformer se emplea una variante complementaria: los mapas de atención, que extraen directamente los pesos de atención de la última capa del modelo. Como se explicó en la sección anterior, los Transformers deciden cuánta importancia prestar a cada parche de la imagen; esos pesos de atención indican, por tanto, qué regiones de la radiografía está "mirando" el modelo (Chefer, Gur, & Wolf, 2021). En este proyecto, los mapas de atención se obtienen promediando las cabezas de atención de la última capa.

1.6.3 La evaluación cuantitativa

Observar mapas de calor a simple vista es necesario, pero no es suficiente para una evaluación científica rigurosa: dos observadores pueden opinar de forma distinta sobre la calidad de un mismo mapa. Por ello, este proyecto incorpora métricas cuantitativas que puntúan de forma objetiva distintas propiedades de las explicaciones generadas. La evaluación de fidelidad mediante borrado e inserción se fundamenta en Petsiuk, Das y Saenko (2018), mientras que la consideración de medidas de concentración y complejidad se encuadra en la literatura general sobre evaluación de métodos XAI (Linardatos, Papastefanopoulos, & Kotsiantis, 2021):

- **Deletion AUC** (área bajo la curva de borrado): mide qué ocurre cuando se borran progresivamente de la imagen las regiones que el mapa considera más importantes. Si al borrarlas la predicción del modelo se degrada rápidamente, el mapa señala realmente las regiones determinantes (cuanto más baja la curva, mejor la explicación).
- **Insertion AUC** (área bajo la curva de inserción): el proceso inverso; se parte de una imagen borrosa y se van añadiendo progresivamente las regiones importantes. Si el modelo acierta pronto al incorporarlas, la explicación es buena (cuanto más alta, mejor).
- **Sparsity** (dispersión): mide si el mapa es compacto y se concentra en pocas regiones, o si por el contrario es una mancha difusa que lo abarca todo. Un buen mapa debe ser directo (Linardatos, Papastefanopoulos, & Kotsiantis, 2021).
- **Entropy** (entropía): evalúa la concentración de la información del mapa desde el punto de vista de la teoría de la información; complementa a la dispersión (Linardatos, Papastefanopoulos, & Kotsiantis, 2021).
- **Stability SSIM**: mide la consistencia de las explicaciones ante pequeñas perturbaciones de la imagen de entrada, utilizando el índice de similitud estructural SSIM (Wang & al., 2004). Un modelo estable produce mapas similares para imágenes casi idénticas.

1.6.4 La calibración

Existe una dimensión adicional de fiabilidad que a menudo se pasa por alto y que este proyecto incorpora como mejora respecto al estudio de referencia: la calibración. Cuando una red dice "neumonía con un 95% de confianza", ese 95% debería significar que, de cada cien casos clasificados con ese nivel de confianza, aproximadamente noventa y cinco son efectivamente neumonía. En la práctica, muchas redes modernas están mal calibradas: son demasiado optimistas, y sus probabilidades declaradas no se corresponden con su precisión real (Guo, Pleiss, Sun, & Weinberger, 2017).

La métrica estándar para medir esta desviación es el Expected Calibration Error (ECE), que cuantifica la diferencia media entre la confianza declarada por el modelo y su precisión real, agrupando las predicciones en intervalos de confianza. Complementariamente se utiliza el Brier Score, una métrica que evalúa la calidad global de las probabilidades predichas. La calibración es un aspecto crítico en entornos clínicos: una alta confianza en una predicción errónea puede tener consecuencias graves, y un sistema que sobrestima su seguridad no es clínicamente fiable. Incluir estas métricas permite evaluar no solo si los modelos aciertan, sino también si saben cuándo aciertan.

En este punto conviene aclarar, por honestidad con el lector, cómo se distribuyen las cuatro técnicas XAI entre las dos vías de uso de la plataforma. En la vía de diagnóstico de la aplicación web, el módulo de explicabilidad implementa las cuatro técnicas visuales descritas (Saliency Maps, SmoothGrad y Grad-CAM para redes convolucionales, y mapas de atención para Transformers), que se generan de forma automática en cada consulta del profesional sanitario. En la vía de laboratorio MLOps, los scripts del pipeline de entrenamiento generan los mapas de explicabilidad cualitativos (Grad-CAM y mapas de saliencia para las CNN, y mapas de saliencia para los Transformers) y calculan las métricas cuantitativas y de calibración descritas en esta sección.

1.7 Las barreras para la adopción clínica de la IA

A pesar del potencial demostrado por la inteligencia artificial en el diagnóstico por imagen, la adopción clínica de estos sistemas se enfrenta a tres barreras fundamentales que este Trabajo Fin de Grado aborda de manera específica. La primera de ellas es la falta de transparencia, que se acaba de describir: los modelos generan predicciones sin proporcionar una justificación comprensible para el facultativo. vitalXAI aborda esta barrera mediante la inteligencia artificial explicable presentada en la sección anterior, incorporando mapas de calor interpretativos y métricas de fiabilidad en cada diagnóstico, de modo que las decisiones del modelo puedan ser verificadas clínicamente.

La segunda barrera es la falta de robustez y generalización. Un modelo puede alcanzar resultados prometedores en las condiciones controladas del laboratorio, pero su rendimiento se degrada de forma significativa al aplicarse sobre conjuntos de datos procedentes de poblaciones diferentes, equipos de adquisición distintos o protocolos de imagen alternativos (Tang & al., 2021). Un sistema que solo funciona bien con las imágenes con las que se entrenó tiene poco valor clínico real. Por ello, la validación externa sobre cohortes independientes —imágenes que el modelo nunca ha visto, procedentes de otro centro y de otra población— constituye un requisito indispensable para cualquier sistema que aspire a un despliegue clínico real (Alsaedi & Alshehri, 2024). En el marco de este proyecto, se emplea el COVID-19 Radiography Database como fuente de imágenes de pacientes adultos para la validación externa, siguiendo la metodología establecida en el estudio de referencia.

Conviene detenerse en este punto por las implicaciones del dataset elegido. El COVID-19 Radiography Database fue desarrollado durante la pandemia de COVID-19 por investigadores de la Universidad de Qatar y la Universidad de Dhaka, precisamente para apoyar la investigación de herramientas de cribado basadas en radiografías de tórax durante la crisis sanitaria (Chowdhury & al., 2020; Rahman & al., 2021). Durante la pandemia, el mundo asistió a la necesidad acuciante de herramientas que ayudaran a clasificar y priorizar pacientes mediante imágenes médicas, y numerosos estudios exploraron el uso de redes neuronales para la detección de COVID-19 en radiografías de tórax (Apostolopoulos & Mpesiana, 2020; Minaee & al., 2020). La existencia de una herramienta robusta de diagnóstico asistido de neumonía podría haber contribuido a aliviar la saturación de los servicios de radiología en contextos de alta demanda. Sin embargo, es esencial ser escrupulosos con lo que este proyecto afirma: todos los entrenamientos y todos los resultados presentados en esta memoria se basan en la detección de neumonía, y no en la detección específica de COVID-19. La elección de este dataset como banco de pruebas de la validación externa responde a que no sabemos con certeza cómo se comportarán los modelos entrenados con radiografías pediátricas al enfrentarse a una población adulta distinta, con equipos y protocolos de adquisición diferentes. Precisamente porque no podemos asumir que el rendimiento se mantenga, lo sometemos a esta prueba de estrés. Sin embargo, un resultado favorable no debe interpretarse por sí solo como evidencia concluyente de robustez: la mezcla de fuentes del dataset puede introducir factores de confusión que permitan distinguir el origen de las imágenes en lugar de la presencia de neumonía (DeGrave, Janizek, & Lee, 2021). Por ello, los resultados sobre esta cohorte se considerarán evidencia complementaria y deberán interpretarse junto con sus limitaciones, no como una demostración directa de potencial utilidad clínica durante una pandemia.

La tercera barrera es la usabilidad. La mayoría de las soluciones de inteligencia artificial desarrolladas en el ámbito académico requieren conocimientos técnicos avanzados para su ejecución: saber programar, instalar entornos, ejecutar scripts o interpretar archivos de resultados. Esto excluye precisamente a los profesionales sanitarios que deberían poder beneficiarse de estas herramientas. Las investigaciones recientes en MLOps (del inglés Machine Learning Operations, las prácticas que automatizan el ciclo de vida completo de los modelos: entrenamiento, validación, despliegue y monitorización) subrayan la necesidad de plataformas que acerquen estas capacidades a usuarios no especializados a través de interfaces intuitivas (Argaw & al., 2024; Kreuzberger, Kühl, & Hirschl, 2023).

VitalXAI ataca esta barrera desde un principio de diseño muy concreto: si los asistentes conversacionales modernos —similares a los sistemas de diálogo basados en grandes modelos de lenguaje que millones de personas utilizan a diario— han conseguido que interactuar con una máquina sea natural e inmediato para cualquiera, ¿por qué no adoptar ese mismo paradigma en una plataforma de diagnóstico médico? La interfaz de vitalXAI se ha concebido siguiendo la apariencia y la dinámica de estos asistentes conversacionales, tipo ChatGPT, porque es algo con lo que cualquier persona está hoy en día familiarizada. El laboratorio de entrenamiento de la plataforma incorpora un asistente conversacional con el que el facultativo puede hablar en lenguaje natural para configurar y lanzar experimentos de entrenamiento, sin escribir una sola línea de código y sin conocer el motor computacional que hay detrás. La complejidad técnica queda completamente oculta tras una conversación: esa es la forma de reducir la curva de aprendizaje y hacer que la tecnología sea realmente utilizable para el personal sanitario.

1.8 Los asistentes conversacionales y los grandes modelos de lenguaje

1.8.1 ¿Qué son los grandes modelos de lenguaje?

Dado que el asistente conversacional es uno de los componentes diferenciales de vitalXAI, es necesario situar esta tecnología en su contexto. Los grandes modelos de lenguaje (del inglés large language models, LLM) son redes neuronales Transformer de enormes dimensiones, entrenadas sobre cantidades ingentes de texto procedente de internet, libros, artículos científicos y otras fuentes. Su tarea de entrenamiento fundamental es sorprendentemente sencilla: predecir la siguiente palabra de un texto. A fuerza de practicar esa tarea sobre millones de documentos, el modelo aprende patrones estadísticos enormemente ricos sobre el lenguaje, el conocimiento del mundo y el razonamiento (Brown & al., 2020; Vaswani & al., 2017).

Pero un modelo que solo predice la siguiente palabra no es todavía un "chatbot útil". Para convertirlo en un asistente conversacional se realiza un paso adicional denominado ajuste por instrucciones y aprendizaje por refuerzo con retroalimentación humana (RLHF): el modelo se entrena, además, para seguir instrucciones de forma útil y para responder en formato de diálogo (Ouyang & al., 2022). El resultado es lo que conocemos como asistentes de inteligencia artificial conversacionales, capaces de mantener conversaciones fluidas, responder preguntas, razonar sobre un problema y, como en el caso que nos ocupa, extraer la información que el usuario introduce en lenguaje natural para realizar una acción técnica concreta.

La aplicación de los grandes modelos de lenguaje al ámbito de la medicina es un campo de investigación muy activo. Se han publicado numerosos trabajos que exploran su uso como apoyo en tareas clínicas: responder preguntas médicas, redactar informes o asistir en el diagnóstico, con resultados prometedores pero también con advertencias importantes sobre la necesidad de validación rigurosa antes de su uso clínico (Thirunavukarasu & al., 2023; Singhal & al., 2023). En este proyecto, sin embargo, el modelo de lenguaje no desempeña funciones de diagnóstico: su papel es el de un asistente de configuración que interpreta las peticiones del usuario sobre qué experimento de entrenamiento quiere lanzar y las traduce a los parámetros técnicos que necesita el sistema.

1.8.2 El panorama de los modelos de lenguaje y la elección de Llama 3

En el momento de desarrollo de este proyecto existen numerosos grandes modelos de lenguaje, con características muy diferentes. Por un lado están los modelos propietarios y cerrados, como GPT-4 de OpenAI, Claude de Anthropic o Gemini de Google, accesibles únicamente a través de sus APIs comerciales. Estos modelos son muy potentes, pero su código no está disponible, el usuario no puede adaptarlos ni ejecutarlos en su propia infraestructura, y su uso está sujeto a las políticas y precios de cada proveedor. Por otro lado están los modelos de acceso abierto, entre los que destacan la familia Llama de Meta y los modelos de Mistral AI, cuyos pesos son publicados y pueden descargarse y utilizarse libremente (Touvron & al., 2023; Jiang & al., 2023).

La elección de Llama 3 (Meta AI, 2024) para el asistente de vitalXAI se justifica por varias razones. En primer lugar, es un modelo de pesos abiertos, lo que se alinea con el espíritu del proyecto: transparencia, reproducibilidad y ausencia de dependencia de un proveedor concreto. En segundo lugar, ofrece un rendimiento competitivo con los mejores modelos propietarios en razonamiento y seguimiento de instrucciones, según las evaluaciones publicadas por el propio equipo de Meta, manteniendo un tamaño razonable (el modelo empleado, Llama 3.3 de 70.000 millones de parámetros, ofrece una relación calidad-coste excelente). En tercer lugar, su licencia permite tanto el uso en investigación como el uso comercial, lo que facilita la eventual transferencia de la tecnología. Y en cuarto lugar, al tratarse de un modelo abierto, es posible adaptarlo o sustituirlo en el futuro sin necesidad de reescribir el sistema.

La ejecución del modelo se delega en la plataforma Groq (Groq, 2024). Groq ofrece inferencia de modelos de lenguaje a través de una API en la nube, pero con una particularidad hardware: utiliza unos chips propios denominados unidades de procesamiento de lenguaje (LPU, del inglés Language Processing Units), diseñados específicamente para ejecutar modelos de Transformer a velocidades extremadamente altas y con una latencia muy reducida, muy por debajo de la que ofrecen las GPUs convencionales. Esta velocidad es clave para la experiencia de usuario: un asistente conversacional que responde con fluidez se siente natural y cercano a lo que el usuario espera de un sistema tipo ChatGPT, mientras que un asistente lento arruinaría la sensación de conversación. Además, Groq ofrece un plan gratuito con cuotas suficientes para un proyecto académico, lo que encaja con las restricciones de coste de un Trabajo Fin de Grado.

El asistente conversacional de vitalXAI se configura con un prompt de sistema cuidadosamente diseñado (una técnica denominada prompt engineering) que define su comportamiento como experto en MLOps médico. El asistente guía al usuario, en lenguaje natural, por los cinco parámetros que definen un experimento de entrenamiento: la ruta del dataset, las arquitecturas a entrenar, el número de épocas, el tamaño de lote y la tasa de aprendizaje. Cuando el usuario ha proporcionado todos los datos, el asistente devuelve un objeto JSON estructurado con la configuración completa, que el sistema interpreta automáticamente para lanzar el pipeline de entrenamiento. El asistente está además internacionalizado en cuatro idiomas (español, inglés, chino e hindi), ampliando su accesibilidad a usuarios de distintas procedencias.

1.9 Metodología experimental

La metodología experimental de este proyecto se estructura como un pipeline completo que amplía y mejora el marco de referencia establecido por Kermany y colaboradores (Kermany & al., 2018). El pipeline se organiza en cuatro fases que se corresponden, de forma natural, con la numeración de los siete scripts de entrenamiento y evaluación que componen el motor computacional del sistema, y que son orquestados automáticamente por el laboratorio MLOps de la plataforma. A lo largo de esta sección se describe cada fase, explicando los conceptos estadísticos y de aprendizaje automático necesarios para comprenderla sin conocimientos previos. Para favorecer la legibilidad, los conceptos se explican la primera vez que aparecen y no se repiten en fases posteriores.

1.9.1 Fase 1: Entrenamiento con validación cruzada

El objetivo de esta primera fase es entrenar cada una de las diecinueve arquitecturas (dieciséis CNN y tres Transformers) y obtener una estimación fiable y comparable de su rendimiento diagnóstico.

Un problema previo: cómo evaluar un modelo de forma fiable. Si entrenásemos un modelo con todas las imágenes y lo evaluásemos con esas mismas imágenes, el resultado sería engañosamente optimista: la red podría haber memorizado las imágenes en lugar de aprender a generalizar. La forma correcta de evaluar es separar las imágenes en dos grupos disjuntos: un conjunto de entrenamiento, con el que el modelo aprende, y un conjunto de validación, con el que se mide su rendimiento sobre datos que no ha visto. Ahora bien, si elegimos de forma arbitraria qué imágenes van a cada grupo, podríamos tener mala suerte y obtener una partición poco representativa. Para evitar este problema se utiliza la validación cruzada de k pliegues (k -fold cross-validation), una técnica que repite la evaluación varias veces con particiones distintas. Concretamente, el proyecto emplea cinco pliegues (5-fold): las imágenes se dividen aleatoriamente en cinco grupos del mismo tamaño; en cada una de las cinco iteraciones, cuatro de los grupos se usan para entrenar y el grupo restante para validar, de modo que cada imagen forma parte del conjunto de validación exactamente una vez. El modelo se entrena, por tanto, cinco veces, y el rendimiento final se resume con la media y la desviación típica de las métricas de las cinco iteraciones. Este diseño proporciona una estimación robusta del rendimiento y reduce la dependencia de una partición concreta.

El problema del desbalanceo. Como se comentó en la sección de datos, el dataset contiene muchas más imágenes de neumonía que de pacientes sanos (4.273 frente a 1.583). Un modelo entrenado con un conjunto desbalanceado puede sesgarse hacia la clase mayoritaria, porque predecir siempre la clase más frecuente reduce su error medio. Existen varias respuestas estándar a este problema, como ponderar la función de pérdida, sobremuestrear la clase minoritaria o utilizar pérdidas diseñadas para datos desbalanceados (He & Garcia, 2009). Este proyecto aplica submuestreo aleatorio (random undersampling): en cada pliegue, del conjunto de entrenamiento se seleccionan aleatoriamente tantas imágenes de la clase mayoritaria como imágenes tiene la clase minoritaria, descartando el excedente para ese pliegue. La decisión ofrece un entrenamiento sencillo y perfectamente balanceado, pero destruye información: sobre el conjunto completo se descartan 2.690 imágenes de neumonía y, dado que cada pliegue utiliza aproximadamente el 80 % de los datos para entrenar, se descartan alrededor de 2.150 imágenes mayoritarias en cada partición de entrenamiento. Además, la selección aleatoria introduce variabilidad adicional en las estimaciones; fijar la semilla y repetir los cinco pliegues la reduce,

pero no elimina el efecto de depender de una muestra parcial. Por tanto, los resultados quedan condicionados a esta decisión y no permiten concluir que el submuestreo sea superior a la ponderación de clases o al sobremuestreo, alternativas que no se han sometido a un contraste experimental independiente. La evaluación sobre el conjunto de validación se realiza siempre sobre la partición completa, sin submuestrear, para que las métricas reflejen el rendimiento sobre datos con su distribución natural.

Transferencia de conocimiento. En las CNN, se cargan los pesos aprendidos sobre ImageNet, se elimina la cabeza de clasificación original y se conserva la base convolucional como extractor de características con todos sus pesos congelados. Sobre ella se añade un cabezal específico para la clasificación binaria, formado por Global Average Pooling, capas densas con activación ReLU, Dropout y una salida sigmoide; únicamente este cabezal se actualiza durante el entrenamiento. Esta decisión reduce el número de parámetros que deben aprenderse con las radiografías y limita el riesgo de sobreajuste, pero también puede impedir que las características preentrenadas se adapten completamente al dominio médico (Yosinski & al., 2014). En los Transformers, en cambio, se parte igualmente de pesos preentrenados para imágenes, pero se ajusta el modelo completo con el optimizador AdamW, regularización por decaimiento de pesos y una tasa de aprendizaje más conservadora para favorecer la convergencia sin destruir las representaciones iniciales. En ambos casos se minimiza una pérdida de entropía cruzada binaria, aunque la implementación de los Transformers la aplica sobre sus logits. Durante el entrenamiento de ambas familias se utiliza la misma aumentación ligera de datos —volteos horizontales aleatorios y pequeños ajustes de brillo y contraste—, que crea variaciones sintéticas y ayuda a reducir el sobreajuste.

Hiperparámetros y criterios de parada. El entrenamiento se configura mediante cuatro hiperparámetros controlables desde la plataforma: el número de épocas (pasadas completas sobre el conjunto de entrenamiento), el tamaño de lote (número de imágenes procesadas por cada actualización de pesos), la tasa de aprendizaje (la magnitud de cada paso de optimización) y la arquitectura a entrenar. Además, el entrenamiento incorpora mecanismos automáticos de control: se guarda en cada pliegue la versión del modelo con menor pérdida de validación (best_foldN), se detiene el entrenamiento de forma anticipada si la pérdida de validación no mejora durante varias épocas (early stopping), y se reduce la tasa de aprendizaje cuando el progreso se estanca (reduce learning rate on plateau). La semilla aleatoria se fija de forma global (42) para que los experimentos sean reproducibles.

Métricas de rendimiento. Para medir la calidad de cada modelo, la implementación calcula cinco métricas estándar y, para una interpretación clínica completa, deben considerarse además la especificidad y el valor predictivo negativo. Para entenderlas es imprescindible presentar primero la matriz de confusión, una tabla que resume los cuatro posibles resultados de un clasificador binario: verdaderos positivos (VP, imágenes con neumonía correctamente clasificadas), verdaderos negativos (VN, sanos correctamente clasificados), falsos positivos (FP, sanos clasificados como enfermos) y falsos negativos (FN, enfermos clasificados como sanos). A partir de esta matriz se definen:

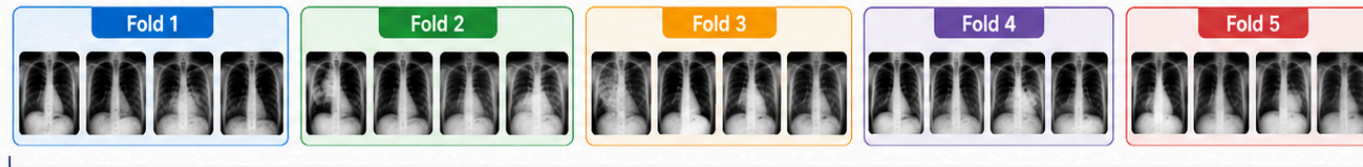
- **Exactitud** (accuracy): proporción de aciertos sobre el total de predicciones. Es intuitiva, pero puede resultar engañosa cuando las clases están desbalanceadas.
- **Precisión** (precision): de todas las imágenes que el modelo clasifica como neumonía, qué proporción son realmente neumonía ($VP / (VP + FP)$). Una precisión baja implica muchas falsas alarmas.

- **Sensibilidad o cobertura (recall):** de todas las imágenes que realmente tienen neumonía, qué proporción detecta el modelo ($VP / (VP + FN)$). En medicina esta es una métrica crítica: el peor error posible es un falso negativo, dejar pasar a un enfermo real. Una sensibilidad baja significa que el modelo pasa por alto pacientes con la enfermedad.
- **F1-score:** la media armónica de precisión y sensibilidad, que resume ambas en una única cifra.
- **Especificidad:** de todas las imágenes sanas, qué proporción identifica correctamente como normales ($VN / (VN + FP)$). Es la medida complementaria a la sensibilidad y permite cuantificar la capacidad de evitar falsos positivos.
- **Valor predictivo negativo (VPN):** de todas las imágenes clasificadas como normales, qué proporción es realmente sana ($VN / (VN + FN)$). Resulta especialmente relevante cuando una predicción negativa se utiliza para descartar enfermedad.
- **Área bajo la curva ROC (AUC):** permite evaluar la discriminación sin fijar un único umbral de decisión. La curva ROC representa, para todos los umbrales posibles, la tasa de verdaderos positivos frente a la tasa de falsos positivos (Fawcett, 2006). El área bajo esa curva (AUC) resume la capacidad del modelo para distinguir enfermos de sanos con un único número entre 0,5 (equivale a lanzar una moneda) y 1 (clasificación perfecta). Un AUC de 0,9 significa que, ante una imagen con neumonía y una sana elegidas al azar, el modelo asigna mayor probabilidad de enfermedad a la correcta en el 90% de los casos. Esta estimación puntual debería acompañarse de un intervalo de confianza, preferiblemente del 95 %, para reflejar la incertidumbre muestral; el pipeline actual almacena el AUC por pliegue y su desviación típica, pero no calcula todavía intervalos de confianza para la AUC, por lo que esta constituye una limitación metodológica que debe declararse.

Cada script de entrenamiento (CNN o Transformer) genera, para cada modelo y cada pliegue, los pesos del mejor modelo, las predicciones sobre el conjunto de validación en formato CSV, y un archivo resumen con las cinco métricas por pliegue junto con su media y desviación típica.

VALIDACIÓN CRUZADA DE CINCO PLIEGUES PARA RADIOGRAFÍAS DE TÓRAX

CONJUNTO DE DATOS
Dividido en 5 pliegues (folds)



El conjunto total se divide en 5 pliegues de tamaño similar



Ilustración x - Representación de Validación Cruzada

1.9.2 Fase 2: Comparación estadística entre modelos

Una vez entrenados todos los modelos de una sesión, la pregunta natural es: ¿qué modelo es el mejor y las diferencias observadas entre ellos son reales o fruto del azar? La comparación estadística no es un detalle metodológico: dos modelos pueden diferir en su AUC medio por unas centésimas, y esa diferencia puede ser puramente aleatoria, consecuencia de la variabilidad inherente al entrenamiento.

El proyecto calcula de forma exploratoria el test de los rangos con signo de Wilcoxon (Wilcoxon, 1945) sobre las cinco AUC obtenidas por cada par de modelos en los cinco pliegues. Sin embargo, este resultado no puede utilizarse como un contraste confirmatorio: con $n = 5$ pares, el p-valor bilateral exacto mínimo alcanzable es 0,0625, incluso cuando todas las diferencias apuntan en la misma dirección. Por construcción, ninguna comparación puede alcanzar el umbral convencional de 0,05. Además, los folds de una validación cruzada no son muestras independientes, porque sus conjuntos de entrenamiento se solapan; por ello, el supuesto de independencia de muchos contrastes pareados no se cumple (Nadeau & Bengio, 2003). La matriz debe interpretarse como un artefacto diagnóstico de las diferencias observadas, no como evidencia de significación estadística.

El script genera además un ranking global de modelos, ordenando las arquitecturas por su AUC medio y mostrando la desviación típica. También construye una matriz de calor con los p-valores exploratorios de las comparaciones por pares. Con diecinueve arquitecturas se realizan 171 comparaciones, y en la validación externa se plantea un número análogo de comparaciones mediante DeLong; por tanto, no es válido presentar p-valores sin corregir como si cada contraste fuese independiente. Si se conserva este análisis exploratorio, debe aplicarse una corrección por comparaciones múltiples, como Holm, y declarar que la dependencia entre folds limita la inferencia. Un análisis confirmatorio requeriría un procedimiento diseñado para comparar clasificadores con remuestreos dependientes, por ejemplo un test corregido de error de generalización (Nadeau & Bengio, 2003), o un rediseño con repeticiones independientes. El ranking y la matriz se integran posteriormente en el informe PDF, pero la captura no debe mostrar celdas interpretadas como significativas con $\alpha = 0,05$: el valor mínimo exacto esperable con cinco pares es 0,0625.

RANKING DE SESIÓN			
POSICIÓN	MODELO	MEDIA AUC	STD DEV
🏆	swin_base	0.9867	±0.0110
2	MobileNetV2	0.9854	±0.0072
3	deit	0.9783	±0.0229
4º	ConvNeXtTiny	0.8038	±0.0448
5º	MobileNetV3Large	0.7389	±0.0648

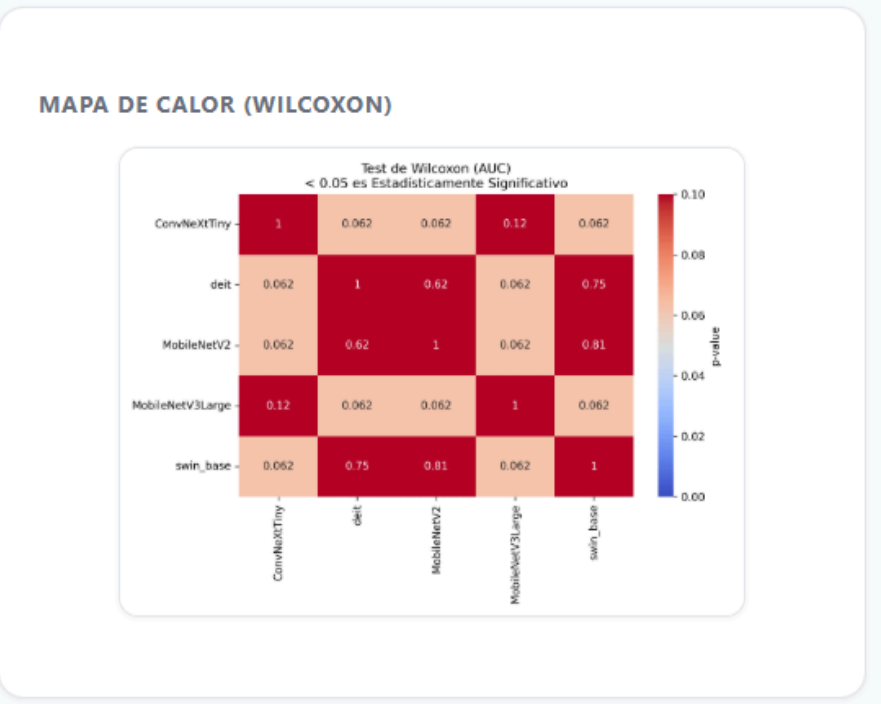


Ilustración x - Ranking global y Matriz de Wilcoxo

1.9.3 Fase 3: Validación externa y test de DeLong

La validación cruzada evalúa a los modelos sobre el mismo dataset pediátrico con el que se entrenaron, aunque en pliegues distintos. Pero el objetivo último de este proyecto es comprobar si los modelos son realmente capaces de generalizar a un entorno diferente. Esta es la pregunta que responde la validación externa.

En esta fase, los modelos entrenados se evalúan —congelados, sin ningún tipo de reaprendizaje— sobre el COVID-19 Radiography Database: quinientas radiografías de pacientes adultos sanos y quinientas de pacientes adultos con neumonía viral, que ningún modelo ha visto jamás. Se vuelven a calcular las cinco métricas de rendimiento sobre este conjunto y se construyen las curvas ROC correspondientes, que representan gráficamente la capacidad de cada modelo para distinguir enfermos de sanos sobre esta población desconocida. Si un modelo mantiene un AUC alto sobre esta cohorte adulta, se obtiene evidencia complementaria de su comportamiento ante un cambio de población y de adquisición, pero no una prueba aislada de robustez debido al posible confusor asociado al origen de las imágenes; si su rendimiento se desploma, habremos identificado una limitación importante de la generalización que debe documentarse con honestidad. La implementación actual utiliza, para cada arquitectura, el archivo `best_fold1`: se trata del mejor punto de control dentro del pliegue 1, seleccionado por su pérdida de validación, y no del mejor modelo entre los cinco pliegues. Esta elección queda fijada antes de evaluar el conjunto externo y evita seleccionar el modelo a partir de su rendimiento externo, pero hace que los resultados representen únicamente ese pliegue y no una agregación de los cinco. Por tanto, la validación externa debe interpretarse como una evaluación complementaria de un checkpoint preespecificado; una estimación más completa requeriría evaluar los cinco checkpoints y agregar sus resultados mediante un procedimiento definido de antemano. La comparación entre arquitecturas se realiza así bajo la misma regla de selección, aunque sus resultados no deben presentarse como el rendimiento externo medio de la validación cruzada (Varma & Simon, 2006).

Para determinar si las diferencias en las curvas ROC observadas en la validación externa son estadísticamente significativas se aplica el test de DeLong (DeLong, DeLong, & Clarke-Pearson, 1988). Este test está específicamente diseñado para comparar las áreas bajo dos curvas ROC obtenidas sobre las mismas muestras, teniendo en cuenta la correlación existente entre ambas predicciones. El script calcula el p-valor de la comparación entre todos los pares de modelos y genera una matriz de calor de significación, análoga a la del test de Wilcoxon, pero referida a la validación externa. De esta manera, el proyecto combina dos aproximaciones estadísticas complementarias: el test de Wilcoxon, que compara las distribuciones de rendimiento a partir de los cinco pliegues de la validación cruzada, y el test de DeLong, que compara las curvas ROC de la validación externa.

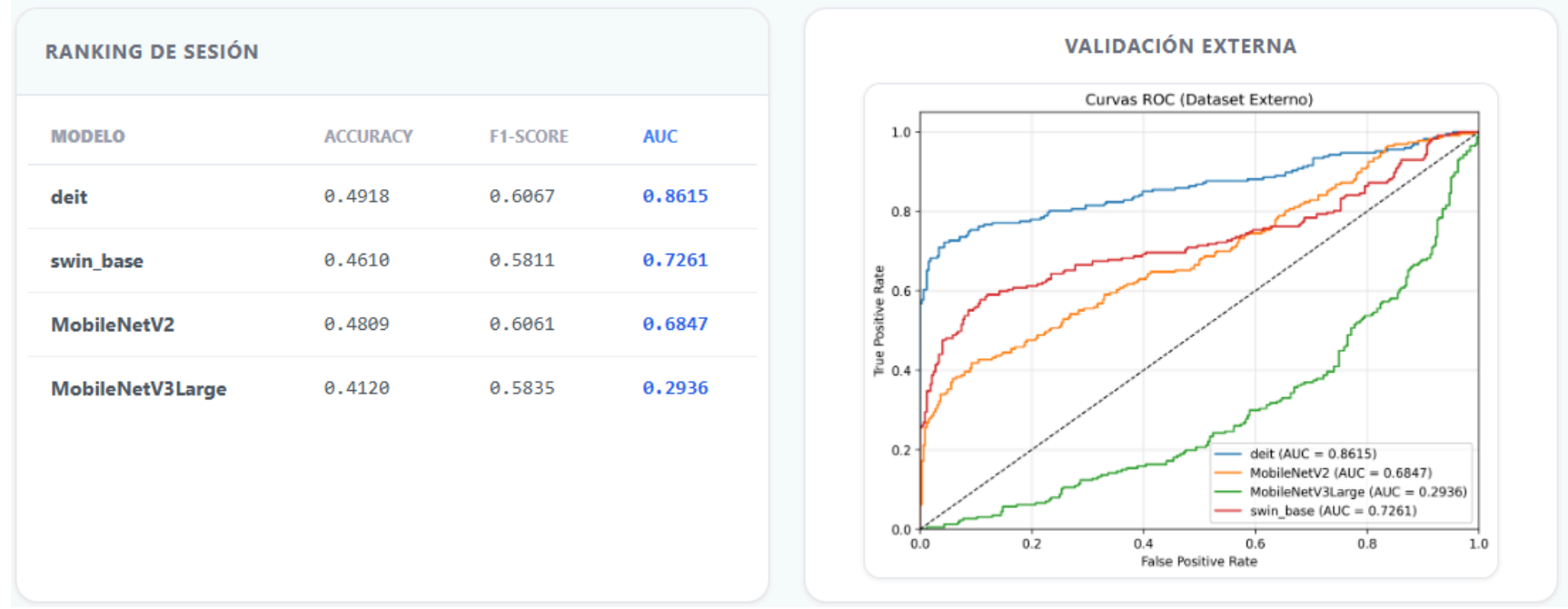


Ilustración x - Ranking de validación externa y curvas ROC

1.9.4 Fase 4: Análisis de explicabilidad

La última fase del pipeline evalúa la calidad de las explicaciones que los modelos son capaces de ofrecer, complementando las métricas de rendimiento predictivo con las métricas de fiabilidad descritas en la sección 1.6.

Análisis cualitativo: sobre un conjunto de imágenes de ejemplo del dataset de entrenamiento, se generan los mapas de explicabilidad visuales para cada modelo: Grad-CAM y mapas de saliencia para las redes convolucionales, y mapas de saliencia para los Transformers. El resultado es una galería de imágenes en las que se superpone el mapa de calor sobre la radiografía original, permitiendo inspeccionar visualmente si el modelo se fija en las regiones pulmonares con opacidad o si, por el contrario, sus explicaciones apuntan a zonas irrelevantes. Este análisis cualitativo es el primer filtro de coherencia clínica.

Análisis cuantitativo: sobre un conjunto de imágenes, se calculan las métricas objetivas de fidelidad de las explicaciones presentadas en la sección 1.6.3: Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM. Además, se calcula la calibración del modelo mediante el Expected Calibration Error (ECE) y el Brier Score, evaluando si la confianza declarada por el modelo se corresponde con su precisión real. El resultado se almacena en archivos comparativos que permiten puntuar numéricamente la calidad de la explicabilidad de cada arquitectura y comparar unas con otras de forma objetiva.

Interpretación de los resultados ilustrativos. Las capturas incluidas en esta fase muestran tanto el funcionamiento de la interfaz como dos comportamientos distintos del pipeline. El resultado de MobileNetV3Large, con exactitud 0,8000, precisión 0,8000, sensibilidad 1,0000 y F1 0,8889 idénticos en los cinco pliegues, junto con desviaciones típicas nulas, es compatible con un clasificador que predice siempre la clase positiva sobre una partición con un 80 % de positivos. Esa combinación no demuestra capacidad diagnóstica y debe considerarse una señal de posible fallo o de comportamiento degenerado que exige revisar las predicciones, las etiquetas, la partición y el preprocesamiento. Del mismo modo, un AUC externo de 0,2936 para MobileNetV3Large está por debajo del azar y puede indicar inversión sistemática de las clases, un problema de codificación de etiquetas, una incompatibilidad de preprocesamiento o un fuerte cambio de dominio; no debe interpretarse como evidencia ordinaria de generalización. La captura de MobileNetV2 complementa este caso al mostrar un comportamiento más coherente en la validación cruzada, pero ambos resultados deben leerse como ejemplos concretos del sistema y no como una selección de resultados representativos de todo el banco de pruebas.

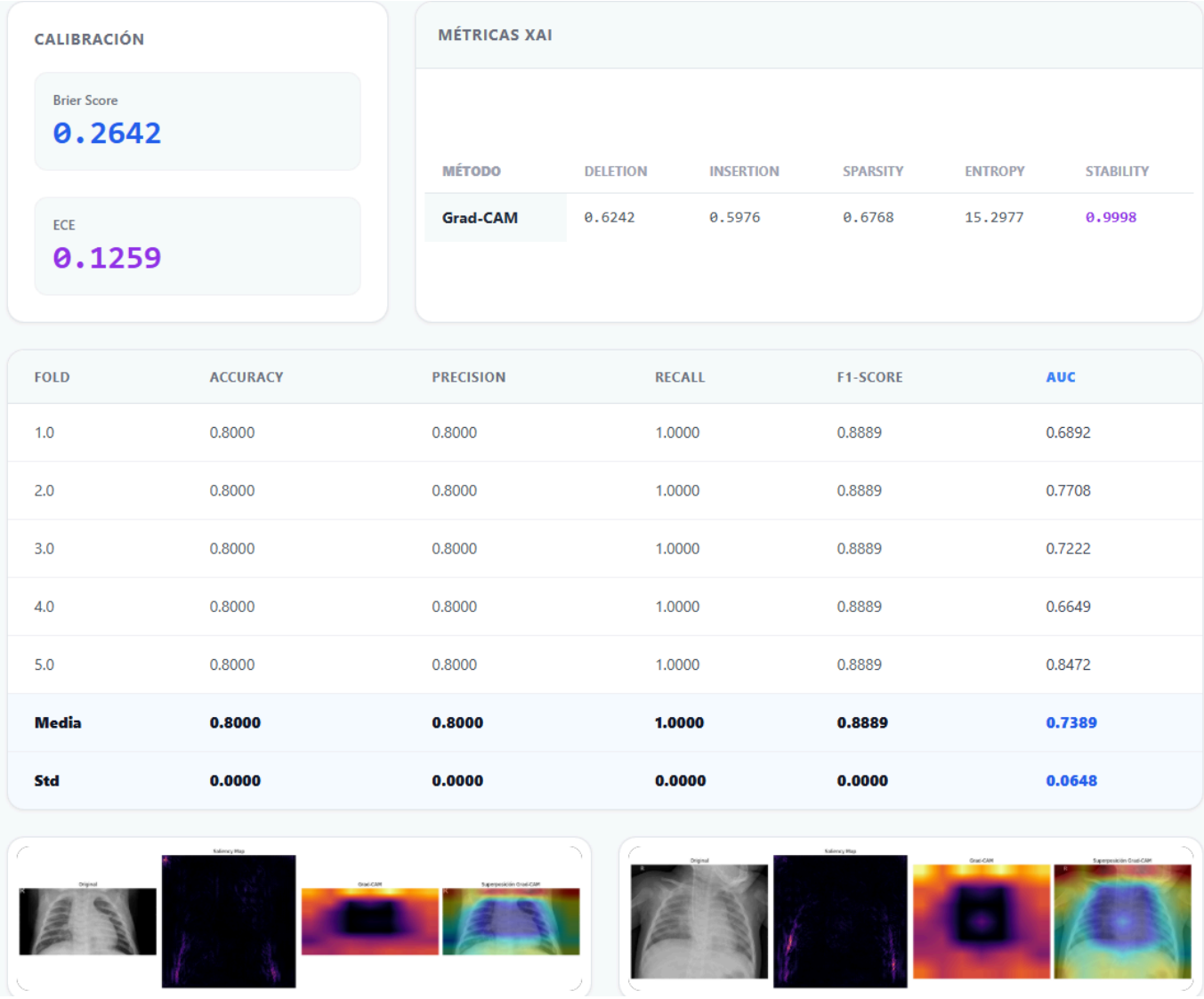


Ilustración x - Análisis cualitativo y mapas de calor MobileNetV3Large

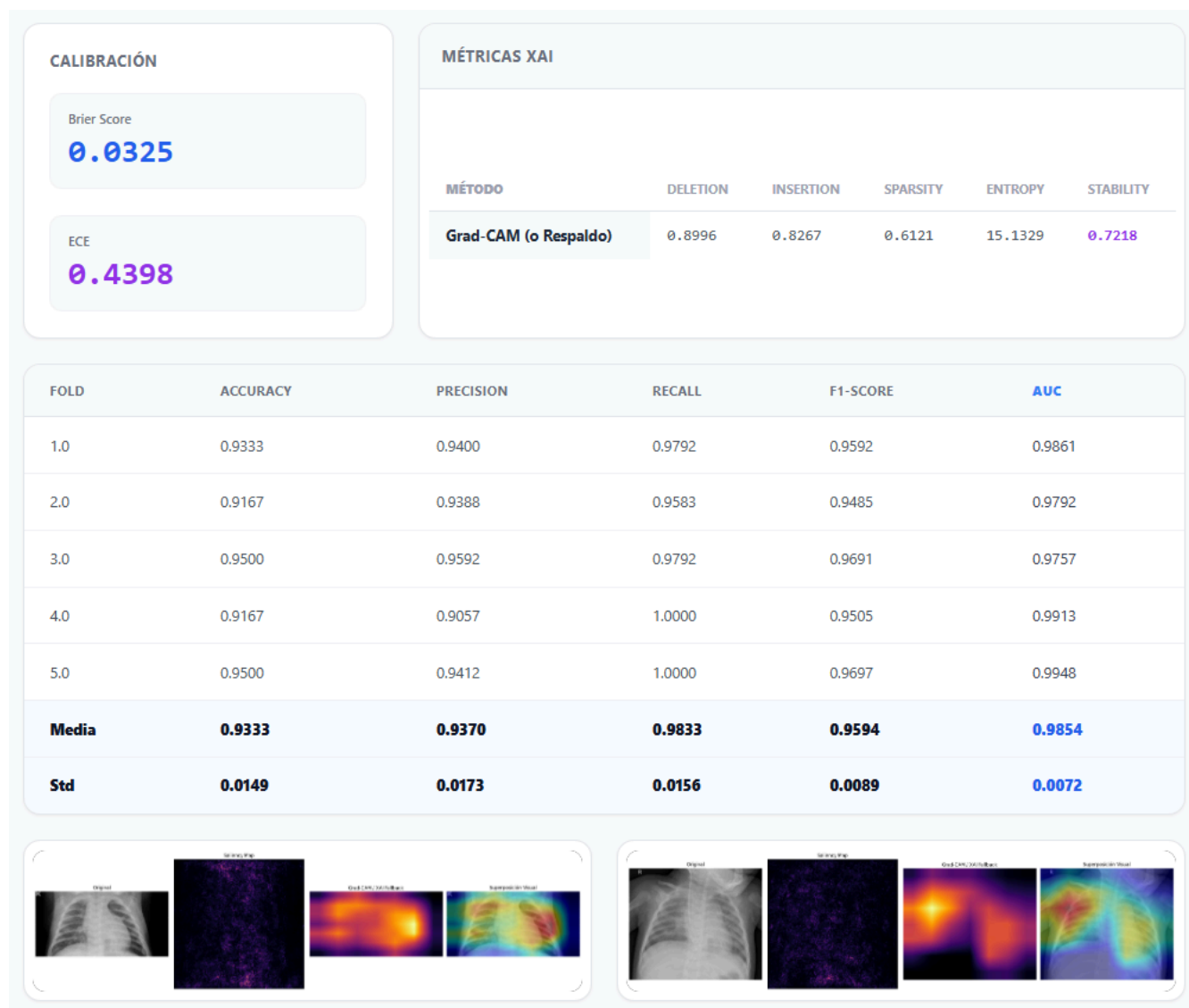


Ilustración x - Análisis cualitativo y mapas de calor MobileNetV2

1.10 Tecnologías y arquitectura del sistema

Para cerrar el capítulo, se resumen las tecnologías sobre las que se construye el sistema, que se describirán en detalle en las partes de análisis y diseño de la memoria. El backend se desarrolla con Python, el lenguaje de referencia en el ecosistema del aprendizaje profundo, y con el framework FastAPI, un framework moderno y de alto rendimiento para construir servicios web REST. La capa de persistencia se apoya en la base de datos relacional MySQL, que almacena los usuarios, el historial de consultas de diagnóstico y los experimentos de entrenamiento. El motor de aprendizaje profundo utiliza TensorFlow y Keras para las arquitecturas convolucionales y la librería HuggingFace Transformers para las arquitecturas Transformer. El asistente conversacional se implementa sobre la API de Groq con el modelo Llama 3 de Meta. El frontend se construye con HTML, CSS y JavaScript vanilla, priorizando la simplicidad y el funcionamiento sin herramientas de compilación, con una interfaz conversacional inspirada en los asistentes modernos tipo ChatGPT. El sistema se organiza en dos grandes bloques: una interfaz clínica para el diagnóstico asistido (carga de radiografías, selección de arquitectura, visualización de mapas de explicabilidad y descarga de informes PDF) y un laboratorio MLOps que automatiza el pipeline experimental descrito en la sección anterior.

2 Metas y Propósitos del Proyecto

El Trabajo Fin de Grado que se presenta surge de la necesidad de contar con una plataforma que reúna, en un mismo sistema, capacidades de diagnóstico automático mediante inteligencia artificial, mecanismos de explicabilidad que permitan a los clínicos comprender el razonamiento de los modelos, y procedimientos de validación rigurosos que garanticen la fiabilidad de los resultados obtenidos. Actualmente, los profesionales sanitarios y los investigadores se encuentran con un panorama fragmentado. Por un lado, repositorios de código y cuadernos de trabajo que implementan modelos prometedores pero que requieren conocimientos avanzados de programación para su ejecución, y que rara vez reportan de forma completa los detalles metodológicos necesarios para reproducir sus resultados (Liu & al., 2019; Varoquaux & Cheplygina, 2019). Por otro lado, plataformas comerciales cerradas que no permiten inspeccionar ni adaptar los algoritmos subyacentes. Este escenario dificulta tanto la investigación traslacional como la adopción clínica de unas tecnologías que han demostrado, en entornos controlados, un potencial extraordinario, alcanzando en algunos dominios de la imagen médica un rendimiento comparable al de los especialistas humanos (Esteva & al., 2017; De Fauw & al., 2018; McKinney & al., 2020). No obstante, la literatura científica también advierte de los riesgos de trasladar estas herramientas a la práctica clínica sin una validación rigurosa, una evaluación estadística adecuada y una documentación transparente de sus limitaciones (Nagendran & al., 2020; Kelly & al., 2019; Aggarwal & al., 2021).

Este capítulo establece los fines que persigue el proyecto. En primer lugar se define el objetivo general, la meta última que orienta todo el trabajo. A continuación se presentan los objetivos específicos, que descomponen el objetivo general en metas concretas y verificables. Estos objetivos se formulan a nivel de resultados y alcance, sin enumerar exhaustivamente todas las decisiones técnicas. La concreción de las arquitecturas, las técnicas y las decisiones de implementación se documenta posteriormente en los capítulos de contextualización, decisiones adoptadas, análisis y diseño, según corresponda.

El objetivo general de este Trabajo Fin de Grado consiste en el desarrollo de vitalXAI, una plataforma web MLOps que integre inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax. La plataforma debe ofrecer a la comunidad clínica e investigadora un entorno unificado, accesible, reproducible y rigurosamente validado que permita ejecutar diagnósticos, comparar arquitecturas de deep learning y evaluar la calidad de las explicaciones generadas por los modelos. El propósito último es poner al alcance de los profesionales sanitarios, sin necesidad de conocimientos técnicos en programación o aprendizaje automático, unas capacidades que tradicionalmente han quedado confinadas a los entornos de investigación, de forma que el potencial de la inteligencia artificial se traduzca en una herramienta realmente utilizable en la práctica clínica.

Del objetivo general se derivan los once objetivos específicos que se describen a continuación. Para facilitar su lectura, se organizan en torno a dos bloques complementarios. El primer bloque, de carácter científico-metodológico, agrupa los objetivos relativos a la fundamentación teórica, al diseño experimental y a la evaluación rigurosa de los modelos. El segundo bloque, de carácter de ingeniería web, agrupa los objetivos relativos a la construcción de la plataforma: su interfaz, su seguridad, su usabilidad y los servicios que ofrece al usuario final.

OE1 — Revisar el estado del arte. Analizar exhaustivamente la literatura científica sobre la aplicación de la inteligencia artificial al diagnóstico por imagen médica, con especial atención a la detección de neumonía mediante radiografías de tórax, a los paradigmas de inteligencia artificial explicable relevantes y a las prácticas de MLOps para la gestión del ciclo de vida de los modelos. Esta revisión, ejecutada en la tarea 0.2 del Sprint 0, tiene como finalidad fundamentar las decisiones de diseño e implementación sobre una base científica sólida y actualizada, tomando como referencia el estudio de Kermany y colaboradores (Kermany & al., 2018) y extendiendo su marco de evaluación con aspectos como la calibración de los modelos, la validación estadística y la reproducibilidad de los experimentos, dimensiones que la literatura reciente señala como imprescindibles para la fiabilidad de los sistemas de inteligencia artificial en medicina (Park & Han, 2018; Varoquaux & Cheplygina, 2019).

OE2 — Diseñar e implementar la arquitectura de persistencia. Crear una infraestructura robusta y segura que soporte el acceso de múltiples usuarios de manera concurrente y la gestión completa del historial de consultas de diagnóstico y de experimentos de entrenamiento. Esta capa debe garantizar el registro estructurado de los parámetros de cada inferencia —imagen, modelo empleado, predicción, nivel de confianza y artefactos generados— así como del histórico de sesiones de entrenamiento con sus configuraciones, métricas y resultados, permitiendo su consulta y comparación posterior.

OE3 — Diseñar e implementar el control de acceso. Desarrollar un mecanismo de autenticación, autorización y seguridad que garantice la integridad y confidencialidad de los datos de cada profesional sanitario, de modo que cada usuario disponga de un espacio de trabajo aislado desde el que gestione sus diagnósticos y resultados de manera exclusiva. Este objetivo incluye el registro de nuevos usuarios, el inicio y el cierre de sesión seguros y la protección frente a las amenazas más habituales de las aplicaciones web (OWASP, 2021), como los ataques de inyección, la falsificación de peticiones entre sitios o los intentos de acceso por fuerza bruta.

OE4 — Implementar el pipeline de entrenamiento. Automatizar el flujo completo de entrenamiento y evaluación de un conjunto amplio y representativo de arquitecturas de deep learning —tanto redes convolucionales como arquitecturas basadas en mecanismos de atención— mediante validación cruzada con particiones balanceadas. El pipeline debe automatizar el preprocesamiento de las imágenes, el entrenamiento con transferencia de conocimiento, el cálculo de las métricas de rendimiento y el almacenamiento de todos los artefactos generados, garantizando la comparabilidad de las arquitecturas mediante un diseño común de particiones, validación y métricas, con protocolos de optimización documentados según cada familia y resultados reproducibles.

OE5 — Desarrollar el módulo de explicabilidad (XAI). Implementar el módulo de inteligencia artificial explicable en sus dos vertientes complementarias. En la vertiente cualitativa, se generarán mapas visuales que permitan inspeccionar las regiones de la radiografía determinantes para cada predicción, facilitando la verificación clínica de las decisiones del modelo. En la vertiente cuantitativa, se calcularán métricas objetivas que puntúen la fidelidad de las explicaciones generadas. Adicionalmente, se incorporará la evaluación de la calibración de las predicciones —es decir, de si la confianza declarada por el modelo se corresponde con su precisión real—, un factor crítico para la fiabilidad clínica del sistema.

OE6 — Ejecutar la validación externa y el análisis estadístico. Implementar el proceso de validación externa sobre una cohorte de imágenes independiente, con el fin de evaluar la capacidad de generalización de los modelos a poblaciones, equipos de adquisición y protocolos de imagen distintos de los del entrenamiento, sin ningún tipo de reaprendizaje. Asimismo, se implementará un análisis estadístico descriptivo y, cuando el diseño muestral lo permita, inferencial, aplicando procedimientos adecuados para datos dependientes y correcciones por comparaciones múltiples tanto a los resultados de la validación cruzada como a las curvas ROC de la validación externa. Las limitaciones de tamaño muestral y dependencia entre folds deberán quedar explícitamente documentadas (Park & Han, 2018; Tang & al., 2021; Nadeau & Bengio, 2003).

OE7 — Garantizar la reproducibilidad y la trazabilidad de los experimentos. Fijar las semillas aleatorias de las librerías implicadas, almacenar la configuración completa de cada experimento y registrar los resultados y artefactos generados, de modo que cualquier resultado empírico pueda verificarse y replicarse bajo las mismas condiciones. Esta dimensión responde directamente a las carencias metodológicas que la literatura ha identificado en numerosos estudios de inteligencia artificial aplicada a la imagen médica (Varoquaux & Cheplygina, 2019).

OE8 — Desarrollar la interfaz clínica de diagnóstico asistido. Diseñar una interfaz web intuitiva, pensada para usuarios sin formación técnica en inteligencia artificial o informática, que permita a los facultativos cargar radiografías de tórax, obtener un diagnóstico con su nivel de confianza, visualizar los mapas de explicabilidad generados por el sistema y descargar un informe en formato PDF, todo ello desde un panel de control claro y sin complejidad técnica. El diseño debe priorizar la reducción de la curva de aprendizaje, siguiendo las pautas recomendadas para la interacción humano-inteligencia artificial (Amershi & al., 2019).

OE9 — Desarrollar el laboratorio MLOps y el asistente conversacional. Construir un laboratorio de entrenamiento integrado en la plataforma web, dotado de un asistente conversacional basado en grandes modelos de lenguaje que guíe al usuario, en lenguaje natural, en la configuración y el lanzamiento de experimentos de entrenamiento, eliminando por completo la necesidad de escribir código. La plataforma debe orquestar automáticamente la ejecución del pipeline experimental descrito en el OE4, el OE5 y el OE6, y poner a disposición del usuario los resultados, las comparativas y los informes generados. La elección del paradigma conversacional responde a la evidencia de que los agentes conversacionales pueden mejorar la accesibilidad de las aplicaciones de salud para usuarios no técnicos (Laranjo & al., 2018; Kocaballi & al., 2019), así como a la familiaridad que el público general tiene con este tipo de asistentes en la actualidad.

OE10 — Diseñar e implementar el procesamiento asíncrono de tareas. Crear un mecanismo de procesamiento asíncrono que permita ejecutar las tareas de larga duración —entrenamientos, análisis de explicabilidad o validación externa— sin bloquear la interfaz web. El sistema debe ofrecer monitorización del estado de cada tarea, consulta de su progreso y, cuando resulte posible, su cancelación. La gestión de las tareas de larga duración y de la complejidad operativa asociada constituye una de las dimensiones clave de la ingeniería de sistemas de aprendizaje automático (Sculley & al., 2015).

OE11 — Incorporar internacionalización. Añadir soporte multilingüe al sistema, de modo que la interfaz web, los informes generados y el asistente conversacional puedan presentarse en múltiples idiomas (español, inglés, chino e hindi). Este objetivo amplía la accesibilidad de la plataforma a entornos sanitarios internacionales.

En conjunto, estos once objetivos específicos conforman una hoja de ruta completa que abarca desde la fundamentación teórica y el diseño de la infraestructura hasta la implementación de los pipelines de entrenamiento, la evaluación de la explicabilidad, la validación estadística rigurosa y la construcción de una plataforma web segura, usable y accesible. Su articulación en dos bloques complementarios (el científico-metodológico y el de ingeniería web) garantiza que el resultado final no sea únicamente un conjunto de modelos validados, sino un sistema realmente utilizable que ponga esas capacidades al servicio de la comunidad clínica. Cada uno de estos objetivos se traduce, a lo largo de la memoria, en requisitos, casos de uso y componentes de diseño, y su cumplimiento se verifica mediante el plan de pruebas y los resultados experimentales presentados en las partes finales del documento. La administración avanzada de la plataforma queda fuera de este conjunto de objetivos y del alcance planificado del proyecto.

3 Estructura organizativa y partes interesadas del proyecto

Este capítulo describe la estructura organizativa del proyecto y las personas e instituciones que participan en él. La organización de un proyecto de las características de este —un desarrollo unipersonal de carácter multidisciplinar— exige una definición clara de quién forma parte del proyecto, qué papel desempeña cada actor y cómo se relacionan entre sí. El capítulo se estructura en tres apartados. El apartado 3.1 presenta el organigrama del proyecto, que representa gráficamente la estructura organizativa y las relaciones entre los interesados. El apartado 3.2 identifica y caracteriza a cada una de las partes interesadas, describiendo su rol general en el proyecto. Por último, el apartado 3.3 detalla las responsabilidades y funciones de cada interesado a lo largo del ciclo de vida y presenta la matriz RACI, que asigna de forma explícita y sin ambigüedades el nivel de implicación de cada parte interesada en las principales fases y entregables

3.1 Estructura organizativa del proyecto

El organigrama del proyecto, que representa la estructura organizativa de todos los interesados involucrados, se presenta en esta sección. El diagrama está compuesto por nueve nodos: Marc Ríos Cadenas, Iván Segura Carmona, Aurelio López Fernández, Luis Carmona Berdugo, el Consultor de Persistencia y Bases de Datos (Vicente de Vides Rodríguez), Rubén Pérez Garrido, el Tribunal de Evaluación del TFG, el Laboratorio Synergia y los facultativos e investigadores clínicos. Estos dos últimos actúan como entidades externas de validación y no están subordinados de manera jerárquica a ningún integrante del proyecto. Esta independencia estructural blindará el proyecto y garantiza una auditoría completamente objetiva de la plataforma.

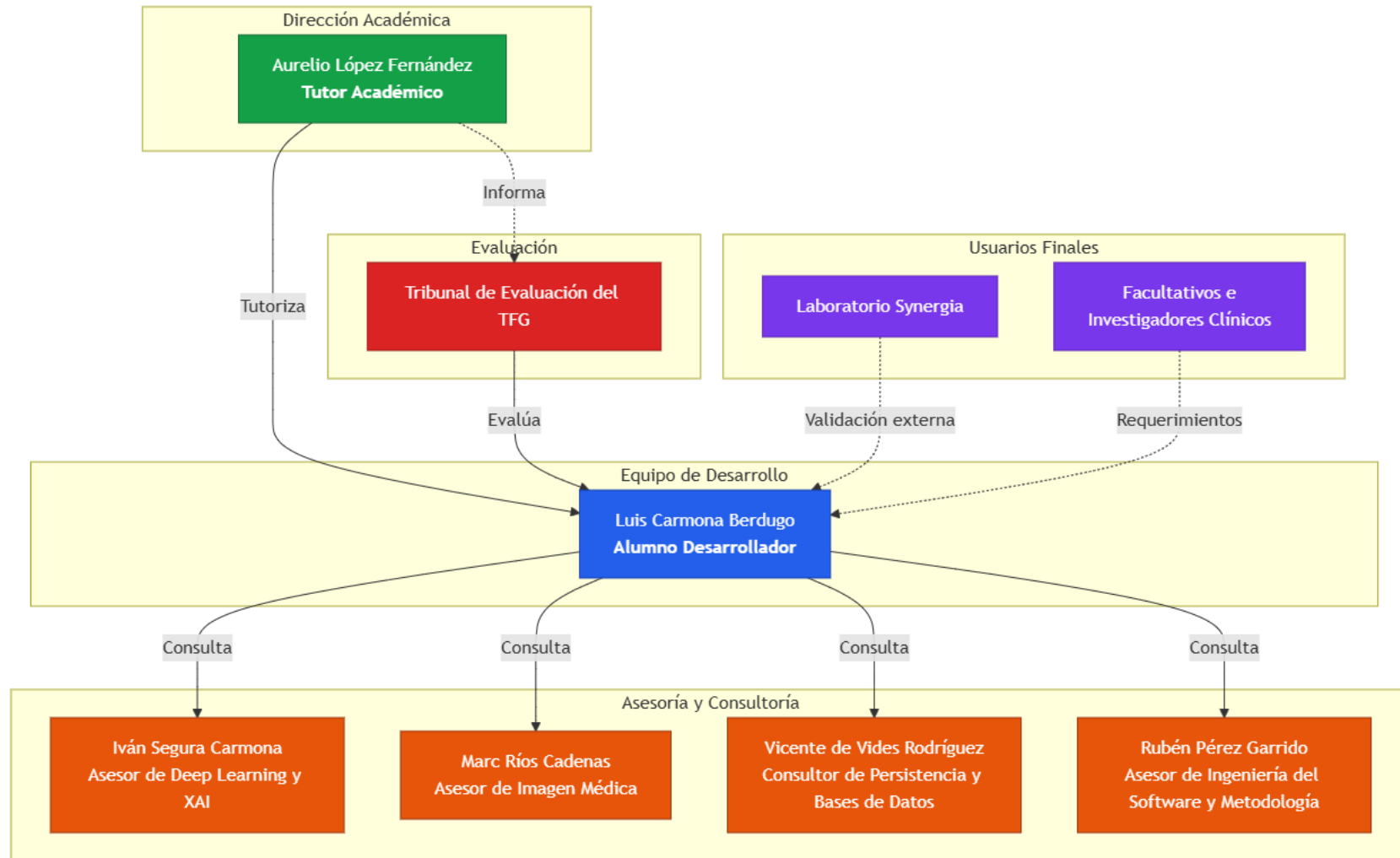


Ilustración x - Organigrama del Proyecto

3.2 Partes interesadas del proyecto

Realizar una identificación correcta de los interesados dentro de la planificación de cualquier proyecto es un paso fundamental porque permite saber quiénes son los afectados por el desarrollo, qué personas contribuyen a él y qué expectativas tienen depositadas en el resultado. En este proyecto, conviven distintos perfiles que van desde el alumno que lleva el peso técnico y documental, hasta investigadores del ámbito clínico que representan el usuario final del sistema. A continuación, se describen en detalle cada uno de los interesados junto con su rol general en el proyecto::

- **Luis Carmona Berdugo (Alumno Desarrollador):** Asume la responsabilidad íntegra del proyecto. Ejecuta y orquesta el desarrollo en su doble dimensión: técnica y documental. Él construye el sistema.
- **Aurelio López Fernández (Tutor Académico):** Dirige el rumbo de la investigación. Proporciona la brújula científica, metodológica y académica que garantiza la solvencia del Trabajo Fin de Grado.
- **Tribunal de Evaluación del TFG (Cliente / Evaluador):** Ejerce como máxima autoridad formal. Este órgano audita el proyecto, califica el trabajo y certifica si el sistema cumple los requisitos académicos exigidos. Su veredicto determina la viabilidad del resultado entregado..
- **Vicente de Vides Rodríguez (Consultor de Persistencia y Bases de Datos, Rol Consultivo):** Aporta su experiencia estructural. Resuelve consultas puntuales sobre el diseño, el modelado y la optimización de la persistencia de datos. (Asignación de especialista pendiente de confirmación).
- **Iván Segura Carmona (Asesor de Deep Learning y XAI):** Domina el núcleo algorítmico. Su consultoría especializada arroja luz sobre el comportamiento de las redes neuronales, los mecanismos de explicabilidad y la correcta interpretación de los resultados del benchmarking.
- **Marc Ríos Cadenas (Asesor de Imagen Médica):** Custodia la veracidad clínica del trabajo. Interviene de forma selectiva para asegurar que el contexto médico y radiológico del proyecto refleje la realidad hospitalaria.
- **Rubén Pérez Garrido (Asesor de Ingeniería del Software y Metodología):** experto en ingeniería del software que ha colaborado en el desarrollo de krill, una herramienta y modelo de trabajo basado en especificaciones y guiado por pruebas (SDD/TDD) que estructura el proceso de desarrollo del proyecto, y que es consultado de manera puntual sobre la metodología de desarrollo y las prácticas de ingeniería aplicadas a la implementación.
- **Laboratorio Synergia (Usuario Externo / Validador):** Representa el primer escalón de adopción. Las líneas de investigación de este equipo convergen con los objetivos de vitalXAI, convirtiéndolos en probadores empíricos de la herramienta.
- **Facultativos e investigadores clínicos (Usuarios Finales):** Constituyen el objetivo último del despliegue. Este colectivo integraría el diagnóstico asistido en su rutina médica e investigadora diaria. Sus necesidades operativas dictan la utilidad real del sistema.

3.3 Responsabilidades, funciones y participación de las partes interesadas

Realizar una definición precisa de las responsabilidades y funciones de cada uno de los interesados es esencial para garantizar que el trabajo se ejecute de manera ordenada, sin solapamientos y con una asignación clara de las principales fases y entregables. Dado que este proyecto tiene un carácter multidisciplinar, esta definición es especialmente relevante ya que las aportaciones de cada uno de los interesados se realizan en fases distintas del ciclo de vida y desde ámbitos de conocimiento muy diferentes entre sí.

Alumno Desarrollador (Luis Carmona Berdugo, LCB): Sobre él recae toda la responsabilidad principal e íntegra de la ejecución del proyecto. Desde un punto de vista científico, es la persona encargada de revisar y sintetizar el estado del arte en inteligencia artificial aplicada al diagnóstico por imagen médica, técnicas XAI y arquitecturas de deep learning, realizando una documentación de todos los hallazgos realizados. Desde el punto de vista técnico, es el encargado de diseñar la arquitectura del sistema, implementar todos los módulos de software, incluyendo los pipelines de entrenamiento para arquitecturas CNN y Transformer, los módulos de generación de explicaciones XAI y los scripts de validación externa y análisis estadístico, y también es el encargado de asegurar que los resultados sean válidos y reproducibles bajo las mismas condiciones. Tomando en cuenta el punto de vista metodológico, diseña y ejecuta el proceso de benchmarking, de manera que se garantice que las comparativas de rendimiento sean representativas en condiciones reales de uso. Desde el punto de vista de la documentación, es el encargado de elaborar toda la documentación del sistema con la calidad y profundidad exigidas para un proyecto de estas características. Por último, es el encargado de mantener la planificación del proyecto, gestionar los riesgos que puedan surgir y coordinar la comunicación con el tutor y los asesores tomando como referencia el prisma de la gestión.

Tutor Académico (Aurelio López Fernández, ALF): Es el responsable de garantizar tanto el rigor académico como el científico del proyecto en todas sus fases. Revisa y proporciona una retroalimentación detallada sobre cada uno de los entregables, señalando errores, inconsistencias y áreas de mejora con el nivel de exigencia propio de un Trabajo Fin de Grado. Orienta al alumno en las decisiones estratégicas más relevantes y actúa como puente entre el perfil técnico del desarrollo y el dominio científico del sistema.

Tribunal de Evaluación del TFG (T. TFG): La principal función de este interesado es la de evaluar el trabajo presentado con base en los criterios académicos establecidos por la universidad, que son el rigor científico, la calidad técnica, la coherencia documental y la capacidad de defensa oral del alumno. Aunque no participa de manera directa en el desarrollo, es el interesado con mayor influencia formal sobre el resultado del proyecto, ya que su valoración determinará la calificación final. Sus expectativas implícitas —claridad expositiva, aplicación de estándares de ingeniería y justificación de decisiones técnicas— se deberán tener en cuenta a lo largo de toda la documentación.

Consultor de Persistencia y Bases de Datos (Vicente de Vides Rodríguez, VVR): Su intervención es puntual y solamente de carácter consultivo. Aporta su experiencia estructural: cuando lo necesita el estudiante, proporciona criterios técnicos acerca de la eficacia de las consultas, la consistencia del modelo de persistencia con los requerimientos del sistema y el diseño, el modelado y la optimización del esquema de datos. Siempre será el estudiante desarrollador quien tome la decisión definitiva sobre cómo se diseñará e implementará la base de datos.

Asesor de Deep Learning y XAI (Iván Segura Carmona, ISC): Participa bajo un esquema puramente consultivo. Ejerce como experto de dominio. Resuelve dudas técnicas complejas acerca del funcionamiento interno de las arquitecturas algorítmicas de deep learning. Guía la correcta aplicación práctica de las técnicas de inteligencia artificial explicable. Aporta un criterio experto fundamental para interpretar correctamente los resultados del benchmarking. Analiza estas métricas desde la perspectiva estricta del rendimiento de modelos algorítmicos. La responsabilidad ejecutiva recae de nuevo sobre el alumno. El diseño arquitectónico, la implementación programada y la validación matemática de todas estas variantes algorítmicas pertenecen exclusivamente a las obligaciones del desarrollador.

Asesor de Imagen Médica (Marc Ríos Cadenas, MRC): Interviene de manera selectiva ofreciendo un asesoramiento consultivo indispensable. Actúa como experto clínico de dominio. Garantiza que el contexto médico y radiológico del proyecto quede representado con absoluta fidelidad. Ofrece contexto sobre cómo los profesionales sanitarios examinan las radiografías de tórax durante el diagnóstico hospitalario de neumonías. Define los rasgos innegociables que debe poseer la herramienta para resultar verdaderamente efectiva en la trinchera clínica. El estudiante recibe este conocimiento y asume la responsabilidad de traducirlo. Aplica este contexto médico a decisiones algorítmicas específicas, impactando directamente en el diseño estructural y en la implementación de la plataforma.

Asesor de Ingeniería del Software y Metodología (Rubén Pérez Garrido, RPG): Su participación es puntual y consultiva. Ha colaborado en el desarrollo de krill, una herramienta y modelo de trabajo basado en especificaciones y guiado por pruebas (SDD/TDD), que estructura el proceso de desarrollo del proyecto en el repositorio. Aporta criterios sobre la aplicación de la metodología, las prácticas de desarrollo guiado por pruebas y la calidad del proceso de ingeniería en las tareas de implementación. La ejecución concreta de dichas tareas y la aplicación de la metodología son responsabilidad del alumno.

Laboratorio Synergia y Facultativos e investigadores clínicos (LS/FIC): Mantienen una posición externa al núcleo de programación. No participan de manera activa en el desarrollo diario del proyecto. Representan, sin embargo, las necesidades reales del usuario final. Sus requerimientos implícitos constituyen el único criterio válido para medir la utilidad empírica del sistema desarrollado. Las líneas de investigación del Laboratorio Synergia convergen de forma natural con los objetivos de esta herramienta. El diseño del ecosistema prioriza constantemente este perfil médico. Garantizamos así que la accesibilidad visual de las interfaces y la interpretabilidad matemática de la inteligencia artificial satisfagan las exigencias de la práctica clínica real.

La Matriz RACI es una herramienta estándar de gestión de proyectos que permite hacer explícito el nivel de implicación de cada uno de los interesados en las principales fases o entregables del proyecto. Su nombre es un acrónimo de los cuatro roles que contempla: Responsable (Responsable), Accountable (Aprobador), Consulted (Consultado) e Informed (Informado). La aplicación de esta matriz es especialmente relevante en proyectos multidisciplinares como el que se está desarrollando, ya que conviven perfiles técnicos, académicos y científicos que tienen diferentes niveles de implicación, y donde una definición imprecisa de los roles podría derivar en solapamientos o en entregables sin un responsable claro.

Cada uno de los cuatro roles tiene un significado preciso dentro del contexto de la gestión de proyectos. El rol R (Responsable) identifica a la persona que realiza físicamente el trabajo, es decir, quien produce el entregable o ejecuta la tarea; en el caso de este proyecto, este rol recaerá siempre sobre el alumno desarrollador. El rol A (Accountable) asigna a la persona que tiene la autoridad final sobre ese entregable y responde por su calidad frente al resto del proyecto; a diferencia del rol anterior, solo puede existir uno por tarea, lo que evita ambigüedades en la toma de decisiones. El rol C (Consulted) corresponde a quienes, sin ejecutar ni decidir, son consultados porque poseen un conocimiento especializado relevante para la tarea; su aportación se produce antes o durante la ejecución y tiene carácter bidireccional; en el caso de este proyecto, serían los asesores y el consultor. Por último, el rol I (Informed) designa a las personas que únicamente son notificadas del avance o resultado de una tarea sin tener intervención en ella.

A continuación, se presenta la Matriz RACI del proyecto, en la que las filas recogen las principales fases y entregables del ciclo de vida del proyecto, y las columnas representan los interesados identificados en el apartado anterior.

Fase / Entregable	LCB	ALF	T. TFG	VVR	ISC	MRC	RPG	LS/FIC
Revisión del estado del arte	R	A	I	-	C	C	-	-
Diseño de la arquitectura del sistema	R	A	I	C	-	-	-	-
Diseño e implementación de la base de datos	R	A	I	C	-	-	-	-
Implementación del pipeline de entrenamiento CNN y Transformer	R	A	I	-	C	-	C	-
Implementación del módulo XAI (cualitativo y cuantitativo)	R	A	I	-	C	C	C	-
Desarrollo de la interfaz web y chatbot conversacional	R	A	I	-	-	-	C	I

Fase / Entregable	LCB	ALF	T. TFG	VVR	ISC	MRC	RPG	LS/FIC
Seguridad y control de acceso	R	A	I	-	-	-	C	-
Pruebas y verificación del sistema	R	A	I	-	C	C	C	-
Laboratorio MLOps	R	A	I	-	C	-	C	-
Procesamiento asíncrono de tareas	R	A	I	-	-	-	C	-
Internacionalización de la plataforma	R	A	I	-	-	-	C	C
Panel de administración	R	A	I	-	-	-	C	-
Gestión de riesgos	R	A	I	-	-	-	C	-
Benchmarking y análisis de resultados	R	A	I	-	C	C	-	-
Validación del contexto clínico	R	A	I	-	-	C	-	-
Validación externa y tests estadísticos	R	A	I	-	C	-	-	-
Documentación del TFG	R	A	I	-	-	-	I	-
Defensa y evaluación final	R	C	A	-	-	-	-	-

Tabla x - Matriz RACI

4 Marco Metodológico de Gestión

Todo proyecto necesita un marco de gestión que ordene su ejecución, y la elección de dicho marco debe ser coherente con la naturaleza del trabajo que se va a desarrollar. Este capítulo describe cómo se gestiona el presente proyecto: los criterios que llevaron a elegir un enfoque ágil, la forma en que se adapta el marco de trabajo Scrum a un proyecto de estas características, las herramientas de seguimiento utilizadas y el alcance de la metodología. El objetivo es que la gestión no sea un mero formalismo, sino un mecanismo que aporte orden, transparencia y capacidad de adaptación al desarrollo.

4.1 *Justificación del enfoque metodológico*

La naturaleza de este proyecto combina desarrollo de software con investigación computacional aplicada al diagnóstico por imagen médica. Esta doble naturaleza introduce un grado de incertidumbre técnica inherente que condiciona la elección de la metodología. Por un lado, el comportamiento de las arquitecturas de deep learning frente a los conjuntos de datos no siempre es predecible: los resultados del benchmarking pueden revelar que es necesario ajustar el diseño, los hiperparámetros o incluso el alcance de lo planificado, y estos ajustes no pueden anticiparse en su totalidad desde el inicio. Por otro lado, el proyecto cuenta con un único desarrollador y con una duración acotada, lo que descarta metodologías de gran peso procedimental, como Métrica V3, cuyo nivel de burocracia resultaría desproporcionado para la escala de trabajo. En la literatura de gestión de proyectos se reconoce que no existe un marco universal válido para todo: la metodología debe elegirse ponderando el tamaño del proyecto, la dinámica de sus requisitos y el entorno en el que se desarrolla (Boehm & Turner, 2004).

A la vista de estas características, se ha adoptado una metodología ágil, concretamente el framework Scrum (Schwaber & Sutherland, 2020), complementado con un tablero Kanban como herramienta de gestión visual del flujo de trabajo. Tres motivos específicos justifican esta elección frente a un enfoque tradicional en cascada. El primero es el componente exploratorio del desarrollo, particularmente en lo relacionado con la comparación de arquitecturas de deep learning y con el ajuste de sus hiperparámetros: la naturaleza iterativa de Scrum permite reorientar los esfuerzos sobre la base de los resultados obtenidos en cada ciclo, en lugar de quedar atados a una planificación rígida. El segundo motivo guarda relación con la estructura iterativa que proporciona Scrum, que posibilita detectar y corregir errores en fases tempranas, reduciendo la probabilidad de que el producto final no se ajuste a los objetivos establecidos. En tercer lugar, tanto la literatura empírica sobre métodos ágiles (Dybå & Dingsøyr, 2008) como la experiencia acumulada de su aplicación (Sutherland, 2014) evidencian que Scrum puede implementarse de manera satisfactoria en contextos de desarrollo individual, adaptando las ceremonias y los roles sin sacrificar las ventajas intrínsecas del marco de trabajo.

4.2 Roles y recursos del proyecto

4.2.1 Roles dentro del marco de trabajo

Scrum define tres roles con responsabilidades bien delimitadas. En este proyecto, al tratarse de un desarrollo unipersonal, la asignación de estos roles se adapta a la realidad del equipo:

- **Product Owner (Aurelio López Fernández):** El tutor académico asume este rol ante la ausencia de un cliente externo. Él valida personalmente las entregas, jerarquiza la importancia de los componentes algorítmicos y define los criterios de aceptación que rigen cada segmento de trabajo.
- **Scrum Master y Developer (Luis Carmona Berdugo):** Al ser un proyecto de desarrollo individual, el alumno concentra ambos roles. Como Scrum Master, es el responsable de la organización del trabajo en sprints, del mantenimiento de los artefactos del proceso y de garantizar que el proyecto avanza de manera coherente con la planificación. Como Developer, asume la totalidad de la responsabilidad técnica de la implementación.
- **Asesores científicos (consultor de persistencia y bases de datos, asesores de deep learning y XAI, de imagen médica y de ingeniería del software):** La terminología de Scrum omite esta figura. Nosotros la asimilamos al concepto de domain experts. Estos especialistas inyectan conocimiento crítico puntualmente durante las encrucijadas del desarrollo, manteniendo una posición externa sin integrarse formalmente en la célula de programación.

4.2.2 Dedicación y recursos materiales

La dedicación de los recursos humanos no es uniforme a lo largo del proyecto, sino que varía en función de la naturaleza de cada tarea y de la participación concreta de cada persona. El alumno asume la mayor parte de la ejecución, pero su porcentaje de dedicación también se ajusta a cada actividad: en las reuniones inicial y final comparte la dedicación al 50 % con el tutor, mientras que en las tareas técnicas suele asumir el 100 %. El tutor, en su papel de Product Owner, participa al 50 % en las tareas 0.1 y 8.3. Los asesores intervienen con porcentajes variables, que en el cronograma oscilan entre el 3 %, el 5 %, el 8 %, el 10 %, el 15 % y el 25 %, según la tarea y el tipo de revisión o consulta requerida. Estas proporciones son las que sustentan el cálculo de horas y costes del apartado 5.3; no existe, por tanto, una dedicación uniforme de los asesores.

En cuanto a los recursos materiales, el proyecto requiere un equipo con capacidad de cómputo GPU para el entrenamiento de los modelos de deep learning, particularmente durante las fases de validación cruzada de las arquitecturas CNN y Transformer. Se emplea una estación de trabajo local equipada con una GPU NVIDIA compatible con CUDA, cuyo coste de uso está incluido en la infraestructura hardware disponible para el desarrollo del TFG, sin necesidad de recurrir a instancias de computación en la nube. Todo el software utilizado es de código abierto y gratuito, por lo que no se incurre en costes de licenciamiento.

4.3 Ciclo de trabajo iterativo

4.3.1 Los sprints y la cadencia del trabajo

El evento central del marco Scrum es el Sprint, entendido en este proyecto como un ciclo iterativo de trabajo con un objetivo y un incremento verificable. La aplicación es una adaptación de Scrum a un TFG unipersonal: los nueve sprints tienen duración desigual, porque la planificación real reserva más tiempo a la fase inicial, a los bloques técnicos de mayor complejidad y a la documentación final. Por tanto, no se afirma que exista una cadencia temporal fija ni que se aplique Scrum de forma ortodoxa; se conservan sus elementos útiles de planificación, revisión, retrospectiva, backlog e incremento, ajustados a las restricciones académicas y al volumen variable de cada bloque. Cada sprint procura producir un incremento potencialmente entregable, integrando diseño, implementación y pruebas cuando la naturaleza de la tarea lo permite. Las pruebas se realizan de forma continua e incremental, incluyendo las pruebas de seguridad en los mismos sprints en que se desarrollan los componentes afectados.

La ejecución de cada tarea de implementación se apoya, además, en el modelo de desarrollo basado en especificaciones y guiado por pruebas (SDD/TDD), diseñado con la colaboración del asesor de ingeniería del software y descrito en el capítulo de organización. Este modelo organiza el trabajo de programación en ciclos en los que primero se define la especificación de la funcionalidad, después se escriben las pruebas que deben satisfacerla y, finalmente, se implementa el código necesario para que esas pruebas pasen y se refactoriza el resultado (Beck, 2000). Esta práctica garantiza que cada funcionalidad quede cubierta por pruebas automatizadas y contribuye a la calidad y a la reproducibilidad del desarrollo.

4.3.2 Ceremonias y reuniones de seguimiento

Cada Sprint se organiza, cuando la fase y la disponibilidad de los participantes lo permiten, alrededor de las actividades adaptadas de planificación, revisión y retrospectiva. El alumno desglosa los elementos del Product Backlog en tareas concretas y utiliza el tablero para realizar el seguimiento. Las revisiones con el tutor se celebran en los hitos previstos y mediante reuniones puntuales, sin asumir dieciocho ceremonias formales con dedicación presupuestada. La retrospectiva se realiza como revisión del proceso al cierre de cada bloque cuando resulta aplicable.

En este proyecto no se realiza ninguna Daily Scrum, ya que no resulta operativo en un contexto de desarrollo individual. Para cubrir la función de estas reuniones diarias se realizan reuniones periódicas de seguimiento con el tutor y, cuando es necesario, con los asesores. En estas reuniones el alumno presenta los avances realizados, expone las dudas técnicas surgidas, recibe retroalimentación sobre los resultados obtenidos y acuerda las tareas a acometer hasta el siguiente encuentro. El registro de estas reuniones queda recogido en el Anexo de Seguimiento del Proyecto, que actúa como bitácora formal del proceso a lo largo del ciclo de vida del desarrollo.

La ejecución del proyecto no ha seguido un calendario artificialmente uniforme, sino la cadencia real que impone un trabajo académico con un único desarrollador. El proyecto se inició a finales de noviembre de 2025. Durante el periodo comprendido entre ese momento y finales de febrero de 2026, la actividad se limitó a un par de reuniones de seguimiento con el tutor y a una pequeña inicialización del marco tecnológico: la selección de las librerías, la preparación del entorno y el arranque del repositorio. A partir de principios de marzo de 2026 comenzó el desarrollo completo

del sistema, que se prolongó hasta principios de junio de 2026, con algunas reuniones puntuales —pocas, pero decisivas— con el tutor y, cuando fue necesario, con los asesores, para revisar los avances y acordar los pasos siguientes. Desde principios de junio de 2026 hasta septiembre de 2026 se desarrolló la documentación completa del proyecto, junto con el benchmarking final, las correcciones y la entrega, de nuevo con algunas reuniones de seguimiento puntuales. Esta distribución temporal, lejos de ser un detalle administrativo, refleja la naturaleza real del trabajo: la fase de implementación concentró la aplicación efectiva de los sprints, mientras que las fases inicial y final fueron más intensivas en documentación, revisión y cierre.

4.3.3 Artefactos del proceso de gestión

El proceso de gestión se apoya en tres artefactos definidos por Scrum. El Product Backlog es la lista ordenada y priorizada de todo el trabajo pendiente: los requisitos funcionales y no funcionales del sistema, las tareas de implementación de los pipelines de entrenamiento para arquitecturas CNN y Transformer, los módulos de generación de explicaciones XAI, los experimentos de benchmarking planificados y los documentos del TFG pendientes de elaboración. Es gestionado por el alumno y revisado de manera conjunta con el tutor en cada Sprint Planning. El Sprint Backlog es el subconjunto de elementos del Product Backlog seleccionados para el Sprint en curso, desglosados en tareas concretas y estimables que guían el trabajo durante el ciclo. El Increment es el conjunto de todos los elementos completados durante el Sprint, que deben cumplir la Definition of Done —el criterio acordado que define cuándo una tarea puede considerarse terminada— antes de ser presentados en el Sprint Review.

4.4 Seguimiento visual del trabajo con Kanban

Además del marco Scrum, se utiliza un tablero Kanban, gestionado a través de GitHub Projects, como instrumento visual para administrar el flujo de trabajo. La elección de GitHub Projects responde a que el repositorio del proyecto está alojado en GitHub, lo que permite asociar de manera directa las tareas del tablero a los commits, pull requests e issues del código, unificando toda la trazabilidad del desarrollo en una única plataforma.

El tablero se organiza en cuatro columnas que representan los estados por los que transita cada tarea: Por Hacer (To Do), para las tareas planificadas y aún no iniciadas; En Progreso (In Progress), para las tareas en desarrollo activo; En Revisión (In Review), para las tareas completadas por el alumno y pendientes de validación por el tutor; y Hecho (Done), para las tareas validadas que cumplen la Definition of Done.

La combinación de Scrum y Kanban, conocida en la literatura como Scrumban, permite aprovechar la estructura iterativa y los ritmos de inspección de Scrum junto con la visibilidad continua del flujo de trabajo que proporciona el tablero (Kniberg & Skarin, 2010; Reddy, 2016). El tablero Kanban aporta, por sí mismo, un mecanismo de gestión del flujo que ayuda a limitar el trabajo en curso y a detectar cuellos de botella (Anderson, 2010), algo especialmente valioso en un proyecto unipersonal en el que la capacidad de autogestión es fundamental.

4.5 Alcance de la metodología

El marco de gestión descrito en este capítulo se aplica exclusivamente al trabajo comprendido en este Plan de Proyecto, es decir, a la concepción, la implementación y la documentación de vitalXAI hasta su entrega. No gobierna, en cambio, las actividades que solo tendrían sentido si la plataforma pasara a explotarse de forma continuada —el mantenimiento del software, el soporte a usuarios, la actualización de los modelos o su despliegue en un centro sanitario—, que quedan fuera del alcance de un trabajo académico con un plazo de entrega fijado. La planificación tiene, por tanto, un horizonte claro: la finalización y la defensa de la memoria. Las posibles líneas de evolución futura de la plataforma se describen en la memoria como trabajo futuro, sin que ello amplíe el alcance de lo planificado ni comprometa la estimación de recursos de este documento. En la práctica, y de acuerdo con el calendario real descrito en el apartado 4.3.2, la metodología se aplicó de forma plena durante la fase de desarrollo activo, mientras que las fases inicial y final, más centradas en la planificación y en la documentación, se apoyaron en las reuniones de seguimiento y en el tablero Kanban más que en la cadencia formal de los sprints.

5 Cronograma y Plan de Trabajo

5.1 *Marco general y objetivos de la planificación*

Este capítulo recoge el programa de trabajo del proyecto: el marco temporal en el que se desarrolla, el desglose de las tareas en sprints y la asignación de recursos y responsabilidades que lo sustentan. El punto de partida es vitalXAI, la plataforma web MLOps que integra inteligencia artificial explicable para el diagnóstico asistido de neumonía mediante radiografías de tórax, definida en el capítulo de metas y propósitos. Sobre esa base, la planificación debe dar respuesta a los once objetivos específicos del proyecto, organizados en los bloques científico-metodológico y de ingeniería web: desde la revisión del estado del arte hasta la construcción de la interfaz clínica, el laboratorio MLOps, la seguridad y la internacionalización.

El programa de trabajo cubre el ciclo completo de concepción, implementación y documentación del sistema, y tiene un horizonte claro: la entrega y defensa del Trabajo Fin de Grado el 2 de septiembre de 2026. La ejecución se ha orquestado siguiendo la metodología ágil Scrum descrita en el capítulo de gestión, adaptada a la realidad de un proyecto unipersonal: la cadencia formal de los sprints se aplicó de forma plena durante la fase de implementación, mientras que las fases de planificación y de documentación se apoyaron en reuniones periódicas de seguimiento y en el tablero Kanban.

De acuerdo con el calendario real del proyecto, la planificación se articula en tres fases. La fase inicial, de finales de noviembre de 2025 a finales de febrero de 2026, estableció las bases organizativas, técnicas y documentales. La fase de desarrollo, de principios de marzo a principios de junio de 2026, concentró la implementación del sistema en siete sprints. Por último, la fase de documentación, desde principios de junio hasta septiembre de 2026, se dedicó a la elaboración de la memoria, los manuales y el cierre del proyecto.

El esfuerzo total planificado asciende a 474 horas, distribuidas en nueve sprints de duración desigual que reflejan la carga real de cada bloque de trabajo: la infraestructura y el laboratorio MLOps son los bloques más pesados, mientras que la validación externa es el más contenido. Las horas de las tareas representan esfuerzo estimado, mientras que las duraciones del diagrama de Gantt representan ventanas de calendario; ambas magnitudes no se convierten mediante una jornada diaria uniforme. La dedicación efectiva varía según la fase, la naturaleza de las tareas y la participación de los asesores, tal como se detalla en la asignación de recursos del apartado 5.3. Por tanto, el cronograma debe interpretarse como una planificación temporal y de esfuerzo, no como una deducción de horas diarias constantes.

5.2 Desglose de tareas por Sprint

El trabajo se estructura en nueve sprints, cada uno con un propósito definido y un incremento verificable del sistema. Los sprints de desarrollo se conciben como ciclos cerrados de dos semanas de duración, y entre ellos se impone una secuencialidad estricta: ninguno comienza sin que el anterior haya cerrado formalmente su revisión y su retrospectiva. Esta estructura encaja con el calendario real descrito en el capítulo de gestión: el primer sprint corresponde a la fase de planificación, los siete siguientes a la fase de desarrollo y el último a la documentación y el cierre.

El camino crítico del proyecto discurre por las tareas de mayor carga técnica y menor capacidad de paralelización: la implementación del pipeline de entrenamiento de las redes convolucionales (Sprint 4) y su prolongación en el pipeline Transformer (Sprint 5), que deben completarse antes de ejecutar el benchmarking final (Sprint 8). Ninguna de estas tareas puede iniciarse sin que la anterior haya concluido, y su ejecución concentra el mayor riesgo de desviación cronológica de todo el proyecto. En la siguiente figura se representa la estructura de desglose del trabajo (EDT), elaborada manualmente.

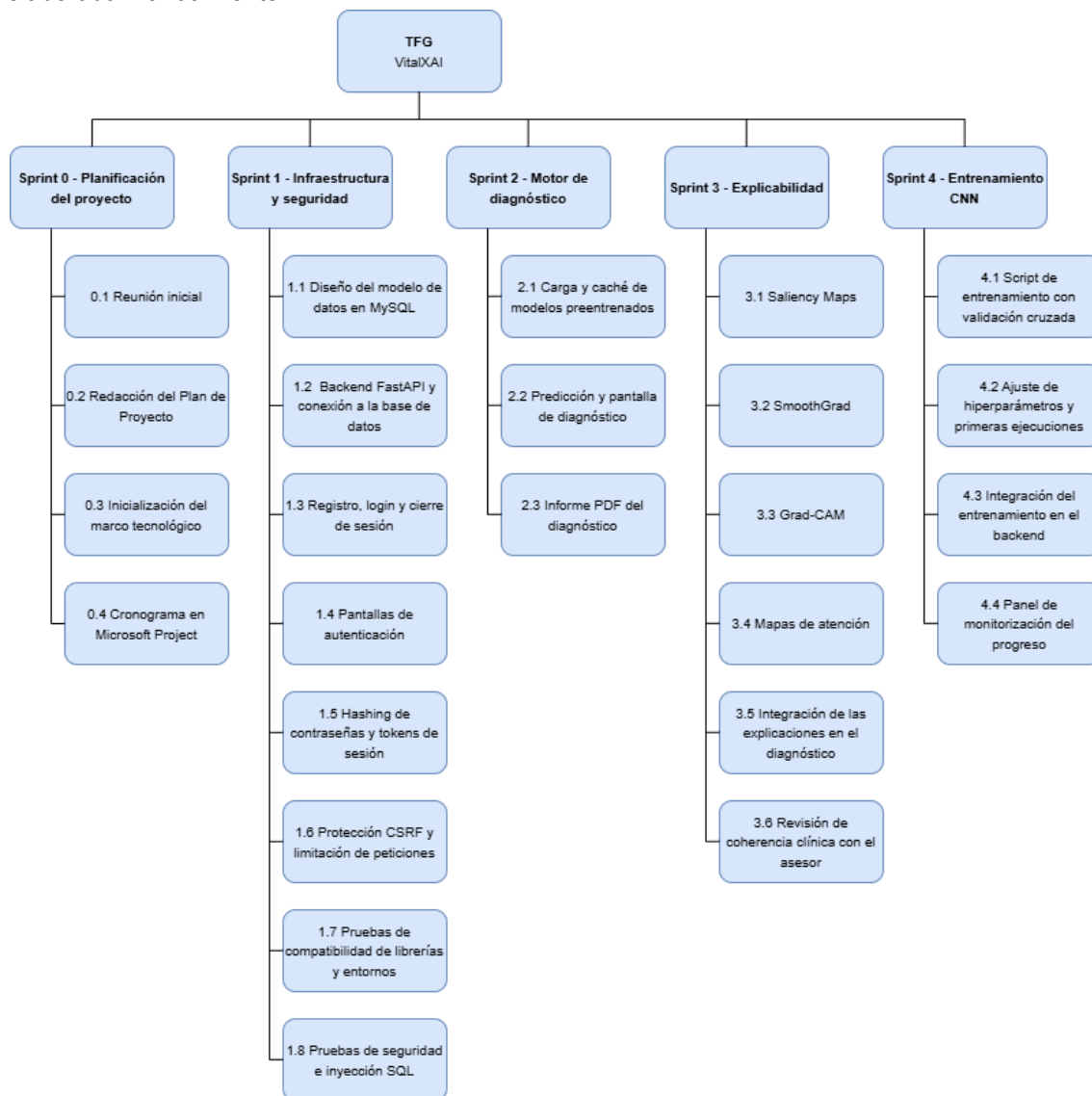


Ilustración x - Diagrama EDT I

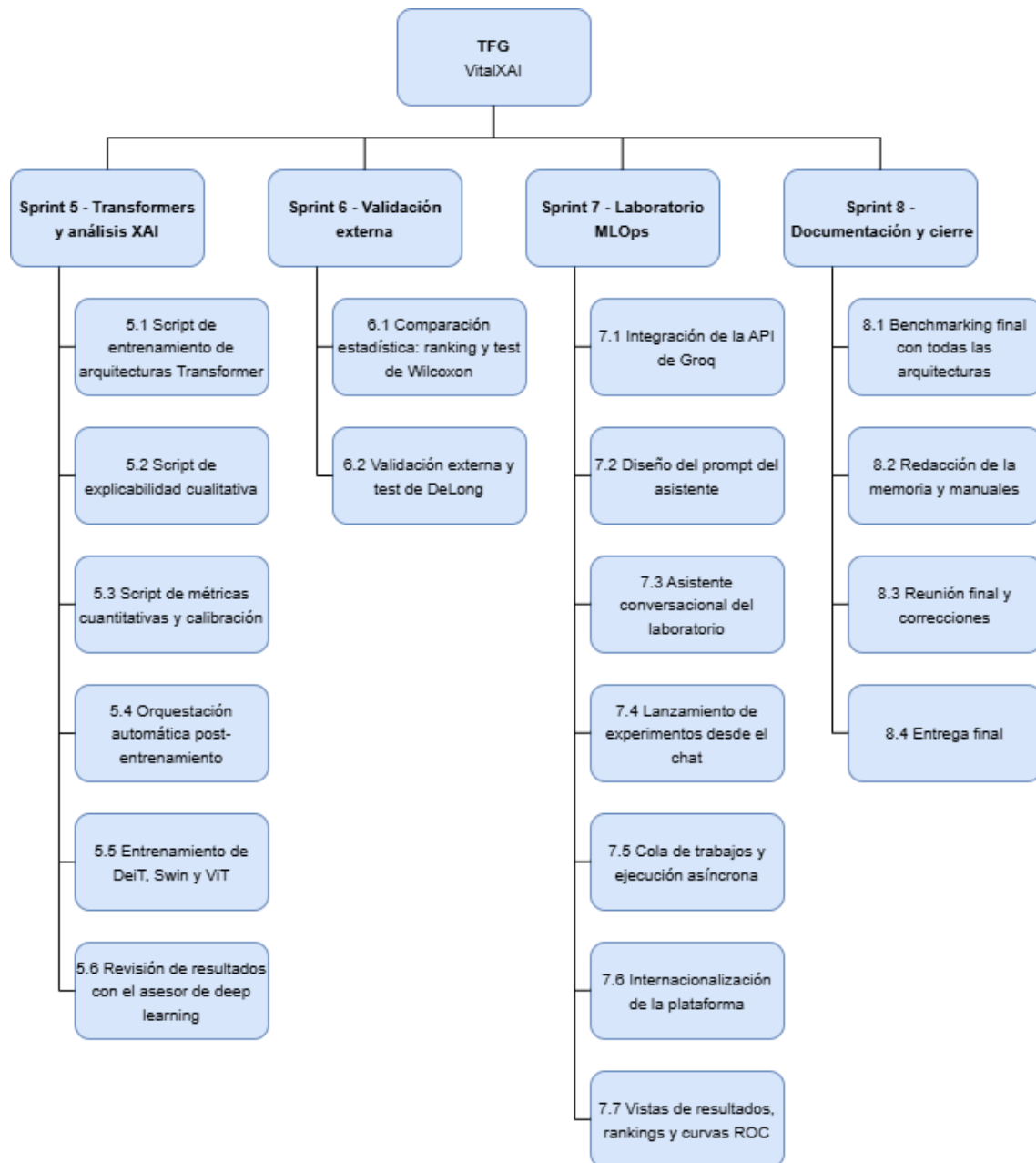


Ilustración x - Diagrama EDT II

5.2.1 Sprint 0 – Planificación del proyecto

Este sprint inicial sienta las bases del proyecto: no hay código, solo la preparación organizativa, técnica y documental. Ocupa la fase de planificación, de finales de noviembre de 2025 a finales de febrero de 2026, con una dedicación ligera y 42 horas de esfuerzo repartidas en cuatro tareas. En este sprint se realiza también la revisión bibliográfica que fundamenta el estado del arte y las decisiones metodológicas posteriores.

Código	Nombre	Responsable(s)	Estimación (h)
0.1	Reunión inicial	Luis Carmona Berdugo, Aurelio López Fernández	3
0.2	Redacción del Plan de Proyecto y revisión bibliográfica	Luis Carmona Berdugo	25
0.3	Inicialización del marco tecnológico	Luis Carmona Berdugo	8
0.4	Cronograma en Microsoft Project	Luis Carmona Berdugo	6

Tabla x - Recursos y estimaciones Sprint 0

Tarea 0.1 – Reunión inicial: encuentro con el tutor para fijar objetivos, alcance, tecnologías (FastAPI, TensorFlow, MySQL y Groq) y metodología.

Tarea 0.2 – Redacción del Plan de Proyecto y revisión bibliográfica: el alumno redacta el Plan de Proyecto, el documento marco que integra la definición de los objetivos, la organización del proyecto, la metodología de gestión, el plan de tareas y la evaluación de riesgos. Esta tarea también fija el marco documental y metodológico sobre el que se desarrolla la revisión del estado del arte. Es la tarea más extensa de este sprint y justifica sus veinticinco horas: sobre ella se sustentan los capítulos de contextualización, organización, metodología y riesgos de esta memoria. La redacción no es un mero trámite administrativo, sino el ejercicio de concreción que da forma al alcance del sistema y a su planificación. El resultado guía la ejecución de todos los sprints posteriores y sirve de contrato frente al tutor y al tribunal.

Tarea 0.3 – Inicialización del marco tecnológico: se fijan las versiones de las librerías, se preparan los entornos virtuales de Anaconda y se crea el repositorio. Esta tarea refleja la pequeña inicialización del marco tecnológico de la fase de planificación y blinda el entorno frente a las incompatibilidades de dependencias (riesgo R02).

Tarea 0.4 – Cronograma en Microsoft Project: se construye el diagrama de Gantt con las tareas, sus dependencias, los recursos y el camino crítico, que servirá de referencia para el seguimiento.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
■ 0. Planificación del Proyecto	70 días	lun 24/11/25	vie 27/02/26		42 horas	
0.1 Reunión Inicial	1 día	lun 24/11/25	lun 24/11/25	Luis Carmona Berdugo[50%];Aurelio López Fernández[50%]	3 horas	
0.2 Redacción del Plan de Proyecto	49 días	mar 25/11/25	vie 30/01/26	Luis Carmona Berdugo	25 horas	3
0.3 Inicialización del marco tecnológico	10 días	lun 02/02/26	vie 13/02/26	Luis Carmona Berdugo	8 horas	4
0.4 Cronograma en Microsoft Project	10 días	lun 16/02/26	vie 27/02/26	Luis Carmona Berdugo	6 horas	5

Ilustración x - Sprint 0 en Project

5.2.2 Sprint 1 – Infraestructura y seguridad

El primer sprint de desarrollo materializa el incremento funcional inaugural del sistema y es, junto con el laboratorio MLOps, el bloque de mayor carga de todo el proyecto, con 70 horas de esfuerzo. El código cobra vida: se construye el backend sobre FastAPI, se despliega la base de datos relacional MySQL que asume la persistencia de la información, se implementa el módulo de autenticación y se sientan las bases de la seguridad transversal. Su amplitud responde a que en este sprint se concentra, además de la construcción de la infraestructura, su blindaje completo: el cifrado de contraseñas, la gestión de sesiones y tokens, la protección frente a falsificación de peticiones, la limitación de acceso, las pruebas de compatibilidad de librerías sobre los entornos de Anaconda y las pruebas de seguridad. Con ocho tareas, es el sprint con más entregables del cronograma. Se desarrolla durante las dos primeras semanas de marzo de 2026 y deja la plataforma lista para recibir el motor de inferencia.

Código	Nombre	Responsable(s)	Estimación (h)
1.1	Diseño del modelo de datos en MySQL	Luis Carmona Berdugo, Vicente de Vides Rodríguez	10
1.2	Backend FastAPI y conexión a la base de datos	Luis Carmona Berdugo	14
1.3	Registro, login y cierre de sesión	Luis Carmona Berdugo	12
1.4	Pantallas de autenticación	Luis Carmona Berdugo	8
1.5	Hashing de contraseñas y tokens de sesión	Luis Carmona Berdugo	6
1.6	Protección CSRF y limitación de peticiones	Luis Carmona Berdugo	6
1.7	Pruebas de compatibilidad de librerías y entornos	Luis Carmona Berdugo	6
1.8	Pruebas de seguridad e inyección SQL	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 1

Tarea 1.1 – Diseño del modelo de datos en MySQL: se diseña el esquema relacional del sistema —usuarios, consultas de diagnóstico, sesiones de entrenamiento, cola de trabajos y tokens de refresco—. El consultor de persistencia y bases de datos aporta su criterio sobre el modelado y la optimización del esquema.

Tarea 1.2 – Backend FastAPI y conexión a la base de datos: se monta el backend con FastAPI y se configura la conexión a MySQL mediante un pool de conexiones, lo que permite atender el acceso concurrente de múltiples usuarios sin degradar el rendimiento. El renderizado de las páginas con Jinja2 y el servidor Uvicorn quedan operativos al cierre de esta tarea. Sobre este esqueleto se construyen todos los endpoints posteriores del sistema.

Tarea 1.3 – Registro, login y cierre de sesión: se implementan los flujos de creación de cuenta, inicio de sesión y cierre de sesión. Las contraseñas se cifran con bcrypt y la sesión se gestiona mediante tokens JWT almacenados en cookies seguras, con tokens de refresco y su rotación en cada uso.

Tarea 1.4 – Pantallas de autenticación: se construyen las interfaces de login y registro con el sistema de estilos de Tailwind. El flujo completo de acceso se verifica de forma integral, desde el formulario hasta la entrada al panel.

Tarea 1.5 – Hashing de contraseñas y tokens de sesión: se integra la capa de cifrado de contraseñas y la emisión y validación de los tokens de sesión y de refresco.

Tarea 1.6 – Protección CSRF y limitación de peticiones: se incorpora el middleware de doble cookie frente a la falsificación de peticiones y la limitación de intentos para frenar los ataques por fuerza bruta.

Tarea 1.7 – Pruebas de compatibilidad de librerías y entornos: se ejecutan pruebas sobre los entornos virtuales de Anaconda para garantizar que las versiones de TensorFlow y Keras conviven sin conflictos antes de construir el motor de inferencia (riesgo R02).

Tarea 1.8 – Pruebas de seguridad e inyección SQL: se validan los mecanismos de autenticación y autorización y se prueban los endpoints frente a inyección SQL y exposición de datos sensibles.

Nombre de tarea	Duración	Comienzo	Fin	Nombres de los recursos	Trabajo	Predecesoras
1. Infraestructura y Seguridad	12 días	lun 02/03/26	mar 17/03/26		70 horas	
1.1 Diseño del modelo de datos en MySQL	2 días	lun 02/03/26	mar 03/03/26	Luis Carmona Berdugo[85%];Vicente de Vides Rodríguez[15%]	10 horas	6
1.2 Backend FastAPI y conexión a la BD	2 días	mié 04/03/26	jue 05/03/26	Luis Carmona Berdugo	14 horas	8
1.3 Registro, login y cierre de sesión	2 días	vie 06/03/26	lun 09/03/26	Luis Carmona Berdugo	12 horas	9
1.4 Pantallas de autenticación	2 días	lun 09/03/26	mar 10/03/26	Luis Carmona Berdugo	8 horas	
1.5 Hashing de contraseñas y tokens de sesión	3 días	mar 10/03/26	jue 12/03/26	Luis Carmona Berdugo	6 horas	
1.6 Protección CSRF y limitación de peticiones	2 días	mié 11/03/26	vie 13/03/26	Luis Carmona Berdugo	6 horas	
1.7 Pruebas de compatibilidad de librerías y entorno	2 días	jue 12/03/26	lun 16/03/26	Luis Carmona Berdugo	6 horas	11
1.8 Pruebas de seguridad e inyección SQL	2 días	lun 16/03/26	mar 17/03/26	Luis Carmona Berdugo	8 horas	

Ilustración x - Sprint 1 en Project

5.2.3 Sprint 2 – Motor de diagnóstico

Este sprint construye el núcleo del diagnóstico clínico: la carga de los modelos preentrenados, el endpoint de predicción y la interfaz que permite al facultativo subir una radiografía y ver el resultado. Con 41 horas en tres tareas, cierra la primera versión funcional del flujo de diagnóstico en la segunda mitad de marzo.

Código	Nombre	Responsable(s)	Estimación (h)
2.1	Carga y caché de modelos preentrenados	Luis Carmona Berdugo	15
2.2	Predicción y pantalla de diagnóstico	Luis Carmona Berdugo	18
2.3	Informe PDF del diagnóstico	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 2

Tarea 2.1 – Carga y caché de modelos preentrenados: se desarrolla el motor de carga de los modelos CNN y Transformer desde sus pesos. Un sistema de caché en memoria evita recargar el modelo en cada petición, de modo que el primer diagnóstico de cada arquitectura carga los pesos y los siguientes son instantáneos. Esta optimización es clave para la experiencia del facultativo en la consulta.

Tarea 2.2 – Predicción y pantalla de diagnóstico: se implementa el endpoint de predicción y la pantalla de diagnóstico en la que el facultativo sube la radiografía, selecciona la arquitectura y obtiene el diagnóstico con su nivel de confianza. El procesamiento se encola de forma asíncrona para no bloquear la interfaz durante la inferencia. Es la tarea central de este sprint: sobre ella se apoyan los mapas de explicabilidad del sprint siguiente. La pantalla presenta el resultado de forma clara y sin tecnicismos, pensando en un usuario sin formación técnica.

Tarea 2.3 – Informe PDF del diagnóstico: se implementa la generación del informe PDF que recoge la radiografía, el diagnóstico, el nivel de confianza y el modelo empleado, listo para su descarga y archivo.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
2. Motor de diagnóstico	10 días	mié 18/03/21	mar 31/03/26		41 horas	
2.1 Carga y caché de modelos preentrenados	3 días	mié 18/03/21	vie 20/03/26	Luis Carmona Berdugo	15 horas	15
2.2 Predicción y pantalla de diagnóstico	3 días	lun 23/03/26	mié 25/03/26	Luis Carmona Berdugo	18 horas	17
2.3 Informe PDF del diagnóstico	4 días	jue 26/03/26	mar 31/03/26	Luis Carmona Berdugo	8 horas	18

Ilustración x - Sprint 2 en Project

5.2.4 Sprint 3 – Explicabilidad

Este sprint incorpora la inteligencia artificial explicable al sistema. Se implementan las tres familias de técnicas visuales —Saliency Maps, SmoothGrad y Grad-CAM, además de los mapas de atención para los Transformers— y se integran en el flujo de diagnóstico, de modo que cada predicción pasa a ser una decisión verificable. Con 48 horas repartidas en seis tareas, el sprint se desarrolla a lo largo de las dos primeras semanas de abril. La revisión del asesor de imagen médica cierra el bloque garantizando la coherencia clínica de las explicaciones.

Código	Nombre	Responsable(s)	Estimación (h)
3.1	Saliency Maps	Luis Carmona Berdugo, Iván Segura-Carmona	10
3.2	SmoothGrad	Luis Carmona Berdugo	8
3.3	Grad-CAM	Luis Carmona Berdugo, Iván Segura-Carmona	12
3.4	Mapas de atención	Luis Carmona Berdugo	8
3.5	Integración de las explicaciones en el diagnóstico	Luis Carmona Berdugo	6
3.6	Revisión de coherencia clínica con el asesor	Luis Carmona Berdugo, Marc Ríos Cadenas	4

Tabla x - Recursos y estimaciones Sprint 3

Tarea 3.1 – Saliency Maps: se implementa el mapa de prominencia que calcula el gradiente de la clase predicha respecto a cada píxel, resaltando las regiones más influyentes en la decisión del modelo. La revisión del asesor de deep learning y XAI valida la corrección del cálculo.

Tarea 3.2 – SmoothGrad: se implementa la variante que promedia los mapas de saliencia sobre versiones con ruido gaussiano. El resultado es un mapa más suave y estable, en el que los patrones consistentes del modelo destacan sobre el ruido.

Tarea 3.3 – Grad-CAM: se implementan los mapas de activación de clase a partir de la última capa convolucional, que superponen a la radiografía un mapa de calor grueso y fácil de interpretar. Su suavidad y su correspondencia con las regiones anatómicas lo convierten en la técnica más valiosa para el profesional clínico. Se valida su correcta implementación con el asesor de deep learning y XAI.

Tarea 3.4 – Mapas de atención: se implementa la extracción de los pesos de atención de la última capa de los Transformers, promediando las cabezas de atención para obtener el mapa de relevancia de cada parche.

Tarea 3.5 – Integración de las explicaciones en el diagnóstico: se integra la generación de los mapas en el flujo de predicción, de modo que cada consulta genera automáticamente sus explicaciones.

Tarea 3.6 – Revisión de coherencia clínica con el asesor: el asesor de imagen médica comprueba que los mapas apuntan a las regiones pulmonares relevantes y no a artefactos.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
4 3. Explicabilidad	8 días	mié 01/04/21	vie 10/04/26		48 horas	
3.1 Saliency Maps	2 días	mié 01/04/21	jue 02/04/26	Luis Carmona Berdugo[90%];Iván Segura Carmona[10%]	10 horas	19
3.2 SmoothGrad	1 día	vie 03/04/26	vie 03/04/26	Luis Carmona Berdugo	8 horas	21
3.3 Grad-CAM	2 días	lun 06/04/26	mar 07/04/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	22
3.4 Mapas de atención	1 día	mié 08/04/21	mié 08/04/26	Luis Carmona Berdugo	8 horas	23
3.5 Integración de las explicaciones en el diagnóstico	1 día	jue 09/04/26	jue 09/04/26	Luis Carmona Berdugo	6 horas	24
3.6 Revisión de coherencia clínica con asesor	1 día	vie 10/04/26	vie 10/04/26	Luis Carmona Berdugo[75%];Marc Ríos Cadena[25%]	4 horas	25

Ilustración x - Sprint 3 en Project

5.2.5 Sprint 4 – Entrenamiento CNN

Este sprint implementa el primer pipeline de entrenamiento automatizado. Se desarrolla el script que entrena las arquitecturas convolucionales con validación cruzada de cinco pliegues y balanceo de clases, se integra con el backend para poder lanzar experimentos desde la plataforma y se construye el panel de monitorización del progreso. Es, junto con el sprint siguiente, el corazón del camino crítico del proyecto: sin el pipeline de entrenamiento no existe benchmarking posible. Con 52 horas en cuatro tareas, ocupa la segunda quincena de abril. El asesor de deep learning y XAI revisa la corrección del pipeline y de sus métricas.

Código	Nombre	Responsable(s)	Estimación (h)
4.1	Script de entrenamiento con validación cruzada	Luis Carmona Berdugo, Iván Segura-Carmona	20
4.2	Ajuste de hiperparámetros y primeras ejecuciones	Luis Carmona Berdugo	14
4.3	Integración del entrenamiento en el backend	Luis Carmona Berdugo	8
4.4	Panel de monitorización del progreso	Luis Carmona Berdugo	10

Tabla x - Recursos y estimaciones Sprint 4

Tarea 4.1 – Script de entrenamiento con validación cruzada: se implementa el script que entrena dinámicamente cualquier arquitectura de `tf.keras.applications` con cinco pliegues estratificados y submuestreo aleatorio de la clase mayoritaria. La base preentrenada en ImageNet se mantiene congelada y se sustituye la cabeza de clasificación por una propia, con callbacks de guardado del mejor modelo, parada temprana y reducción de la tasa de aprendizaje. La estimación incluye la resolución de las incompatibilidades entre Keras y `tf_keras`, que consumió más tiempo del previsto. Es la tarea más costosa del sprint y la base de todo el benchmarking posterior.

Tarea 4.2 – Ajuste de hiperparámetros y primeras ejecuciones: se lanzan las primeras ejecuciones sobre arquitecturas ligeras (MobileNetV2, EfficientNetB0) para calibrar el hardware y ajustar el número de épocas, el tamaño de lote y la tasa de aprendizaje. Esta fase acumula experiencia sobre los tiempos reales de entrenamiento y permite estimar con realismo la duración de los sprints siguientes. Es un paso previo imprescindible antes de liberar a las arquitecturas más pesadas.

Tarea 4.3 – Integración del entrenamiento en el backend: se conecta el script de entrenamiento con la API, de modo que un experimento pueda lanzarse desde la plataforma y ejecutarse como un proceso independiente sin bloquear la interfaz.

Tarea 4.4 – Panel de monitorización del progreso: se construye la vista que muestra el estado del entrenamiento en tiempo real, con el progreso por modelo y la consulta del estado de la cola de trabajos. El facultativo investigador puede así seguir la evolución de sus experimentos sin salir de la plataforma.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
4. Entrenamiento CNN	10 días	lun 13/04/26	vie 24/04/26		52 horas	
4.1 Script de entrenamiento con validación cruzada	4 días	lun 13/04/26	jue 16/04/26	Iván Segura Carmona[3%];Luis Carmona Berdugo[97%]	20 horas	26
4.2 Ajuste de hiperparámetros y primeras ejecuciones	2 días	vie 17/04/26	lun 20/04/26	Luis Carmona Berdugo	14 horas	28
4.3 Integración del entrenamiento en el backend	1 día	mar 21/04/26	mar 21/04/26	Luis Carmona Berdugo	8 horas	29
4.4 Panel de monitorización del progreso	3 días	mié 22/04/26	vie 24/04/26	Luis Carmona Berdugo	10 horas	30

Ilustración x - Sprint 4 en Project

5.2.6 Sprint 5 – Transformers y análisis XAI

Este sprint amplía el marco de entrenamiento a las arquitecturas Transformer y desarrolla los módulos de análisis XAI que se ejecutan automáticamente tras cada entrenamiento. Es el segundo bloque del camino crítico del proyecto: el entrenamiento de los Vision Transformers y la evaluación de su explicabilidad condicionan directamente el benchmarking final. Con 62 horas en seis tareas, el sprint se extiende desde finales de abril hasta el 19 de mayo, con el entrenamiento de DeiT, Swin-Base y ViT-384 prolongándose hasta mediados de mayo y la revisión final del asesor realizándose el 18 de mayo. Incluye la implementación de los scripts de análisis cualitativo y cuantitativo, la orquestación automática que garantiza que ningún modelo quede sin su evaluación de explicabilidad y la revisión final del asesor de deep learning y XAI sobre los resultados.

Código	Nombre	Responsable(s)	Estimación (h)
5.1	Script de entrenamiento de arquitecturas Transformer	Luis Carmona Berdugo, Iván Segura-Carmona	16
5.2	Script de explicabilidad cualitativa	Luis Carmona Berdugo	10
5.3	Script de métricas cuantitativas y calibración	Luis Carmona Berdugo, Iván Segura-Carmona	12
5.4	Orquestación automática post-entrenamiento	Luis Carmona Berdugo	6
5.5	Entrenamiento de DeiT, Swin y ViT	Luis Carmona Berdugo	14
5.6	Revisión de resultados con el asesor de deep learning	Luis Carmona Berdugo, Iván Segura-Carmona	4

Tabla x - Recursos y estimaciones Sprint 5

Tarea 5.1 – Script de entrenamiento de arquitecturas Transformer: se implementa el script que entrena DeiT, Swin-Base y ViT-384 con HuggingFace Transformers y validación cruzada de cinco pliegues, reutilizando el esquema de balanceo y de guardado de pesos del pipeline convolucional. La integración con la librería de Transformers obliga a adaptar el manejo de pesos y la tasa de aprendizaje, con ajustes específicos para lograr la convergencia. Es la tarea más exigente del sprint desde el punto de vista técnico. Su correcta ejecución es condición necesaria para el benchmarking final.

Tarea 5.2 – Script de explicabilidad cualitativa: se implementa el script que genera los mapas de explicabilidad visuales sobre imágenes de ejemplo, con Grad-CAM y saliencia para las CNN y saliencia para los Transformers. El resultado es la galería de mapas de cada modelo, que permite la inspección visual de la coherencia de las decisiones.

Tarea 5.3 – Script de métricas cuantitativas y calibración: se implementa el cálculo de las métricas de fidelidad de las explicaciones —Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM— y de la calibración de las predicciones mediante el Expected Calibration Error y el Brier Score. La revisión del asesor de deep learning y XAI valida el cálculo de estas métricas.

Tarea 5.4 – Orquestación automática post-entrenamiento: se automatiza la ejecución de los análisis XAI al finalizar cada entrenamiento, garantizando que ningún modelo quede sin su evaluación de explicabilidad.

Tarea 5.5 – Entrenamiento de DeiT, Swin y ViT: se ejecuta el entrenamiento completo de las tres arquitecturas Transformer sobre el dataset. La duración de esta tarea en el cronograma refleja el coste computacional real de entrenar los Transformers, sensiblemente mayor que el de las CNN. Se aplican los ajustes de tasa de aprendizaje necesarios para lograr la convergencia de los tres modelos.

Tarea 5.6 – Revisión de resultados con el asesor de deep learning: el asesor valida la interpretación de los resultados del entrenamiento y de la explicabilidad de los Transformers.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
➤ 5. Transformers y análisis XAI	17 días	lun 27/04/26	mar 19/05/26		62 horas	
5.1 Script de entrenamiento de arquitecturas Transformer	3 días	lun 27/04/26	mié 29/04/26	Iván Segura Carmona[3%];Luis Carmona Berdugo[97%]	16 horas	31
5.2 Script de explicabilidad cualitativa	3 días	jue 30/04/26	lun 04/05/26	Luis Carmona Berdugo	10 horas	33
5.3 Script de métricas cuantitativas y calibración	3 días	lun 04/05/26	mié 06/05/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	
5.4 Orquestación automática post-entreno	1 día	jue 07/05/26	vie 08/05/26	Luis Carmona Berdugo	6 horas	35
5.5 Entrenamiento de Deit, Swin y ViT	5 días	lun 11/05/26	vie 15/05/26	Luis Carmona Berdugo	14 horas	36
5.6 Revisión de resultados con el asesor de deep learning	1 día	lun 18/05/26	lun 18/05/26	Iván Segura Carmona[25%];Luis Carmona Berdugo[75%]	4 horas	37

Ilustración x - Sprint 5 en Project

5.2.7 Sprint 6 – Validación externa

Este es el sprint más contenido del proyecto, con 26 horas en dos tareas y una duración de cuatro días. Implementa los mecanismos de validación externa y de análisis estadístico que determinan si las diferencias entre modelos son reales o fruto del azar. Se desarrolla en la segunda mitad de mayo.

Código	Nombre	Responsable(s)	Estimación (h)
6.1	Comparación estadística: ranking y test de Wilcoxon	Luis Carmona Berdugo, Iván Segura-Carmona	12
6.2	Validación externa y test de DeLong	Luis Carmona Berdugo	14

Tabla x - Recursos y estimaciones Sprint 6

Tarea 6.1 – Comparación estadística: ranking y test de Wilcoxon: se implementa el script que genera el ranking global de los modelos por su AUC medio y calcula una matriz exploratoria de p-valores por pares. Dado que solo se dispone de cinco folds dependientes, el test de Wilcoxon no se utilizará para afirmar significación confirmatoria; cualquier contraste por pares deberá incorporar una corrección por comparaciones múltiples y sus limitaciones deberán quedar documentadas. Es la tarea más costosa del sprint.

Tarea 6.2 – Validación externa y test de DeLong: se implementa la evaluación de los modelos congelados sobre la cohorte independiente de pacientes adultos y el test de DeLong para comparar las curvas ROC. Esta tarea constituye el examen de generalización del proyecto: sobre sus resultados se sustentan las conclusiones de la memoria. Genera las matrices de significación de la validación externa.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
⚡ 6. Validación externa	4 días	mar 19/05/21	vie 22/05/26		26 horas	
6.1 Comparación estadística: ranking y test de Wilcoxon	2 días	mar 19/05/21	mié 20/05/26	Iván Segura Carmona[8%];Luis Carmona Berdugo[92%]	12 horas	38
6.2 Validación externa y test de DeLong	2 días	jue 21/05/26	vie 22/05/26	Luis Carmona Berdugo	14 horas	40

Ilustración x - Sprint 6 en Project

5.2.8 Sprint 7 – Laboratorio MLOps

El laboratorio MLOps es, junto con la infraestructura, el bloque de mayor carga del proyecto, con 66 horas en siete tareas. Integra el asistente conversacional basado en Groq y Llama 3, que permite al usuario configurar y lanzar experimentos en lenguaje natural sin escribir código: desde la integración de la API de Groq y el diseño del prompt hasta el lanzamiento de los experimentos desde la conversación. El sprint incorpora también la cola de trabajos que garantiza la ejecución asíncrona de diagnósticos, entrenamientos y validaciones externas, la internacionalización de la plataforma en cuatro idiomas y las vistas de resultados con el ranking, las matrices de significación y las curvas ROC. Se desarrolla en la última semana de mayo y la primera de junio y convierte el laboratorio en la puerta de entrada de los experimentos para el usuario investigador. Es, junto con el sprint de infraestructura, el más amplio del cronograma.

Código	Nombre	Responsable(s)	Estimación (h)
7.1	Integración de la API de Groq	Luis Carmona Berdugo	8
7.2	Diseño del prompt del asistente	Luis Carmona Berdugo	12
7.3	Asistente conversacional del laboratorio	Luis Carmona Berdugo	14
7.4	Lanzamiento de experimentos desde el chat	Luis Carmona Berdugo	8
7.5	Cola de trabajos y ejecución asíncrona	Luis Carmona Berdugo	8
7.6	Internacionalización de la plataforma	Luis Carmona Berdugo	8
7.7	Vistas de resultados, rankings y curvas ROC	Luis Carmona Berdugo	8

Tabla x - Recursos y estimaciones Sprint 7

Tarea 7.1 – Integración de la API de Groq: se configura el cliente de Groq y la conexión con el modelo llama-3.3-70b-versatile que alimenta al asistente conversacional del laboratorio.

Tarea 7.2 – Diseño del prompt del asistente: se diseña el prompt de sistema que define al asistente como experto en MLOps médico. Incluye las reglas de extracción de los cinco parámetros del experimento —ruta del dataset, arquitecturas, épocas, lote y tasa de aprendizaje— y el formato JSON de salida. Es una tarea de diseño más que de código, y por eso su estimación es moderada.

Tarea 7.3 – Asistente conversacional del laboratorio: se implementa el endpoint de chat y la sala de conversación, con el historial de mensajes por sesión. El asistente interpreta la configuración devuelta por el modelo y la traduce al panel del experimento, para que el usuario pueda revisarla y confirmarla. Es la tarea más costosa del sprint.

Tarea 7.4 – Lanzamiento de experimentos desde el chat: se conecta la configuración detectada por el asistente con el lanzamiento del pipeline de entrenamiento, de modo que el experimento se encola y arranca sin escribir código.

Tarea 7.5 – Cola de trabajos y ejecución asíncrona: se implementa la cola de trabajos que gestiona los diagnósticos, los entrenamientos y las validaciones externas, con monitorización de estado y cancelación de las tareas pendientes.

Tarea 7.6 – Internacionalización de la plataforma: se incorpora el soporte multilingüe de la interfaz, los informes y el asistente en los cuatro idiomas de la plataforma, mediante atributos de traducción y diccionarios en JavaScript.

Tarea 7.7 – Vistas de resultados, rankings y curvas ROC: se construyen las vistas del laboratorio que muestran el ranking de modelos, las matrices de significación y las curvas ROC de la validación externa.

Nombre de tarea ▾	Duración ▾	Comienzo ▾	Fin ▾	Nombres de los recursos ▾	Trabajo ▾	Predecesoras ▾
🔪 7. Laboratorio MLOps	10 días	lun 25/05/26	vie 05/06/26		66 horas	
7.1 Integración de la API de Groq	1 día	lun 25/05/26	lun 25/05/26	Luis Carmona Berdugo	8 horas	41
7.2 Diseño del prompt del asistente	2 días	mar 26/05/26	mié 27/05/26	Luis Carmona Berdugo	12 horas	43
7.3 Asistente conversacional del laboratorio	2 días	jue 28/05/26	vie 29/05/26	Luis Carmona Berdugo	14 horas	44
7.4 Lanzamiento de experimentos desde el chat	1 día	lun 01/06/26	lun 01/06/26	Luis Carmona Berdugo	8 horas	45
7.5 Cola de trabajos y ejecución asíncrona	1 día	mar 02/06/26	mar 02/06/26	Luis Carmona Berdugo	8 horas	46
7.6 Internacionalización de la plataforma	1 día	mié 03/06/26	mié 03/06/26	Luis Carmona Berdugo	8 horas	47
7.7 Vistas de resultados, rankings y curvas ROC	1 día	jue 04/06/26	jue 04/06/26	Luis Carmona Berdugo	8 horas	48

Ilustración x - Sprint 7 en Project

5.2.9 Sprint 8 – Documentación y cierre

El sprint final abandona el código para concentrarse en el benchmarking definitivo, la redacción de la memoria y de los manuales, las correcciones finales y la entrega. Con 67 horas en cuatro tareas, se desarrolla desde el 5 de junio hasta el 2 de septiembre de 2026, con una dedicación repartida entre la ejecución experimental, la documentación y el cierre. La redacción de la memoria es la tarea más extensa del sprint, mientras que las correcciones finales y la entrega se han separado en tareas específicas. El sprint culmina con la entrega del 2 de septiembre de 2026.

Código	Nombre	Responsable(s)	Estimación (h)
8.1	Benchmarking final con todas las arquitecturas	Luis Carmona Berdugo, Iván Segura-Carmona	18
8.2	Redacción de la memoria y manuales	Luis Carmona Berdugo	30
8.3	Reunión final y correcciones	Luis Carmona Berdugo, Aurelio López Fernández	16
8.4	Entrega final	Luis Carmona Berdugo	3

Tabla x - Recursos y estimaciones Sprint 8

Tarea 8.1 – Benchmarking final con todas las arquitecturas: se ejecuta la evaluación completa del banco de pruebas, consolidando las métricas de rendimiento, calibración y explicabilidad de todas las arquitecturas. El asesor de deep learning y XAI y el asesor de imagen médica revisan la coherencia de los resultados y de los mapas de explicabilidad. Es la tarea técnica más densa del sprint, ya que cierra el proceso experimental del proyecto. Sus conclusiones alimentan directamente los capítulos de resultados y conclusiones de la memoria.

Tarea 8.2 – Redacción de la memoria y manuales: se redacta la memoria completa del Trabajo Fin de Grado, integrando el plan de proyecto, el análisis, el diseño, la implementación, las pruebas y las conclusiones, junto con el manual de usuario orientado a facultativos sin formación técnica. La redacción se realiza de forma incremental a lo largo de la fase de documentación, incorporando la revisión y las correcciones del tutor. Cada capítulo se elabora y se revisa por separado antes de su integración en el documento final. Es la tarea de mayor carga individual del proyecto, y su extensión justifica con holgura las treinta horas estimadas. El manual de usuario se redacta pensando en el profesional sanitario, con un lenguaje no técnico y capturas de los flujos principales.

Tarea 8.3 – Reunión final y cierre: reunión con el tutor y periodo de corrección programados del 28 de agosto al 1 de septiembre. Esta tarea permite revisar el conjunto del trabajo, aplicar las correcciones detectadas, preparar la defensa y dejar la versión final lista antes de la entrega.

Tarea 8.4 – Entrega final: depósito de la memoria y de los entregables del proyecto el 2 de septiembre de 2026. Esta tarea constituye el cierre administrativo posterior a la revisión y corrección final.

Nombre de tarea ▼	Duración ▼	Comienzo ▼	Fin ▼	Nombres de los recursos ▼	Trabajo ▼	Predecesoras ▼
➤ 8. Documentación y cierre	64 días	vie 05/06/26	mié 02/09/26		67 horas	
8.1 Benchmarking final con todas las arquitecturas	10 días	vie 05/06/26	jue 18/06/26	Iván Segura Carmona[5%];Luis Carmona Berdugo[90%];Marc Ríos Cadena[5%]	18 horas	49
8.2 Redacción de la memoria y manuales	50 días	vie 19/06/26	jue 27/08/26	Luis Carmona Berdugo	30 horas	51
8.3 Reunión final y correcciones	3 días	vie 28/08/26	mar 01/09/26	Aurelio López Fernández[50%];Luis Carmona Berdugo[50%]	16 horas	52
8.4 Entrega final	1 día	mié 02/09/26	mié 02/09/26	Luis Carmona Berdugo	3 horas	53

Ilustración x - Sprint 8 en Project

5.3 Recursos y costes del proyecto

Este apartado detalla la asignación de recursos del proyecto, abarcando todo lo necesario para ejecutar las tareas descritas anteriormente. Se establecen dos categorías fundamentales: los recursos de trabajo, centrados en el capital humano, y los recursos materiales, que engloban la infraestructura y el equipamiento tecnológico.

Microsoft Project centraliza la administración del proyecto. Los recursos ocasionales, como el tutor y los asesores, reciben un horario fijo; esto impide que el software reasigne automáticamente su esfuerzo ante variaciones en la duración de las tareas. Todas las asignaciones se han auditado manualmente para que la carga de los recursos laborables sea coherente con la dedicación real de cada fase del proyecto.

Recursos de trabajo:

- **Luis Carmona Berdugo:** asume la carga técnica integral del proyecto. Acumula 455,14 horas distribuidas a lo largo de todos los sprints. Con una tarifa estándar de 20 €/h, su esfuerzo se cuantifica en 9.102,80 €.
- **Aurelio López Fernández (tutor):** interviene estratégicamente en los hitos de apertura (tarea 0.1) y de clausura y correcciones (tarea 8.3). Dedicar 9,5 horas de supervisión a 50 €/h, sumando un total de 475,00 €.
- **Vicente de Vides Rodríguez (consultor de persistencia y bases de datos):** aporta su experiencia estructural en el diseño y la optimización de la persistencia de datos. Participa en la tarea 1.1 con 1,5 horas a 50 €/h, con un total de 75,00 €.
- **Iván Segura Carmona (asesor Deep Learning y XAI):** supervisa el núcleo algorítmico. Valida las implementaciones de explicabilidad, los pipelines CNN y Transformer, la comparación estadística y el benchmarking definitivo. Acumula 5,96 horas a 50 €/h, con un total de 298,00 €.
- **Marc Ríos Cadenas (asesor Imagen Médica):** aporta el rigor clínico. Participa en la revisión de la coherencia de los mapas de explicabilidad y en la validación del benchmarking. Dedicar 1,9 horas a 50 €/h, sumando 95,00 €.

El coste total de los recursos de trabajo asciende, por tanto, a aproximadamente 10.045,80 €.

Nombre del recurso ▼	Trabajo ▼	Costo ▼	Comienzo ▼	Fin ▼
▷ Luis Carmona Berdugo	455,14 horas	9.102,80 €	lun 24/11/25	mié 02/09/26
▷ Aurelio López Fernández	9,5 horas	475,00 €	lun 24/11/25	mar 01/09/26
▷ Vicente de Vides Rodríguez	1,5 horas	75,00 €	lun 02/03/26	mar 03/03/26
▷ Iván Segura Carmona	5,96 horas	298,00 €	mié 01/04/26	jue 18/06/26
▷ Marc Ríos Cadena	1,9 horas	95,00 €	vie 10/04/26	jue 18/06/26

Ilustración x - Vista "Uso de Recursos" en Project

Recursos Materiales:

- **Equipo de desarrollo:** se utiliza un equipo local con GPU NVIDIA compatible con CUDA, disponible para el desarrollo. El plan no especifica el modelo de GPU ni su memoria VRAM, por lo que la viabilidad de las arquitecturas con mayor resolución o capacidad no puede justificarse únicamente desde este apartado. Al tratarse de infraestructura ya disponible, no se imputa una amortización adicional al TFG.
- **Licencias de software:** coste nulo. La plataforma descansa enteramente sobre tecnologías de código abierto (Python, TensorFlow, FastAPI, MySQL), eliminando cualquier gasto de licenciamiento.
- **Servicios cloud:** no se presupuesta ningún servicio cloud en el plan base. La nube aparece únicamente como contingencia ante un fallo del equipo local, pero no existe una reserva económica asignada; si fuera necesario activarla, habría que aprobar un coste adicional o reducir el alcance del benchmarking y documentar la decisión.
- **Electricidad y API de Groq:** no se dispone de una medición o factura imputable exclusivamente al proyecto, por lo que ambos conceptos quedan fuera del presupuesto cuantificado. El consumo energético se reconoce como impacto operativo y el uso de Groq queda sujeto al plan y a las cuotas vigentes del servicio.

El coste económico documentado del proyecto asciende a 10.045,80 €, correspondiente a los recursos humanos presupuestados. Esta cifra no debe interpretarse como un coste completo de explotación: excluye la amortización del equipo ya disponible, la electricidad, una eventual infraestructura cloud y cualquier consumo facturado de la API de Groq.

5.4 Reparto de responsabilidades

Este apartado recoge la distribución de responsabilidades sobre las tareas definidas en el apartado 5.2. Para cada tarea se identifica qué persona asume la titularidad, qué interesados intervienen de forma puntual y en qué proporción. El reparto porcentual de cada responsable sobre cada tarea —recogido en las tablas del apartado 5.2— constituye la base sobre la que se calculan las horas y los costes del apartado 5.3, y se apoya en los roles definidos en la matriz RACI del capítulo de organización. La lógica del reparto es sencilla: el alumno desarrollador concentra la práctica totalidad de la carga de trabajo, mientras que el resto de los interesados participa de forma puntual y consultiva en los hitos donde su conocimiento especializado resulta indispensable.

La responsabilidad absoluta sobre la ejecución del proyecto recae sobre **Luis Carmona Berdugo**. El alumno asume en solitario todo el peso técnico y documental a lo largo de los nueve sprints, acumulando 455,14 horas de trabajo, lo que representa prácticamente la totalidad del esfuerzo planificado. Su figura absorbe íntegramente los roles de analista, diseñador, desarrollador backend y frontend, ingeniero de deep learning, tester y documentador: analiza los requisitos, diseña la arquitectura, implementa tanto el backend como la interfaz, construye los pipelines de entrenamiento y los módulos XAI, ejecuta el benchmarking y redacta toda la documentación del proyecto. Al tratarse de un Trabajo Fin de Grado, el desarrollo carece de un reparto transversal de tareas entre distintos perfiles profesionales, y es precisamente esta concentración la que justifica que el alumno asuma prácticamente el cien por cien de las horas en la mayoría de las tareas. En las tareas en las que intervienen los asesores, su participación sigue siendo mayoritaria, con porcentajes que oscilan entre el 50% de las reuniones y el 97% de las tareas de entrenamiento.

No obstante, la profunda naturaleza multidisciplinar del sistema exige el apoyo puntual y consultivo de varios interesados. Estas figuras aportan su conocimiento especializado y auditan el proceso únicamente en aquellos hitos donde su validación resulta indispensable para garantizar el rigor del proyecto. A continuación se describe de forma pormenorizada la participación exacta de cada uno de ellos.

Aurelio López Fernández asume la figura central de tutor académico y científico del proyecto. Su participación remunerada se concentra en dos hitos concretos del cronograma: la tarea 0.1, correspondiente a la reunión inicial, y la tarea 8.3, correspondiente a la reunión final y al periodo de correcciones. En ambas comparte la dedicación al cincuenta por ciento con el alumno, lo que supone un total de 9,5 horas de supervisión y un coste de 475 €. El seguimiento del resto del ciclo se realiza mediante comunicación y revisiones puntuales dentro de la dedicación académica del proyecto, pero no se computa como dieciocho reuniones formales ni como horas adicionales en el presupuesto.

Iván Segura Carmona actúa como asesor experto en aprendizaje profundo e inteligencia artificial explicable. Su participación, de naturaleza estrictamente consultiva, se distribuye a lo largo de los sprints de implementación técnica, donde valida el trabajo del alumno en los puntos más sensibles del proceso experimental. En el Sprint 3 revisa la implementación de las técnicas XAI (tareas 3.1 y 3.3, con una dedicación del 10% y del 8% respectivamente); en el Sprint 4 inspecciona el pipeline de entrenamiento convolucional (tarea 4.1, 3%); en el Sprint 5 valida el script de entrenamiento de los Transformers y el cálculo de las métricas cuantitativas (tareas 5.1 y 5.3, con un 3% y un 8%) y revisa los resultados junto al alumno (tarea 5.6, 25%); en el Sprint 6 revisa la comparación estadística (tarea 6.1, 8%); y en el Sprint 8 certifica los resultados del benchmarking final (tarea 8.1, 5%). Su dedicación total asciende a 5,96 horas, con un coste de 298 €.

Marc Ríos Cadenas interviene como asesor especialista en imagen médica. Su participación mantiene un perfil consultivo y se concentra en dos hitos que exigen criterio clínico: la revisión de la coherencia de los mapas de explicabilidad en la tarea 3.6, donde comprueba que los mapas apuntan a las regiones pulmonares relevantes y no a artefactos, y la validación de los resultados del benchmarking final en la tarea 8.1, donde certifica la coherencia diagnóstica de las explicaciones frente al criterio médico humano. Su dedicación, del 25% en la tarea 3.6 y del 5% en la tarea 8.1, asciende a 1,9 horas y a un coste de 95 €.

Vicente de Vides Rodríguez actúa como consultor de persistencia y bases de datos. Su intervención es puntual y estrictamente consultiva, concentrándose en la tarea 1.1, correspondiente al diseño del modelo de datos en MySQL. En ella aporta su experiencia estructural y verifica que el modelo de persistencia de la información resulta coherente con las exigencias de almacenamiento del sistema, sin asumir en ningún caso la implementación, que corresponde al alumno. Su dedicación del 15% sobre dicha tarea supone 1,5 horas y un coste de 75 €.

El resto de las partes interesadas identificadas en el capítulo de organización, como el **asesor de ingeniería del software y metodología**, participan únicamente a través de la matriz RACI, sin asumir tareas concretas ni horas asignadas en el cronograma del proyecto. En conjunto, el reparto de responsabilidades refleja una realidad propia de un trabajo académico unipersonal: una carga técnica casi íntegramente asumida por el alumno, complementada por intervenciones puntuales y de alto valor de los asesores, cuyos costes se consolidan en el apartado 5.3.

6 Identificación y Mitigación de Riesgos

Una actividad ineludible dentro de la planificación de cualquier proyecto radica en la gestión exhaustiva de los riesgos. Esta exigencia se multiplica críticamente cuando el sistema incorpora tecnologías de altísima complejidad técnica, como ocurre en este caso con el aprendizaje profundo, la inteligencia artificial explicable y la integración simultánea de múltiples arquitecturas algorítmicas. Se define formalmente como riesgo a cualquier evento latente que, de materializarse, impactaría negativamente sobre el alcance, la planificación temporal, la calidad técnica o el coste global del proyecto. El objetivo de este apartado no persigue la erradicación total de los riesgos, un escenario operativamente imposible en un desarrollo de esta envergadura. La meta consiste en identificar anticipadamente todas las amenazas, evaluando objetivamente su probabilidad y su impacto, para estructurar un plan de actuación sólido que reduzca las posibilidades de fallo y garantice una respuesta protocolizada y ordenada si terminan sucediendo.

Para cada una de las amenazas identificadas, la estrategia de defensa establece dos protocolos diferenciados: el plan de mitigación y el plan de contingencia. El plan de mitigación engloba las acciones estrictamente preventivas que se deben ejecutar antes de que el evento adverso ocurra, buscando asfixiar su probabilidad de aparición o reducir drásticamente su impacto. Por su parte, el plan de contingencia dicta las acciones reactivas y de contención que se despliegan una vez que el riesgo ya se ha materializado de manera inevitable, con el objetivo estricto de minimizar su efecto destructivo sobre el avance del proyecto.

6.1 *Escala de valoración de riesgos*

Antes de exponer el catálogo de amenazas identificadas, resulta imperativo establecer la métrica de evaluación. El sistema define una escala estricta para cuantificar la probabilidad, la severidad y la prioridad de cada riesgo, eliminando así cualquier margen de ambigüedad en su interpretación empírica.

La probabilidad mide la viabilidad de que la amenaza logre materializarse durante el ciclo de vida del proyecto. Se divide en tres umbrales:

- **Baja:** El escenario desencadenante resulta estadísticamente anómalo o se encuentra bajo un control técnico absoluto.
- **Media:** Existe una posibilidad empírica y real de que el evento adverso quiebre las defensas bajo condiciones operativas específicas.
- **Alta:** El vector de fallo representa una vulnerabilidad común, recurrente e inherente a los desarrollos de esta naturaleza tecnológica.

La severidad cuantifica el impacto destructivo sobre el sistema si el riesgo logra ejecutar su amenaza. Se estructuran cuatro niveles de daño:

- **Baja:** Desviación superficial. El ecosistema absorbe el impacto de manera orgánica sin exigir alteraciones significativas.
- **Media:** Alteración notable. El impacto penaliza el tiempo, el coste o la calidad del código, exigiendo maniobras tácticas de corrección para garantizar la continuidad operativa.
- **Alta:** Daño crítico. El evento adverso compromete de forma grave uno o varios pilares fundamentales del proyecto, forzando un replanteamiento estructural o temporal.
- **Muy Alta:** Colapso. La materialización del riesgo amenaza de muerte la viabilidad integral del proyecto o destruye por completo la validez matemática y clínica de los resultados.

Finalmente, la prioridad fusiona ambos vectores (probabilidad y severidad). Esta intersección determina el nivel de alerta y el rigor del blindaje que exige cada amenaza, cristalizando en la matriz de evaluación que se expone en la siguiente tabla:

Probabilidad \ Severidad	Baja	Media	Alta	Muy Alta
Baja	Baja	Baja	Media	Alta
Media	Baja	Media	Alta	Alta
Alta	Media	Alta	Alta	Muy Alta

Tabla x - Matriz de Prioridades

6.2 Catálogo de riesgos identificados

A continuación, se presentan los riesgos identificados para este proyecto, clasificados por su origen, evaluados según su probabilidad de materialización y la severidad de su impacto, y priorizados aplicando la matriz del apartado anterior. Se han detectado un total de nueve riesgos, que abarcan desde la gestión y planificación del proyecto hasta aspectos puramente tecnológicos, incluyendo la incompatibilidad de librerías, la disponibilidad de los conjuntos de datos médicos y el rendimiento de los modelos de aprendizaje profundo.

Id	Origen	Descripción	Probabilidad	Severidad	Prioridad
R01	Gestión	Planificación incorrecta de la duración, terminando el proyecto después de la fecha prevista.	Media	Media	Media
R02	Tecnológico	Incompatibilidades entre versiones de librerías de deep learning (TensorFlow, Keras, Transformers, CUDA Toolkit).	Alta	Alta	Alta
R03	Tecnológico	Dificultad en la integración de los módulos de entrenamiento Python con el backend FastAPI.	Media	Alta	Alta
R04	Gestión	Subestimación del tiempo de entrenamiento de los modelos de deep learning, especialmente en los Sprints de validación cruzada.	Alta	Alta	Alta
R05	Tecnológico	Dataset médico no disponible, inadecuado o con licencia incompatible para las pruebas de validación.	Media	Media	Media
R06	Tecnológico	Bloqueo o degradación de la interfaz web durante la ejecución prolongada de un entrenamiento.	Baja	Media	Baja
R07	Externo	Pérdida de código fuente por fallo del sistema de archivos o del equipo de desarrollo.	Baja	Muy Alta	Alta
R08	Infraestructura	Sobrecalentamiento o agotamiento de recursos de la GPU durante entrenamientos prolongados, provocando interrupciones.	Media	Alta	Alta
R09	Tecnológico	Falta de convergencia o rendimiento insuficiente de los modelos entrenados, invalidando los resultados del benchmarking.	Media	Alta	Alta

Tabla x - Riesgos Identificados

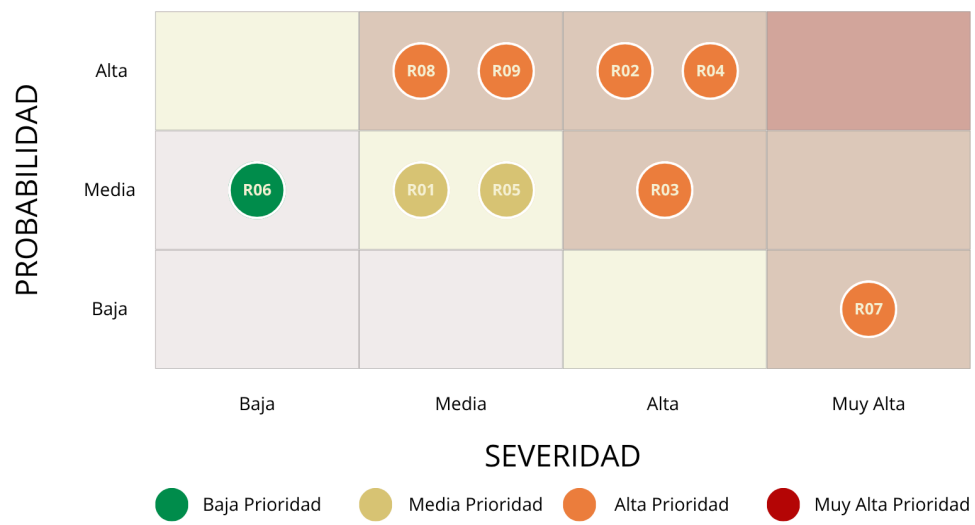


Ilustración x - Mapa de Prioridad

6.3 Plan de mitigación (medidas preventivas)

El plan de mitigación orquesta la barrera defensiva del proyecto. Reúne las medidas preventivas exigidas antes de que la amenaza logre materializarse, con el objetivo innegociable de asfixiar su probabilidad de ocurrencia o amortiguar drásticamente su impacto estructural.

Frente al riesgo de una planificación incorrecta (R01), el sistema impone hitos de control estrictos. El cierre de cada Sprint exige una revisión con el tutor para auditar el progreso real frente al teórico. Si el algoritmo de gestión detecta una desviación acumulada superior al quince por ciento, el cronograma sufre una reestructuración táctica inmediata antes de autorizar el avance hacia la siguiente iteración, garantizando un ajuste realista de la carga pendiente.

Para neutralizar las incompatibilidades entre librerías de aprendizaje profundo (R02), el código ancla las versiones exactas de todas las dependencias desde el minuto cero. El despliegue exige entornos virtuales de Anaconda herméticamente aislados para evitar la contaminación del sistema anfitrión. Adicionalmente, el Sprint 1 ejecuta pruebas de estrés de compatibilidad antes de permitir la construcción del motor de inferencia, erradicando los conflictos desde la base.

Ante la dificultad de integrar los módulos de Python con el backend FastAPI (R03), la arquitectura del Sprint 1 impone una separación quirúrgica entre la capa web y la computación científica. Los algoritmos se invocan como procesos autónomos. El Sprint 4 arranca con una prueba de concepto implacable, validando la fluidez de las comunicaciones entre la API y los scripts de entrenamiento antes de autorizar el desarrollo del resto de los endpoints.

Para combatir la subestimación del tiempo de entrenamiento (R04), el calendario impone una escalada táctica. El motor computa primero las arquitecturas más ligeras (MobileNetV2, EfficientNetB0) para calibrar el hardware, antes de liberar a los colosos pesados (ConvNeXt-Tiny, Transformer). El final del Sprint 4 marca un punto de control ineludible para recalcular las proyecciones temporales futuras basándose exclusivamente en el rendimiento empírico del silicio.

Respecto a la potencial indisponibilidad del conjunto de datos médicos (R05), el protocolo exige la extracción y descarga local de las radiografías durante el Sprint 1. El sistema audita el formato, la integridad estructural y los permisos de licencia. Mantener el volumen de datos anclado en el disco duro local blindo al proyecto frente a caídas de red o alteraciones imprevistas en los repositorios externos durante todo el ciclo de desarrollo.

Para impedir el bloqueo de la interfaz durante los masivos ciclos de cálculo (R06), el diseño estructural asume un paradigma estrictamente asíncrono. FastAPI delega el procesamiento pesado a tareas en segundo plano. El sistema inyecta mecanismos de sondeo continuo y despliega telemetría de progreso en tiempo real sobre el panel de control, desterrando cualquier riesgo de congelación gráfica desde la fase de diseño.

Frente a la amenaza catastrófica de pérdida de código fuente (R07), el protocolo despliega un repositorio privado en GitHub desde el primer día. La normativa impone la ejecución de un commit descriptivo al clausurar cada jornada de trabajo, sin excepciones. Esta red de seguridad se refuerza mediante el volcado sistemático de una copia de seguridad semanal en un hardware de almacenamiento externo.

Para evitar el colapso térmico o el estrangulamiento de la GPU (R08), el motor de ejecución inyecta periodos de latencia programados entre las distintas arquitecturas para disipar el calor del hardware. Herramientas de telemetría auditan la temperatura del procesador gráfico en tiempo real. Las arquitecturas más exigentes quedan relegadas a ventanas temporales de ejecución bajo supervisión humana estricta y activa.

Finalmente, para mitigar el fracaso en la convergencia matemática de los modelos (R09), los sprints de entrenamiento despliegan un sistema de alerta temprana: el Sprint 4 para las arquitecturas CNN y el Sprint 5 para los Transformers. El código monitoriza las curvas de pérdida desde las primeras épocas y utiliza parada temprana y reducción de la tasa de aprendizaje cuando detecta estancamiento. Como línea de base, los algoritmos arrancan con configuraciones de hiperparámetros documentadas y revisadas durante las primeras ejecuciones, que se ajustan cuando los resultados experimentales evidencian problemas de convergencia.

6.4 Plan de contingencia (medidas correctivas)

El plan de contingencia actúa como la última línea de defensa. Despliega maniobras reactivas y de contención una vez que la amenaza ha logrado fracturar las medidas preventivas, con el objetivo innegociable de minimizar el daño estructural y garantizar la supervivencia del proyecto.

R01 – Desviación temporal consolidada: Si el cronograma colapsa, el sistema aplica una poda táctica. Se ejecuta una revisión inmediata de la ruta crítica, amputando el alcance de los elementos no esenciales (variantes accesorias de Transformer o florituras gráficas de la interfaz). El objetivo es salvaguardar la entrega de un núcleo plenamente funcional. Esta reducción del alcance se documentará y justificará explícitamente en la memoria.

R02 – Conflicto estructural de librerías: En el caso de que se produzca un conflicto de incompatibilidad entre versiones de librerías (R02), el protocolo de actuación se estructura en tres niveles. En primer lugar, se aísla la dependencia conflictiva y se consulta la documentación oficial de las librerías implicadas para identificar la combinación de versiones compatible, estableciendo un plazo máximo de cuarenta y ocho horas para localizar la solución. En segundo lugar, la resolución del conflicto se apoya en el uso de entornos virtuales de Anaconda: se crea un entorno conda aislado en el que se instalan las versiones exactas de las dependencias, de modo que la combinación compatible se despliega y valida en ese entorno sin contaminar el resto de instalaciones de la máquina anfitriona, garantizando además la reproducibilidad de los experimentos. En tercer lugar, el despliegue del sistema se apoya en la estrategia de exposición del servicio: vitalXAI se publica a través de un túnel de Cloudflare que expone el servicio alojado en el servidor local, de modo que el entorno de ejecución (versiones de librerías, configuración del toolkit de CUDA y entorno Python) queda confinado y validado en la máquina de despliegue antes de que el servicio se publique a través del túnel. Este diseño garantiza que la versión del sistema que llega a los usuarios se ejecute siempre sobre un entorno controlado y coherente, permitiendo corregir y verificar cualquier incompatibilidad de versiones en la máquina anfitriona con total trazabilidad antes de exponer el servicio.

R03 – Fractura en la integración del backend: Si el acoplamiento orgánico fracasa, el diseño adopta una arquitectura de trinchera. Los scripts de entrenamiento se encapsulan herméticamente como procesos independientes, invocados desde FastAPI mediante el módulo de subprocessos (subprocess). Esta cirugía informática desacopla por completo la carga computacional de la petición web, permitiendo validar la comunicación capa por capa.

R04 – Estrangulamiento del tiempo computacional: Si el silicio no cumple las proyecciones de velocidad, el alcance del benchmarking sufre un recorte quirúrgico. La evaluación masiva se limitará a un subconjunto representativo (por ejemplo, dos arquitecturas CNN y un Transformer), salvaguardando la demostración de viabilidad del concepto. Las arquitecturas restantes quedarán relegadas a trabajo futuro, dejando constancia de esta limitación técnica en la memoria.

R05 – Incompatibilidad sobrevenida del dataset: Ante la corrupción o inutilidad de los datos anclados en las pruebas de validación, la contingencia no puede consistir en un cambio automático de conjunto de datos. Sustituir una cohorte médica por otra altera el protocolo clínico y de validación del estudio: un conjunto de datos distinto introduce variaciones en la población, el equipo de adquisición y los criterios de etiquetado, por lo que cualquier resultado obtenido sin volver a ejecutar el protocolo completo carecería de validez científica. Por ello, la actuación consiste en identificar un conjunto de datos alternativo que cumpla los requisitos del estudio —disponibilidad, anonimización, licencia compatible y coherencia con el problema clínico— y, en caso de adoptarse, reejecutar de forma íntegra el pipeline de entrenamiento, la validación cruzada, la validación externa y los análisis de explicabilidad sobre la nueva cohorte, con la trazabilidad y la reproducibilidad exigidas en la metodología. Cualquier cambio de dataset y sus implicaciones sobre la interpretación de los resultados se documentarán de forma explícita y rigurosa en la memoria, de modo que el estudio no presente resultados que no provengan de un protocolo íntegramente validado.

R06 – Congelación de la interfaz gráfica: Si la sobrecarga algorítmica logra bloquear el frontend durante las pruebas de integración, la implementación de colas asíncronas asume máxima e inmediata prioridad. Si el reloj del Sprint impide una solución estructural definitiva, el código inyectará un parche preventivo: un panel de advertencia informando al clínico sobre el tiempo estimado de espera y habilitando la consulta del estado en diferido.

R07 – Destrucción física del código fuente: Ante un fallo catastrófico del hardware del equipo, el protocolo de recuperación extrae de urgencia el último estado seguro desde el repositorio privado en GitHub hacia una máquina de soporte. Tras restablecer el entorno de trabajo, se cuantifica el volumen del código evaporado y se reestructura internamente el Sprint en curso para absorber el impacto sin alterar la fecha de entrega.

R08 – Colapso térmico de la GPU: El sistema ordena una interrupción instantánea del entrenamiento para purgar el calor del procesador gráfico y salvar su integridad. Los ciclos computacionales masivos se fragmentarán en bloques más pequeños con pausas de refrigeración obligatorias. Si la asfixia del hardware persiste, el algoritmo ejecutará un recorte drástico sobre el tamaño de los lotes (batch size) o el límite de épocas.

R09 – Fracaso en la convergencia matemática: El algoritmo entra en cuarentena investigativa. El desarrollador audita de forma sistemática las tasas de aprendizaje, las capas de preprocesamiento y la hermeticidad de las fronteras de validación. Si el modelo se niega a aprender tras múltiples ajustes, será reemplazado por una arquitectura de solvencia probada en la literatura. Este colapso predictivo se documentará como un hallazgo científico de alto valor sobre la aplicabilidad de dicho modelo en el dominio médico.

7 Planes de apoyo a la gestión del proyecto

Este capítulo reúne los planes de apoyo que complementan el marco de gestión descrito en el capítulo 4. Más allá de la metodología y de la organización del trabajo, un proyecto necesita mecanismos concretos que ordenen tres dimensiones transversales: cómo se comunica el equipo, cómo se garantiza la calidad del producto y del proceso, y cómo se gestiona el código fuente y sus entregables. El capítulo se estructura en tres apartados que desarrollan cada una de estas dimensiones: el plan de comunicación, el plan de calidad y el plan de gestión de la configuración.

7.1 Plan de Comunicación

La comunicación del proyecto se ancla en la cadencia adaptada del marco Scrum y en las particularidades de un desarrollo unipersonal. La planificación y la revisión de cada iteración se realizan mediante el tablero y reuniones de seguimiento programadas según la fase y la disponibilidad del tutor, no mediante dieciocho ceremonias formales con dedicación presupuestada. En la práctica, esta cadencia se materializa en un par de reuniones durante la planificación inicial, reuniones más frecuentes durante el desarrollo activo y reuniones puntuales durante la fase de documentación. Las horas imputadas al tutor se concentran en las tareas 0.1 y 8.3; el resto de las comunicaciones y revisiones puntuales no se contabiliza como una dedicación adicional en el apartado de costes.

Al margen de estos hitos formales, la sincronización operativa fluye de forma asíncrona mediante el tablero Kanban, que proyecta en tiempo real el estado del desarrollo, y el correo electrónico para la gestión logística. Las consultas técnicas con los asesores exigen reuniones quirúrgicas, convocadas exclusivamente cuando la complejidad del avance lo demande, destacando las fases críticas de integración de módulos XAI y la construcción de los pipelines algorítmicos.

7.2 Plan de Calidad

El aseguramiento de la calidad del sistema se blindo mediante cuatro líneas de actuación innegociables.

A nivel de testing, el código se somete a pruebas unitarias sobre los módulos de procesamiento y predicción; pruebas de integración sobre el flujo diagnóstico completo (ingesta de imagen, inferencia, síntesis XAI y exportación PDF); pruebas de seguridad sobre las barreras de autenticación y acceso; y pruebas de rendimiento para auditar que la carga computacional de los entrenamientos respeta los márgenes temporales exigidos. El desarrollo se apoya además en el modelo de desarrollo basado en especificaciones y guiado por pruebas (SDD/TDD) descrito en el capítulo 4, de modo que cada funcionalidad queda cubierta por pruebas automatizadas antes de considerarse completada.

Respecto a los estándares estructurales, todo el ecosistema Python acata implacablemente la normativa PEP 8, auditada mediante herramientas de análisis estático. Los scripts de entrenamiento y el backend imponen una separación arquitectónica estricta, blindando sus funciones con docstrings que documentan su propósito operativo, parámetros de entrada y tipología de retorno.

En el vector de la automatización, el repositorio despliega un flujo de integración continua (CI) impulsado por GitHub Actions. Este motor intercepta cada Pull Request para ejecutar de manera automática los tests con pytest, auditar el estilo con ruff y comprobar que la cobertura del código no desciende por debajo del umbral mínimo establecido, bloqueando inexorablemente cualquier código defectuoso antes de que logre fusionarse con la rama principal.

Finalmente, para mejorar la reproducibilidad de los experimentos frente a la variabilidad debida a la inicialización aleatoria, el código fija semillas en las librerías críticas (TensorFlow, NumPy y Python) y almacena las configuraciones de cada experimento en archivos JSON vinculados a la sesión de entrenamiento. Estas medidas facilitan la repetición y la trazabilidad de los resultados, pero no garantizan por sí solas un determinismo bit a bit: las operaciones concurrentes de la GPU, las versiones del software y otros factores del entorno pueden introducir pequeñas diferencias entre ejecuciones. Por ello, los resultados deben reproducirse bajo las mismas condiciones documentadas y compararse atendiendo a la variabilidad observada.

7.3 Plan de Gestión de la Configuración

El código fuente del proyecto se confina en un repositorio privado alojado en GitHub. La arquitectura de control de versiones adopta el estándar GitHub Flow. Este paradigma protege una rama principal (main) que custodia exclusivamente código estable y desplegable. Cada nueva funcionalidad exige la creación de una rama efímera con nomenclatura descriptiva (por ejemplo, feature/xai-quantitative-metrics o feature/training-pipeline-cnn). La inyección de código hacia main se ejecuta obligatoriamente mediante Pull Requests, incluso operando en solitario, forzando así el disparo automático de las validaciones de GitHub Actions previas a la integración. Como red de seguridad adicional, se realiza un commit descriptivo al final de cada jornada de trabajo y se mantiene una copia de seguridad semanal del repositorio en un almacenamiento externo, en línea con lo establecido en el plan de riesgos del proyecto.

El repositorio integra un archivo .gitignore configurado quirúrgicamente para vetar el rastreo de elementos ajenos al código fuente: los conjuntos de datos radiológicos, los pesos sinápticos de los modelos entrenados (archivos .keras, .h5), los artefactos dinámicos generados (radiografías subidas, PDFs, mapas de calor) y los residuos del entorno de desarrollo (cachés de Python, entornos virtuales, configuraciones locales). Esta barrera mantiene el repositorio ágil y hermético, bloqueando la exposición de datos sensibles y garantizando que solo se almacene lo necesario para replicar el ecosistema.

El versionado del software acata la norma Semantic Versioning (SemVer), estructurando las entregas bajo la sintaxis MAJOR.MINOR.PATCH. El cierre de cada Sprint cristaliza en la generación de etiquetas de Git (por ejemplo, v1.3.0), permitiendo identificar de forma unívoca y auditable el estado exacto del código en cada hito temporal de la planificación.

8 Cuestiones abiertas y decisiones adoptadas

A fecha de elaboración del presente documento, los debates técnicos han quedado definitivamente cerrados. Tras la evaluación empírica de las alternativas, se exponen a continuación las decisiones consolidadas que fijan el rumbo innegociable del proyecto:

Dataset de entrenamiento: Este documento concluye el tema. El sistema utilizará el conjunto de datos público de radiografías de tórax pediátricas publicado por Kermay y colaboradores, procedente del Guangzhou Women and Children's Medical Center y disponible en Kaggle. Este dataset contiene 5.856 radiografías en formato JPEG, clasificadas en dos categorías: NORMAL (1.583 imágenes) y PNEUMONIA (4.273 imágenes), estas últimas subdivididas en neumonía bacteriana y viral. Se ha seleccionado este conjunto por ser el mismo utilizado en el estudio de referencia (Kermay & al., 2018), lo que permite contextualizar los resultados del proyecto respecto a un trabajo publicado que emplea los mismos datos, sin asumir que ambos estudios utilizan el mismo protocolo experimental o comparan las mismas arquitecturas. El desbalanceo natural entre clases se abordará mediante la técnica de submuestreo aleatorio en cada pliegue de la validación cruzada, garantizando particiones balanceadas. Durante la fase de desarrollo y depuración del código se han empleado subconjuntos reducidos de este dataset para agilizar las pruebas, pero la metodología del proyecto está concebida para trabajar con el conjunto original completo.

Dataset de validación externa: Este documento concluye el tema. Se empleará el conjunto de datos COVID-19 Radiography Database, desarrollado por investigadores de la Universidad de Qatar y la Universidad de Dhaka, que contiene radiografías de tórax de pacientes adultos. Para la validación externa se extraerán 500 imágenes de pacientes sanos y 500 imágenes de pacientes con neumonía viral, garantizando un balance perfecto entre clases. Este dataset ha sido seleccionado porque representa una población diferente (adultos frente a niños), con equipos de adquisición y protocolos de imagen distintos, lo que constituye un escenario exigente para evaluar la capacidad de generalización de los modelos. Al igual que con el dataset de entrenamiento, durante las pruebas de desarrollo se han utilizado subconjuntos reducidos del conjunto externo.

Arquitecturas de deep learning: Este documento concluye el tema. El banco de pruebas estará compuesto por dieciséis arquitecturas convolucionales (ResNet50, ResNet101, ResNet152, ResNet50V2, EfficientNetB0, EfficientNetB3, EfficientNetB7, EfficientNetV2S, DenseNet121, DenseNet201, MobileNetV2, MobileNetV3Large, VGG16, InceptionV3, Xception y ConvNeXtTiny) y tres arquitecturas Transformer (DeiT, Swin-Base y ViT-384). Esta selección cubre un espectro representativo de enfoques arquitectónicos, desde modelos ligeros como MobileNetV2 hasta modelos de alta capacidad como ConvNeXt-Tiny y los Vision Transformers, y permite comparar de forma sistemática el paradigma de las redes convolucionales con el de los mecanismos de atención.

Técnicas XAI: Este documento concluye el tema. Se implementarán cuatro técnicas de inteligencia artificial explicable: Saliency Maps, SmoothGrad y Grad-CAM para arquitecturas convolucionales, y mapas de atención para arquitecturas Transformer. Para la evaluación cuantitativa de las explicaciones se calcularán las métricas Deletion AUC, Insertion AUC, Sparsity, Entropy y Stability SSIM. Adicionalmente, se incorporará la métrica Expected Calibration Error (ECE) para evaluar la calibración de las predicciones. Las cuatro técnicas visuales están disponibles en el módulo de explicabilidad de la vía de diagnóstico de la aplicación web. En la vía de laboratorio MLOps, los scripts generan los mapas cualitativos previstos para el análisis experimental —Grad-CAM y mapas de saliencia para las CNN, y mapas de saliencia para los Transformers— y calculan las métricas cuantitativas y de calibración.

Tecnologías del sistema: Este documento concluye el tema. La pila tecnológica queda definida en detalle en este apartado, estructurándola por capas, con el fin de justificar cada elección sobre la base de las tecnologías realmente empleadas en el desarrollo.

En cuanto al **backend**, se ha optado por Python 3.11 como lenguaje principal, dado su ecosistema maduro en el ámbito del aprendizaje profundo y su amplia adopción en la comunidad de investigación médica. Sobre él se asienta el framework FastAPI, un marco ASGI moderno y de alto rendimiento que aprovecha las anotaciones de tipos de Python para generar de forma automática la validación de los datos de entrada y la serialización de las respuestas en JSON mediante Pydantic (versión 2). El servidor se ejecuta bajo Uvicorn, el servidor ASGI de referencia, que atiende las peticiones de forma asíncrona y eficiente. El renderizado de las páginas web se realiza en el servidor con Jinja2, el motor de plantillas estándar de Python, que permite componer las páginas HTML de forma dinámica incorporando los datos del sistema. La capa de persistencia se apoya en la base de datos relacional MySQL (MariaDB en el entorno de desarrollo mediante XAMPP), accedida a través del conector nativo mysql-connector-python, que gestiona un pool de conexiones para soportar el acceso concurrente de múltiples usuarios. El esquema relacional almacena los usuarios, las consultas de diagnóstico, las sesiones de entrenamiento, la cola de trabajos y los tokens de refresco de sesión.

En cuanto a la **seguridad**, el sistema incorpora una capa de protección transversal. Las contraseñas se almacenan aplicando el algoritmo de hash bcrypt, que incluye un factor de coste configurable. La autenticación se gestiona mediante tokens JWT (generados con la librería python-jose, algoritmo HS256) almacenados en cookies httpOnly, complementados con tokens de refresco opacos persistidos en la base de datos y sometidos a rotación en cada uso, con detección de su reutilización para revocar la sesión. Las peticiones que modifican estado están protegidas por un middleware de CSRF de doble cookie. Se aplica además limitación de peticiones con la librería slowapi (un límite general y un límite específico y más estricto sobre el inicio de sesión) y un middleware que inyecta cabeceras de seguridad (Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, entre otras).

En cuanto al **frontend**, la interfaz se construye con HTML5, CSS y JavaScript vanilla, descartándose de forma deliberada los frameworks de componentes como React, Vue o Angular. La decisión no ignora las ventajas de estos frameworks, sino que responde a la adecuación al proyecto: la aplicación no requiere un estado de interfaz altamente reactivo ni componentes complejos, y el renderizado en el servidor con Jinja2 permite entregar una interfaz accesible y mantenible sin introducir una cadena de herramientas de compilación (npm, webpack o similares) que añadiría complejidad operativa. Las páginas de la aplicación —login, registro, panel de diagnóstico, laboratorio de entrenamiento y panel de administración— se componen en el servidor a partir de plantillas Jinja2 que inyectan dinámicamente los datos del sistema y los textos de la interfaz. El estilo visual se apoya en Tailwind CSS, cargado mediante CDN y configurado con soporte de modo oscuro mediante la estrategia de clase (`darkMode: 'class'`), lo que permite alternar entre el tema claro y el oscuro de forma dinámica, y en la librería de iconos Font Awesome (versión 6.4), también cargada por CDN. El uso directo de las utilidades de Tailwind en las plantillas, junto con un pequeño conjunto de estilos propios, elimina la necesidad de escribir hojas de estilo extensas y mantiene la coherencia visual entre las distintas pantallas, incluyendo las tablas de resultados, los gráficos y los visores de imágenes.

La **interactividad** se implementa en JavaScript nativo (ES6), organizado en módulos por ámbito funcional: `dashboard.js` para el panel de diagnóstico, `training.js` para el laboratorio de entrenamiento, `admin.js` para la administración e `i18n.js`, compartido, para la internacionalización. La comunicación con la API se realiza mediante `fetch`, enviando los formularios de forma asíncrona con `FormData` para no recargar la página; un interceptor global de peticiones añade automáticamente la cabecera de protección CSRF (X-CSRF-Token) y gestiona la renovación automática de la sesión cuando el token de acceso expira, reintentando la petición original. El estado de los procesos en segundo plano se actualiza mediante polling periódico del panel de la cola de trabajos, y las imágenes de resultados pueden inspeccionarse a tamaño completo mediante un visor modal. La internacionalización se apoya en atributos `data-i18n` distribuidos en las plantillas y en un diccionario de traducciones en JavaScript que cubre cuatro idiomas (español, inglés, chino e hindú), de modo que al cambiar el idioma la interfaz se re-renderiza al instante, incluyendo los informes y el asistente conversacional.

En cuanto al **motor de inteligencia artificial**, el proyecto utiliza TensorFlow (versión 2.18) y Keras (versión 3) para la construcción, el entrenamiento y la inferencia de las arquitecturas convolucionales. Las arquitecturas se cargan dinámicamente desde `tf.keras.applications` con pesos preentrenados en ImageNet, aplicando transferencia de conocimiento: la base del modelo se mantiene congelada y se sustituye el cabezal de clasificación por uno propio adaptado al problema binario de la neumonía. La librería `tf_keras` se incorpora para garantizar la compatibilidad con los pesos guardados en formatos heredados de Keras. Los modelos Transformer (DeiT, Swin-Base y ViT-384) se integran mediante HuggingFace Transformers, empleando la clase `TFAutoModelForImageClassification` para su entrenamiento y carga en el backend. El preprocesamiento de las imágenes se realiza con OpenCV y con las utilidades de datos de TensorFlow.

En cuanto al **análisis de datos y a la generación de informes**, el sistema utiliza pandas y NumPy para la gestión y el procesamiento de los datos, scikit-learn para las métricas de rendimiento, la validación cruzada estratificada y las curvas ROC, y SciPy para el test estadístico de Wilcoxon; el test de DeLong para la comparación de curvas ROC se implementa de forma propia en el pipeline de validación externa. Las representaciones gráficas —curvas ROC, matrices de calor de significación, rankings y mapas de explicabilidad— se generan con matplotlib y seaborn. Los informes descargables en formato PDF se generan con la librería fpdf2, tanto para los diagnósticos individuales como para los informes completos de las sesiones de entrenamiento.

En cuanto a la **inteligencia conversacional**, la plataforma se conecta a la API de Groq, delegando la inferencia lingüística al modelo Llama 3 de Meta (concretamente la variante llama-3.3-70b-versatile). El asistente conversacional se configura mediante técnicas de prompt engineering, definiendo un prompt de sistema que fuerza la devolución de una salida JSON estructurada con la configuración de los experimentos, y se encuentra internacionalizado en los cuatro idiomas de la plataforma. La elección de Groq responde a la velocidad de inferencia de sus unidades de procesamiento de lenguaje (LPU) y a su disponibilidad gratuita, adecuada para un proyecto académico.

Por último, en cuanto al **despliegue**, el sistema se publica exponiendo el servidor local a través de un túnel de Cloudflare, de modo que el entorno de ejecución queda confinado y validado en la máquina anfitriona antes de que el servicio se ponga a disposición de los usuarios a través del túnel, garantizando la coherencia del entorno de librerías en ejecución.

9 Consideraciones complementarias del proyecto

Las consideraciones éticas y de privacidad de los datos médicos es uno de los aspectos que se debe tener en cuenta, dado que el proyecto utiliza radiografías de tórax de origen humano para el entrenamiento y evaluación de los modelos de inteligencia artificial. Se ha adoptado una política estricta de uso de datos que garantiza el cumplimiento de la normativa vigente.

Todos los datasets que se han utilizado para el entrenamiento y la validación externa proceden únicamente de repositorios públicos y anonimizados. El dataset de Kermany y colaboradores, utilizado para el entrenamiento, fue originalmente obtenido del Guangzhou Women and Children's Medical Center y posteriormente publicado en Kaggle con licencia open data, con todas las imágenes desprovistas de cualquier información de identificación personal. El dataset COVID-19 Radiography Database, utilizado para la validación externa, ha sido igualmente anonimizado por sus autores antes de su publicación. Este enfoque garantiza el cumplimiento del Reglamento General de Protección de Datos (RGPD) y evita cualquier implicación ética derivada del manejo de historiales clínicos o datos sensibles de pacientes.

El sistema desarrollado no almacena ni procesa datos de pacientes individuales identificables. Las radiografías que los facultativos suben a la plataforma para su diagnóstico se almacenan de manera temporal en el servidor únicamente para su procesamiento, y los archivos generados (mapas XAI, informes PDF) quedan asociados a la cuenta del profesional que realizó la consulta, sin exponer información del paciente en la interfaz. Es responsabilidad del personal clínico garantizar que las imágenes que introducen en el sistema han sido previamente anonimizadas y que cumplen con la normativa de protección de datos de su centro sanitario.

Es importante señalar que vitalXAI se concibe como una herramienta de ayuda al diagnóstico, no como un sustituto del criterio clínico. Las predicciones generadas por los modelos de inteligencia artificial deben ser siempre evaluadas por un facultativo cualificado antes de tomar cualquier decisión clínica. El sistema proporciona mapas de explicabilidad para facilitar esta evaluación, pero la responsabilidad última del diagnóstico recae en el profesional sanitario. Esta limitación se indica de manera explícita en la interfaz de la plataforma y en los informes PDF generados.

Por último, desde el punto de vista medioambiental, el entrenamiento de modelos de deep learning conlleva un consumo energético que debe ser considerado. En este proyecto, los entrenamientos se han realizado preferentemente en un equipo local con GPU, limitando el número de épocas y modelos a los estrictamente necesarios para obtener conclusiones significativas. La preferencia por la computación local responde a criterios de coste y de disponibilidad de la infraestructura. No se ha imputado al presupuesto una medición específica de electricidad ni existe una reserva económica para cloud; si el hardware local dejara de estar disponible, el uso de infraestructura en la nube requeriría una decisión adicional de coste o de alcance y quedaría documentado explícitamente. En cualquier caso, la contención en el número de experimentos y en su coste computacional contribuye a reducir la huella de carbono asociada al desarrollo del sistema.